

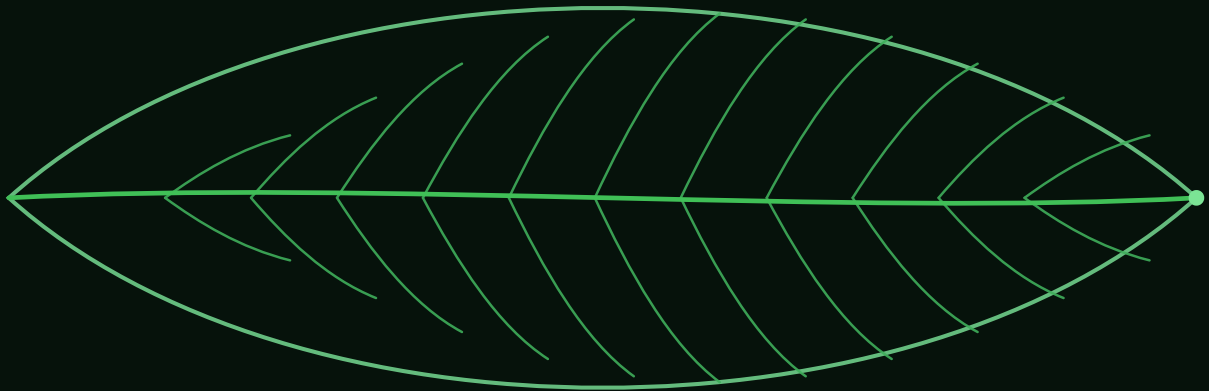


LEAFY

DOCUMENTATION

PROJECT PLAN AND DOCUMENTATION

A plant doctor in your pocket. Snap a leaf, understand the disease, know what to do next.



STACK

Next.js 15, FastAPI, PostgreSQL, MinIO

STATUS

App built. Real ML model pending.

BRAND

Josefin Sans, Poppins, Manrope

BUILT

4 October 2026

Leafy grows out of DAHON. Dahon means leaf in Filipino.

NAVIGATE

CONTENTS

Every line is a link. Bookmarks in your PDF viewer follow the same order.

01	Overview	05	03	App flows (continued)	
	Goals	05		Flow 3b: forgot and reset password	20
	Stack at a glance	05		Flow 4: scan a leaf	21
	Where the project stands	06		Flow 5: history and delete with undo	22
	Confirmed decisions	06		Flow 6: handbook browse	23
	Scope	07		Flow 7: dashboard	24
	Out of scope	08		Flow 8: account	25
	What closes the four open items	08			
02	System architecture	09	04	Scan states and progress	26
	Four tiers	09		States and stages	26
	How to read the diagrams	09		Transitions	26
	System architecture diagram	10		Scan state machine	27
	FastAPI layers	11		Progress stepper, frame by frame	28
	Calls in the architecture diagram	12		Honest progress	29
				Prediction resolver	29
02	System architecture (continued)			Behaviors that keep users informed	29
	What runs in docker compose	13	05	Auth and session sequences	30
	Session cookies	13		Tokens and lifetimes	30
	Key settings in .env.example	13		How to read the sequence diagrams	30
	Local services without Docker	14		Rules the sequences follow	31
03	App flows	15		Sequences in this section	31
	Guests and members	15		Sequence: login with cookies	32
	The flows in this section	15		Sequence: refresh rotation and reuse	33
	Flow 1: first visit	16		Sequence: Google sign in, part 1	34
	Flow 2: sign up and verify email	17			
	Flow 2b: Google sign in	18			
	Flow 3: sign in, session expiry and sign out	19			

Continued on next page

Contents (continued)

05 Auth and session sequences

(continued)

Sequence: Google sign in, part 2	35
Sequence: email verification token, part 1	36
Sequence: email verification token, part 2	37
Sequence: password reset token, part 1	38
Sequence: password reset token, part 2	39
Sequence: account delete, part 1	40
Sequence: account delete, part 2	41
06 REST API	42
Conventions	42
Next.js route handlers	42
Auth endpoints	43
User endpoints	44
Catalog and health endpoints	44
Scan endpoints	45
Error codes	46
Rate limits	47
07 Database	48
Table map	48
Deletion rules	48
Entity relationship diagram	49
Catalog	50
Catalog tables, continued	52
07 Database (continued)	
Accounts and tokens	53
Refresh tokens and outbox	54
Scans and feedback	55
Foreign keys at a glance	56
Migrations	56

08 Design and branding

57

Brand foundation	57
Brand assets and where they live	57
Logo family	58
Minimum size	58
Brand in use	59
Colour tokens	60

08 Design and branding (continued)

Contrast ratios	61
Severity scale	62
Type specimens	62
Type scale	63
Paragraph and title rules	63
More text rules	64

08 Design and branding (continued)

Spacing	64
Custom cursor	65
Component notes	65
Shape and surface	65
Components	66

09 UI behavior

67

Sideways panels	67
Panel content budget	67
Panel navigation	68
Where sideways panels are used	69
Cursor reactions	69
Hover interactions	70
Safeguards for cursor effects	70

Continued on next page

Contents (continued)

09 UI behavior (continued)

Storyboard: a card under the cursor	71
Cursor variants by context	71
Sound effects map	72
How a sound is chosen	73
Sound rules for users	73
Pointer and sound pipeline	74
Splash screen	75

09 UI behavior (continued)

Progress UI for the scan	76
Result feedback	77
Layout rules: fill the screen	78
Global states	79
3D scenes	79

10 The finished app 80

Landing and handbook	81
Plant and disease pages	82
Disease panels	83
Sign in and create account	84
Dashboard	85

10 The finished app (continued)

Scan: upload and preview	86
Scan: progress and result	87
History and account	88
Other scan outcomes	89
On a phone	90

11 Build phases and workflow 91

Timeline in four waves	91
Operating rules and safety	91
Phases	92
Execution workflow	93
Gates	94
Stop and ask	94

12 How to run 95

Quickstart	95
Where things run	95
The model while it is missing	95
Tests	96
Before launch: placeholders and open items	96

13 ML integration 97

The contract	97
Valid slugs	98
Plugging it in	99
Testing your class	99
Where feedback goes	99

14 Security and verification 100

Security checklist	100
Performance targets	101
Verification plan	102
Public extras and engineering basics	103
Risks	105
Open items	105
Glossary	106

SECTION 01

OVERVIEW

Leafy is a plant doctor in your pocket. Snap a leaf, understand the disease, know what to do next. It is a clean rebuild of the DAHON prototype as a monorepo: Next.js talks to FastAPI over REST, and the browser only ever talks to Next.js.

Goals

PRODUCT**Read the leaf, act on it**

A member uploads one leaf photo and gets the plant, the disease, causes, symptoms, treatment and prevention in short panels.

TRUST**Real accounts, private scans**

Real sign in replaces DAHON's client supplied id and plaintext passwords. Scans, history and account data are private.

KNOWLEDGE**A public handbook**

13 plants and 27 diseases with causes and treatments, readable without an account and indexed for search.

QUALITY**Fast, calm, accessible**

Lighthouse scores of 90 or more, WCAG 2.2 AA, a calm scientific voice, and a documented design system.

Stack at a glance

Layer	Technology
Web	Next.js 15 App Router, TypeScript strict, Tailwind, React Three Fiber for 3D scenes.
API	FastAPI on Python 3.12, SQLAlchemy 2 async with asyncpg, Alembic migrations, layered routers, controllers, services and repositories.
Data and services	PostgreSQL 16 (citext, pg_trgm), S3 compatible storage (MinIO in development), SMTP email (Mailpit in development).

Known issue: sample images

Some images in the DAHON dump are corrupted or mislabelled, so the extractor could not produce disease reference photos. Disease pages say "Reference photos coming soon" until a clean image set exists. Provenance and licensing must be checked before any public deploy.

In short: the app is built end to end. Only the real ML model (the teammate's) is missing.

Where the project stands

BUILT

Everything except the model

Seed data (13 plants, 27 diseases). The API: auth, Google, email, account, catalog, scans, feedback. The web app: design system, splash, landing, handbook, auth, member shell, dashboard, account and the scan journey. Plus this document.

NOT BUILT

Four open items

1. The real ML model, written by the teammate.
2. Disease reference images, blocked by corrupt DAHON sample blobs.
3. Real Google credentials.
4. Hosting, which is undecided.

Confirmed decisions

AUTH

Cookies set by Next.js

JWT access token (15 min) and a rotating refresh token (30 days, 90 day family cap) in httpOnly cookies. Tokens never reach page scripts and there is no browser CORS.

AUTH

Google sign in

Continue with Google beside email and password. Authorization Code flow with PKCE, state and nonce. The code exchange runs in FastAPI so the client secret stays on the API.

STORAGE

Two buckets, public handbook

Private `leafy-scans` (presigned links, 10 minutes) and public `leafy-catalog`. The handbook is public; scans, history and account are private. Scan images are not used for training unless the team decides so and updates the Privacy Policy.

DATA

Seed from DAHON

The DAHON dump seeds the catalog now. Joel's dataset swaps in later through the same seed format.

ML

One function contract

`predict(image: bytes) -> dict[str, str]` returns `{"plant": slug, "disease": slug, "healthy" or "unknown", optional "confidence": "0.93"}`. It receives the sanitized JPEG, long edge at most 2048 px, EXIF stripped. The stub raises `NotImplementedError`, which maps to 503 `ml_unavailable`.

ACCOUNT

Account scope for v1

Sign up, sign in, sign out, email verification, forgot and reset password, edit profile, change password, delete account with all scans.

TOOLING

Default toolchain

Python 3.12, uv, ruff, pytest, SQLAlchemy 2 async with asyncpg and Alembic. pnpm, Next.js 15, Tailwind. SMTP email with Mailpit in development.

UI

Leafy brand and rules

Dark forest default with light mode. Josefin Sans Light caps, Poppins, Manrope. Titles of one or two lines, no hyphenation, sideways panels. English only for v1, strings in one dictionary.

UX

Honest progress

Scans return 202 and run in the background. The UI shows real upload percent, then Checking image, Analyzing leaf, Saving result, Done.

SPEED

Lighthouse is the yardstick

Four categories at 90 or more on mobile. LCP at most 2.5 s, INP at most 200 ms, CLS at most 0.1.

Scope

Area	In scope for v1	Status
Monorepo and compose	Scaffold, <code>docker-compose.yml</code> , <code>.env.example</code> , lint and CI skeleton. Dockerfiles for <code>api</code> and <code>app</code> are not written yet.	🕒 Partly
Seed data	13 plants, 27 diseases, <code>labels.json</code> , <code>slug_overrides.yaml</code> , cleaning report, idempotent loader. Raspberry is not seeded, so it resolves to <code>unknown</code> .	✓ Built
API core and account	Config, errors, envelope, models, auth, email tokens, Google sign in (mock until real keys), profile, delete account.	✓ Built
Scans API	Storage, image validator, ML stub, resolver, async 202 flow, retry, janitor, stats, feedback and its CSV export.	✓ Built
Web foundation and brand	Env, <code>apiFetch</code> , cookies, middleware, auth handlers, tokens, logo set, splash, 3D scenes, sound.	✓ Built
Public pages	Landing, handbook (plants, diseases, search), About, Privacy, Terms, SEO, PWA manifest, branded emails.	✓ Built
Member pages	Auth pages, dashboard, scan journey with progress UI, result panels, history, feedback, account.	✓ Built
Browser tests	Playwright suites for auth, member, scan, UI and the full stack, with layout assertions.	✓ Built
Lighthouse and PageSpeed	Lighthouse CI is configured. PageSpeed Insights needs a public URL.	🕒 Waits for hosting

Out of scope

Item	Why and what ships instead
The real <code>predict()</code>	The teammate writes the ML. Leafy ships the abstract class, the stub, <code>labels.json</code> , <code>slug_overrides.yaml</code> and a one page guide (see the ML integration section). A dev only fake model lets the whole journey run.
Joel's dataset	Swapped into the seed format later.
Email change in profile	Needs re-verification, planned for v2.
Admin or CMS for handbook	Content changes by editing the seed JSON and re-running the idempotent seed.
Deployment	Decided later. The repository stays deploy ready.
Filipino translation, Google unlink, profile photo	English only for v1. The other two are not in v1.

What closes the four open items

ML

Plug in the model

The teammate subclasses `MLInferenceService` and sets `ML_SERVICE=package.module:Class`. No other code changes.

IMAGES

A clean image set

Disease reference photos come from a clean source, such as Joel's dataset, through the same seed format. Licensing is checked first.

GOOGLE

Create the OAuth client

The team creates the client in Google Cloud Console, sets the three Google variables and turns `GOOGLE MOCK` off.

HOSTING

Pick a host

Then write the Dockerfiles, set `S3_PUBLIC_ENDPOINT`, and run PageSpeed Insights on the public URL.

SECTION 02

SYSTEM ARCHITECTURE

Leafy has one front door. The browser only talks to Next.js, Next.js talks to FastAPI, and FastAPI is the only thing that touches the database, storage, email, the ML service and Google.

Four tiers

CLIENT**Browser**

Shows the Leafy UI. It holds two httpOnly cookies, `leafy_at` and `leafy_rt`, and never sees a token in JavaScript. It never calls FastAPI, so there is no browser CORS.

WEB TIER**Next.js 15**

Middleware refreshes the session and guards routes. Server components fetch data with `apiFetch`. Route handlers proxy changing calls to FastAPI and set cookies.

API TIER**FastAPI**

REST under `/api/v1` in four layers: router, controller, service, repository. It owns business rules, transactions and the error envelope. Only Next.js can reach it.

SERVICES**Data and outside systems**

PostgreSQL 16 for data, an S3 compatible store for images, SMTP for email, the ML service for predictions and Google for sign in. Only FastAPI calls them, except presigned image links.

How to read the diagrams

ARROWS**Request****Response****Optional**

Every arrow is one call with a label. Line style shows the kind of call and outline style shows the kind of box.

**Plain outline**

Our own code.

**Heavy outline**

A main piece.

**Dashed outline**

An outside system.

**Double rule**

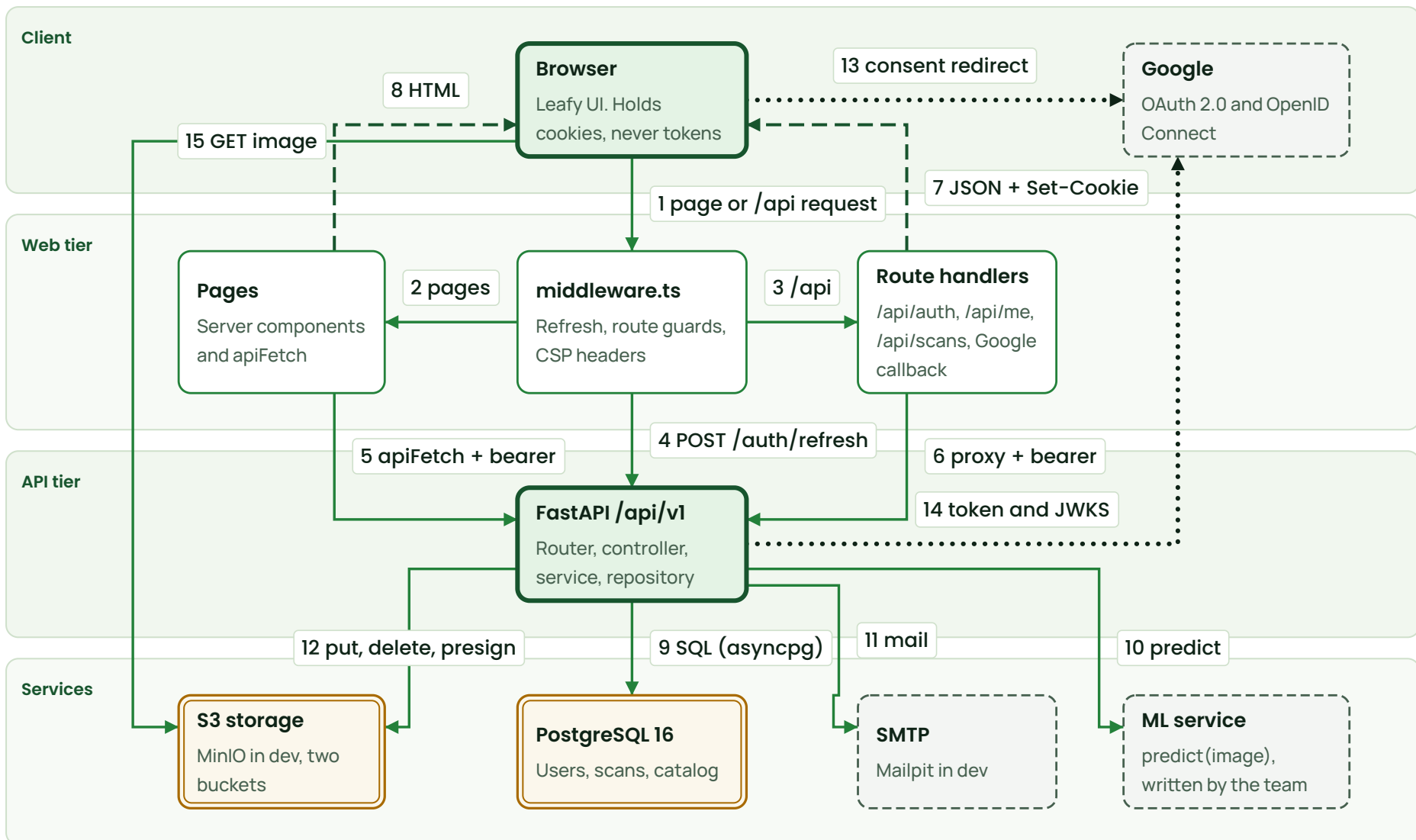
A place that holds data.

Status

Every box here is built. The ML service box runs the stub, or a dev only fake, until the teammate's model arrives. The Dockerfiles are not written yet, so development uses native services.

System architecture diagram

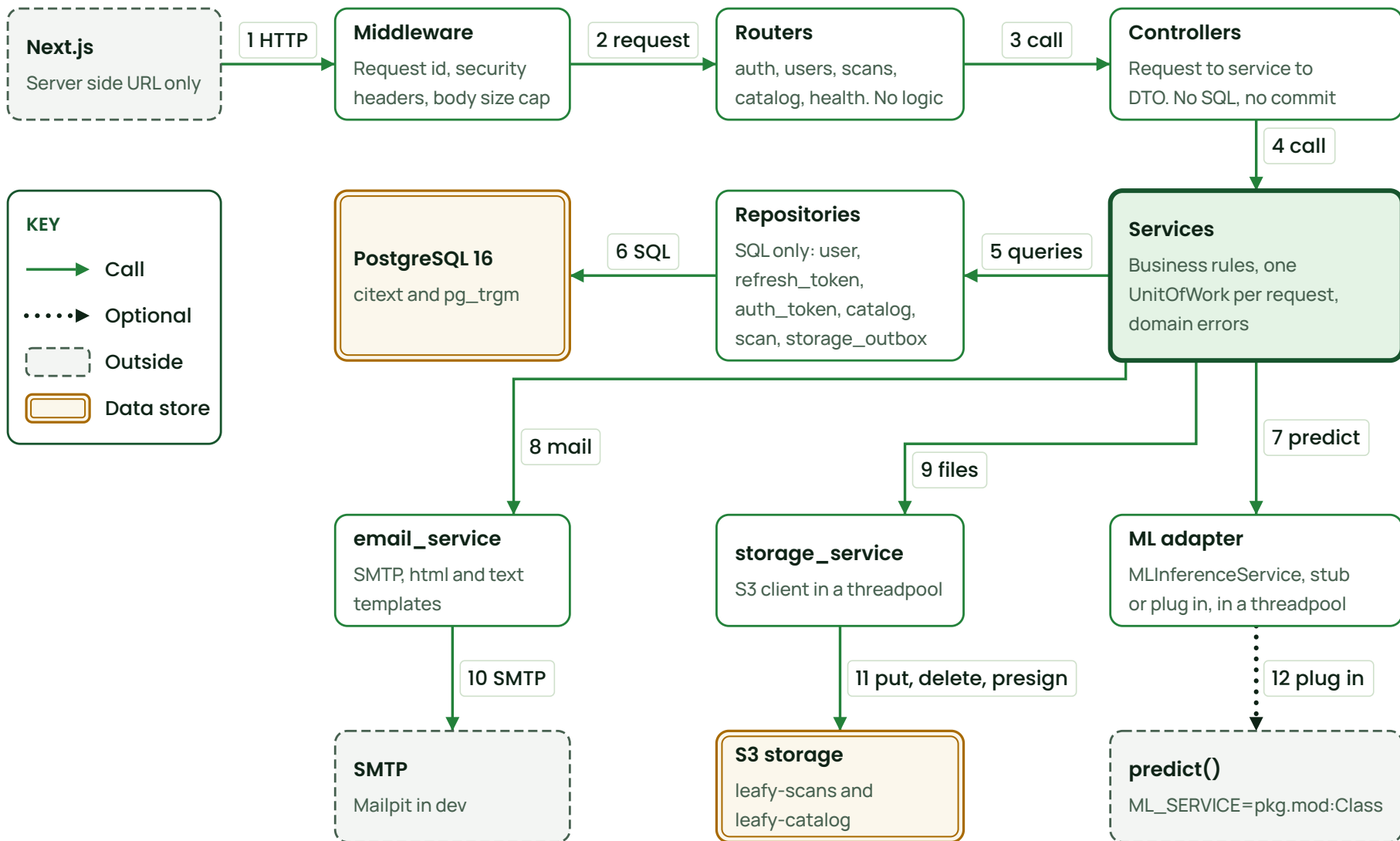
ARROWS  Request  Response  Optional (Google)



The browser only talks to Next.js. Numbers match the calls table on the next pages. Dashed boxes are outside systems, double rule boxes hold data.

FastAPI layers

ARROWS → Call Optional (plug in the real model)



Each layer only calls the next one. Responses return the same way. Blocking work runs in a threadpool. Dashed boxes are outside systems, double rule boxes hold data.

Calls in the architecture diagram

No.	From to	Call	Notes
1	Browser to middleware	HTTPS page or <code>/api</code> request	Cookies <code>leafy_at</code> and <code>leafy_rt</code> ride along. Every request passes the middleware first.
2	Middleware to pages	Allowed page request	Protected routes without a session go to <code>/login?next=</code> . Guest routes go to <code>/dashboard</code> when signed in.
3	Middleware to route handlers	Allowed <code>/api</code> request	State changing handlers call <code>assertSameOrigin</code> .
4	Middleware to FastAPI	<code>POST /auth/refresh</code>	Sent when the access token is missing or within 30 s of expiry. Single flight.
5	Pages to FastAPI	<code>apiFetch</code> with bearer token	On 401 it refreshes and retries once, or redirects through <code>/api/auth/refresh?next=</code> .
6	Route handlers to FastAPI	Proxied REST call with bearer token	Uploads stream through, at most 8 MiB. The API URL is server only.
7	Route handlers to browser	JSON and <code>Set-Cookie</code>	Cookies are <code>httpOnly</code> , <code>SameSite=Lax</code> , path <code>/</code> , <code>__Host-</code> prefix in production.
8	Pages to browser	Server rendered HTML	Public handbook pages use ISR with a one hour cache.
9	FastAPI to PostgreSQL	SQL over <code>asyncpg</code>	SQLAlchemy 2 <code>async</code> , one <code>UnitOfWork</code> per request.
10	FastAPI to ML service	<code>predict(image)</code>	Runs in a threadpool with a timeout. The stub raises <code>NotImplementedError</code> , which becomes 503 <code>ml_unavailable</code> .
11	FastAPI to SMTP	Verify, reset and password changed mails	Mailpit in development.
12	FastAPI to S3 storage	Put, delete, presign	Private <code>leafy-scans</code> with 10 minute presigned GET, public <code>leafy-catalog</code> .
13	Browser to Google	Consent redirect	Started by <code>/api/auth/google</code> . PKCE, state and nonce.
14	FastAPI to Google	Token exchange and JWKS fetch	Holds <code>GOOGLE_CLIENT_SECRET</code> . Verifies the ID token.
15	Browser to S3 storage	GET presigned image	Signed against <code>S3_PUBLIC_ENDPOINT</code> . An expired link is refetched through the API.

What runs in docker compose

Service	Port	Role
<code>app</code>	3000	Next.js. The only service published in production.
<code>api</code>	8000	FastAPI. Runs <code>alembic upgrade head</code> , then the seeds, then uvicorn. A dev override publishes the port for <code>/docs</code> .
<code>postgres</code>	5432	PostgreSQL 16 with citext and pg_trgm.
<code>minio</code> , <code>minio-init</code>	9000, 9001	S3 compatible storage. The init job creates both buckets and a scoped key.
<code>mailpit</code>	1025, 8025	Dev SMTP inbox and its web view.
<code>purge</code>	none	Loop that drains the storage deletion outbox and cleans old tokens.

Session cookies

Cookie	Lifetime	Purpose and flags
<code>leafy_at</code>	900 s	Access token, a JWT valid for 15 minutes. Set by Next.js, never readable by page scripts.
<code>leafy_rt</code>	30 days	Rotating refresh token. The family is capped at 90 days.
Both cookies	see above	httpOnly, Secure in production with the <code>__Host-</code> prefix, SameSite=Lax and path <code>/</code> . Lax is used because Strict would drop the session when someone follows an email link.

Key settings in .env.example

Setting	Read by	What it does
<code>DATABASE_URL</code>	api	PostgreSQL connection string (asyncpg).
<code>JWT_SECRET</code>	api	Signs access tokens. Must be set at startup. Never committed.
<code>APP_ORIGIN</code>	app, api	The public site origin, used for the origin check and email links.
<code>COOKIE_SECURE</code>	app	Turns on Secure cookies and the <code>__Host-</code> prefix in production.
<code>COOKIE_PREFIX</code>	app	Cookie name prefix, so <code>__Host-</code> can be on in production and off locally.
<code>API_INTERNAL_URL</code>	app	Where Next.js reaches FastAPI. Server only, never sent to the browser.
<code>MAX_UPLOAD_BYTES</code>	app, api	Upload size cap, enforced at Next.js and again in FastAPI.
<code>S3_PUBLIC_ENDPOINT</code>	api	Host that presigned image links are signed against.

Setting	Read by	What it does
<code>ML_SERVICE</code>	api	The <code>pkg.mod:Class</code> that provides <code>predict()</code> .
<code>GOOGLE_CLIENT_ID</code>	app, api	OAuth client id from Google Cloud Console. The API checks it as the ID token <code>aud</code> .
<code>GOOGLE_CLIENT_SECRET</code>	api only	OAuth client secret. Never sent to Next.js or the browser.
<code>GOOGLE_REDIRECT_URI</code>	app, api	<code>http://localhost:3000/api/auth/google/callback</code> locally, plus the production callback URL. Must match the console exactly.
<code>GOOGLE MOCK</code>	app, api	Set to <code>1</code> to use a mock Google provider (tiny fake token endpoint and JWKS) in tests and local dev, so nothing waits for real credentials.

Google Cloud Console steps

The README section on this is still a stub. The steps are: create an OAuth consent screen, then a Web client with the redirect URIs above, and copy the id and secret into `.env` with `GOOGLE MOCK=0`. Until then the mock provider is used.

Why the browser never calls FastAPI

Tokens stay out of JavaScript, there is no CORS to configure, and FastAPI can stay unpublished in production. It trusts `X-Forwarded-For` only from the compose network.

Local services without Docker

Service	Address	Notes
PostgreSQL 17 (project local)	<code>127.0.0.1:5433</code>	Database and user <code>leafy</code> . Never port 5432. State lives in <code>.dev/</code> .
S3 compatible storage	<code>127.0.0.1:9000</code>	A <code>moto</code> server with the two buckets. In memory, so data is lost on stop.
Mailpit	<code>:1025</code> and <code>:8025</code>	SMTP and the inbox page. Verify and reset mails land here.
API and web	<code>:8000</code> and <code>:3000</code>	FastAPI with <code>/docs</code> , and the Next.js production build.

SECTION 03

APP FLOWS

Every journey in Leafy, one diagram per page. Each box is a screen or a step, each arrow is something the user does or the system answers.

Guests and members

Who	Pages
Guest	Landing, Handbook (public), Login, Register, Forgot and reset password, Verify email. A signed in user who opens a guest page goes to <code>/dashboard</code> .
Member	Dashboard, Scan, Scans (history), Scan detail, Handbook, Account. A signed out visitor on a member page goes to <code>/login?next=</code> .
Navigation	Desktop: header with Handbook, Scan, History, Dashboard, theme toggle and user menu. Mobile: bottom bar with Home, a centered Scan button, History, Handbook and Account.
Rule	Every page keeps one obvious next action.

The flows in this section

Flow	Journey	Entry point
1	First visit	Splash, then the landing page
2	Sign up and verify email	<code>/register</code> , then the email link
2b	Google sign in	Continue with Google on <code>/login</code> and <code>/register</code>
3	Sign in, session expiry and sign out	<code>/login</code>
3b	Forgot and reset password	<code>/forgot-password</code>
4	Scan a leaf, with four outcomes	<code>/scan</code>
5	History and delete with undo	<code>/scans</code>
6	Handbook browse	<code>/handbook</code>
7	Dashboard	<code>/dashboard</code>
8	Account	<code>/account</code>

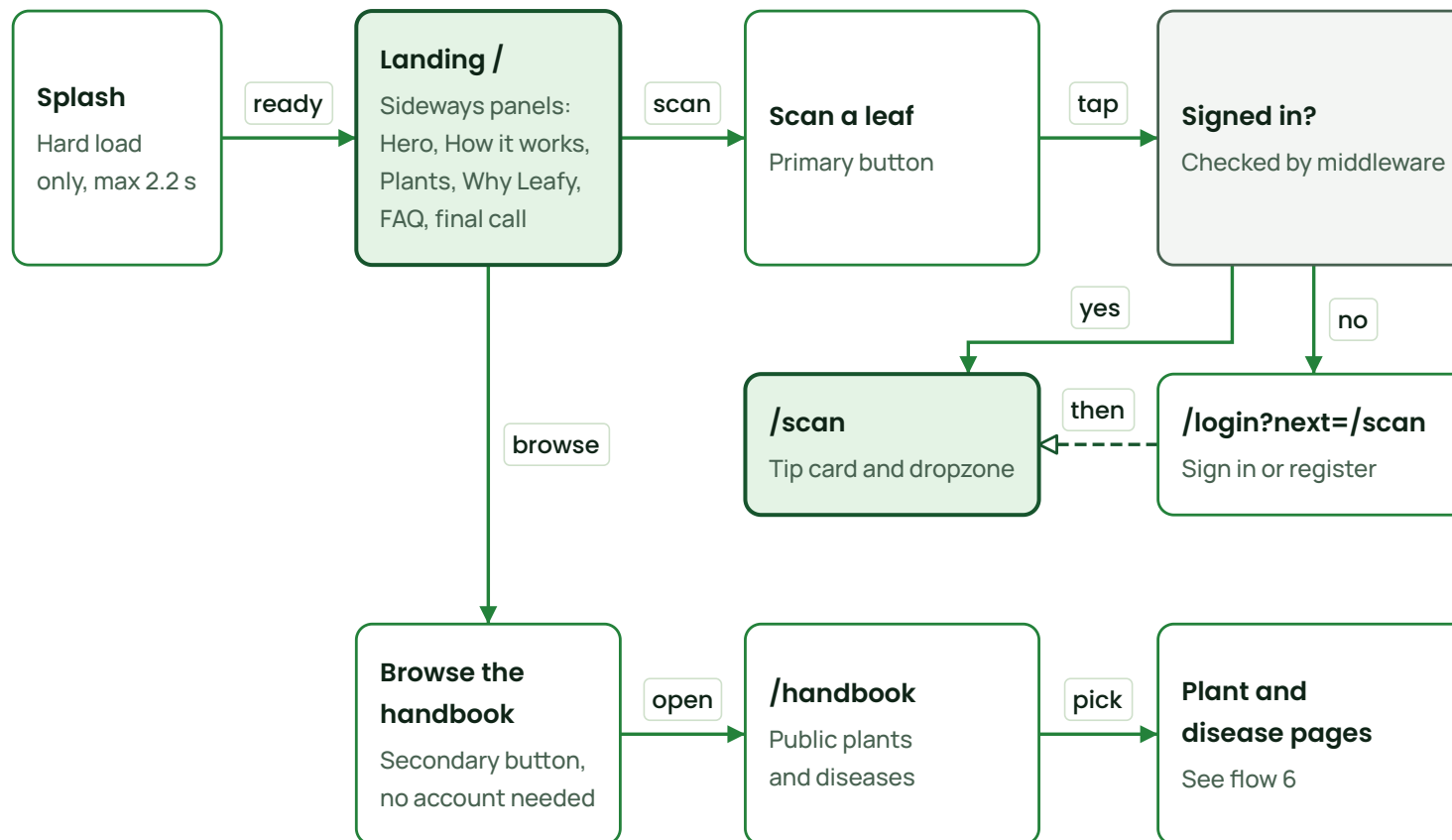
Arrows

 User action or request

 Response

 Optional

Flow 1: first visit



A first visitor either scans a leaf, with sign in on the way, or browses the handbook with no account.

Arrows

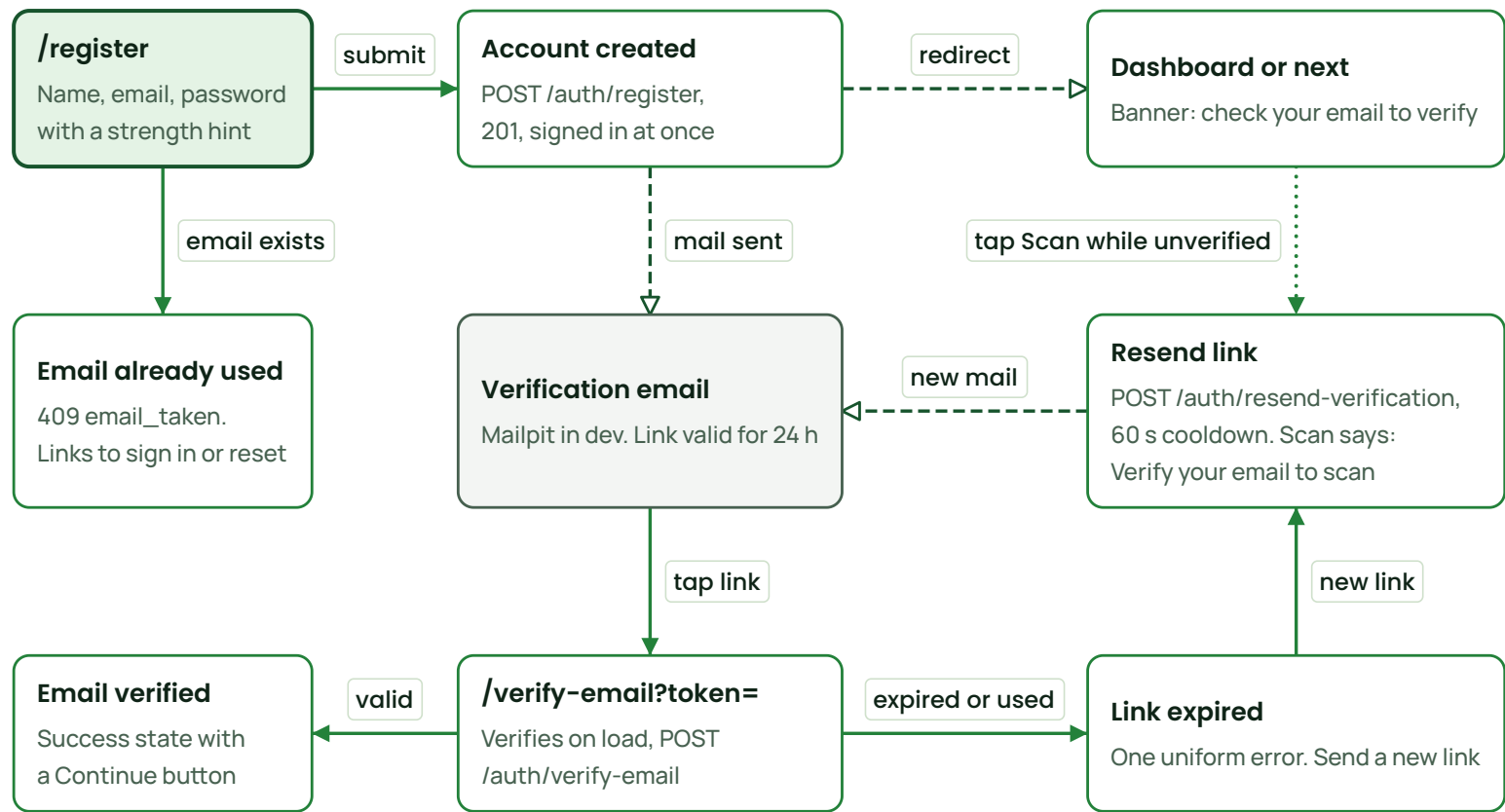


User action



Response or redirect back

Flow 2: sign up and verify email



Sign up signs the user in at once. Verifying the email unlocks scanning, and a new link is always one tap away.

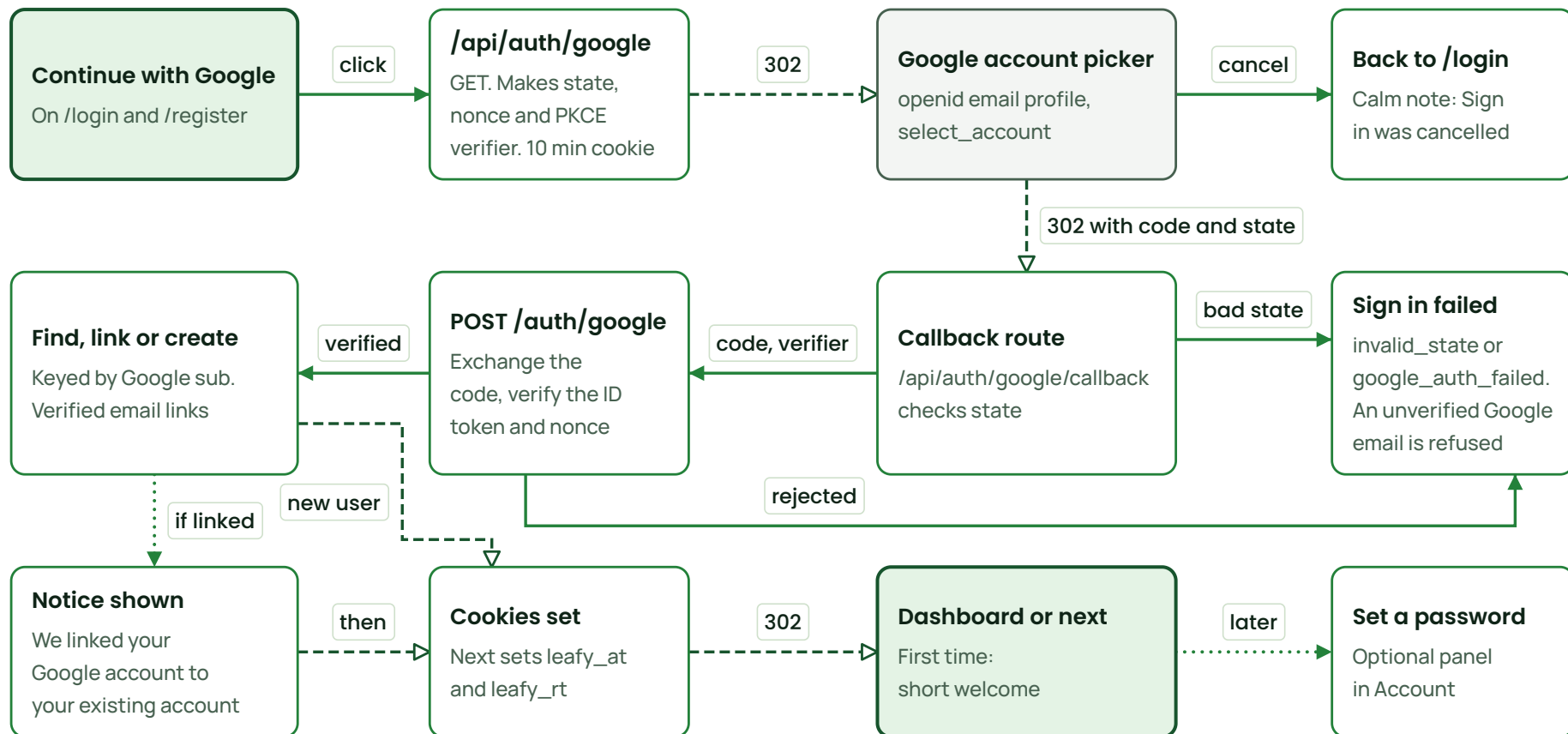
Arrows

User action

Response or email sent

Optional or conditional

Flow 2b: Google sign in



The browser legs run through Next.js. The code exchange and ID token check run in FastAPI, so the client secret never leaves the API.

Arrows



User action or request

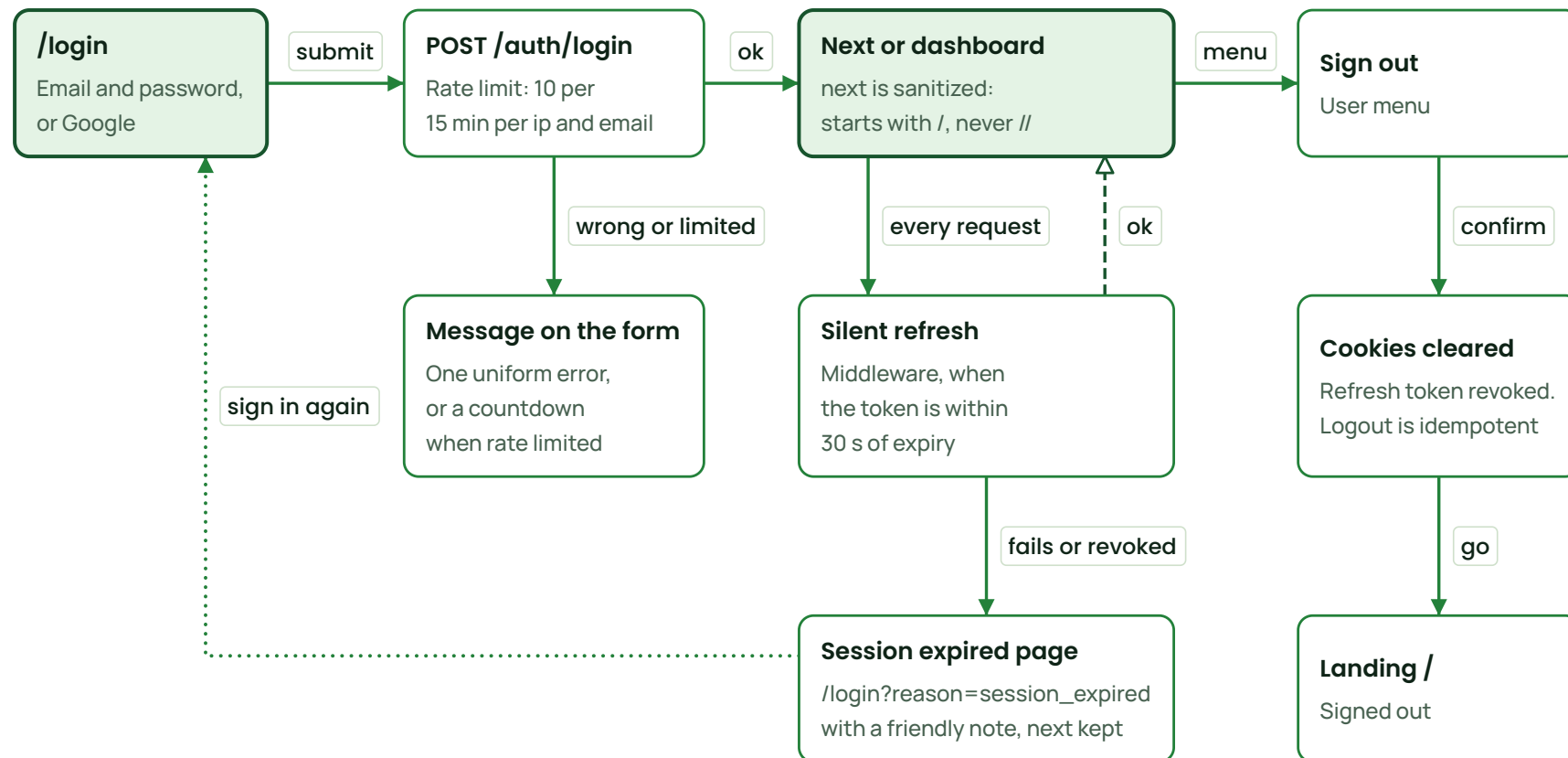


Redirect or response



Optional or conditional

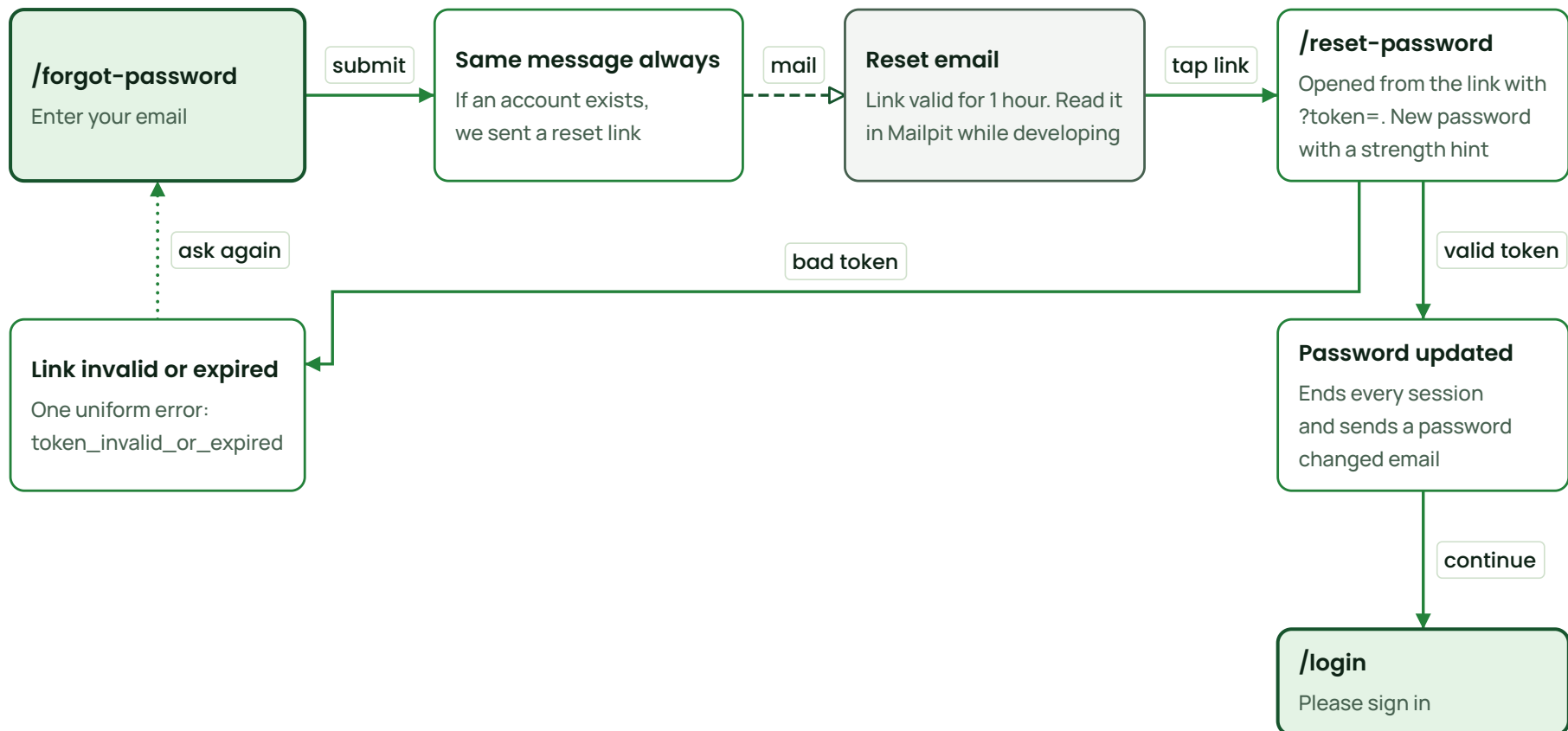
Flow 3: sign in, session expiry and sign out



Sessions renew silently. When a session cannot be renewed the user returns to sign in with the page they wanted kept.

Arrows User action or request Response Optional or conditional

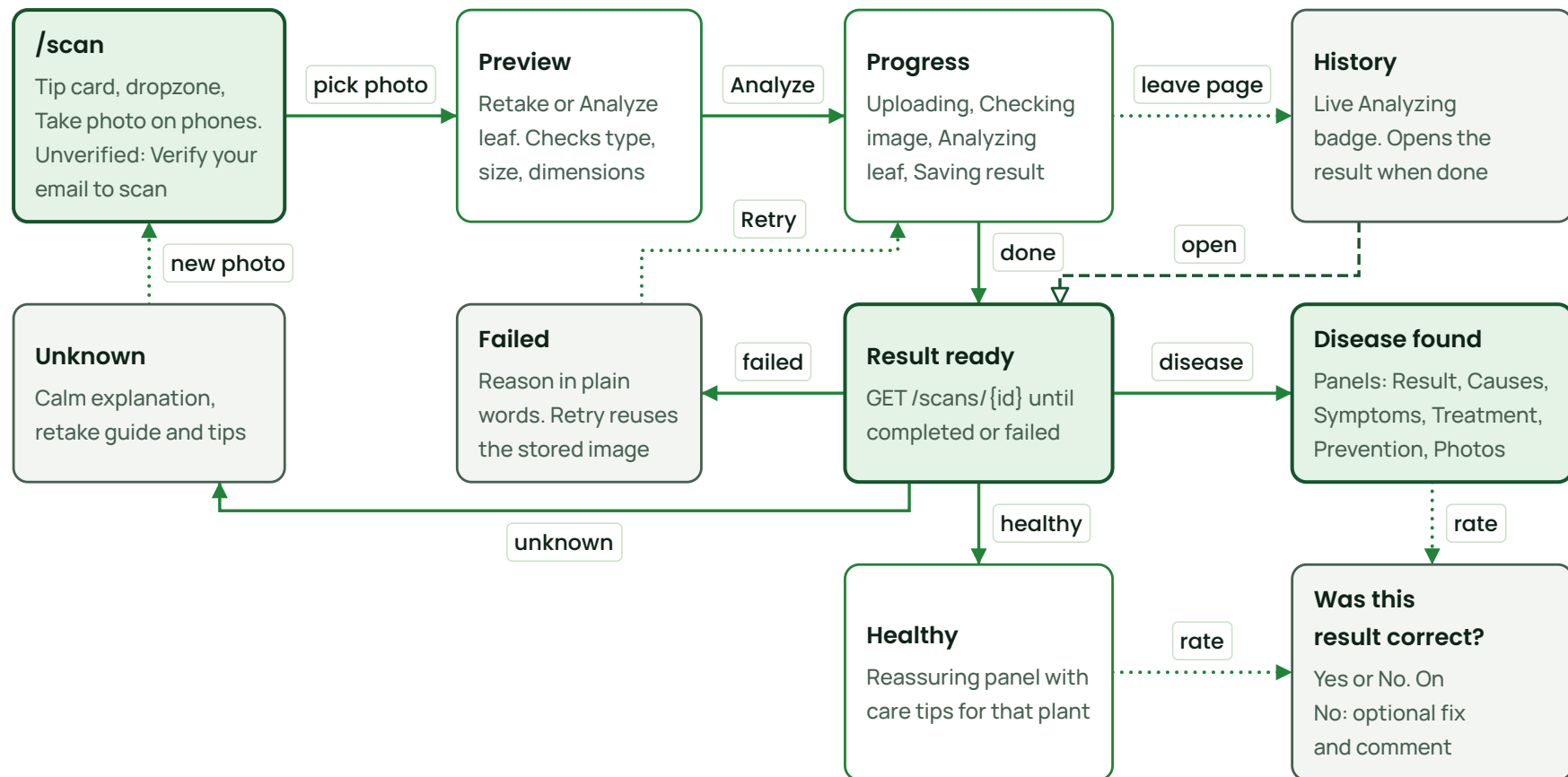
Flow 3b: forgot and reset password



The reply never reveals whether an email has an account. A reset ends every other session.

Arrows User action Email sent Optional or conditional

Flow 4: scan a leaf



The core journey. Every scan ends as disease found, healthy, unknown or failed, and leaving the page is always safe.

Arrows



User action or step

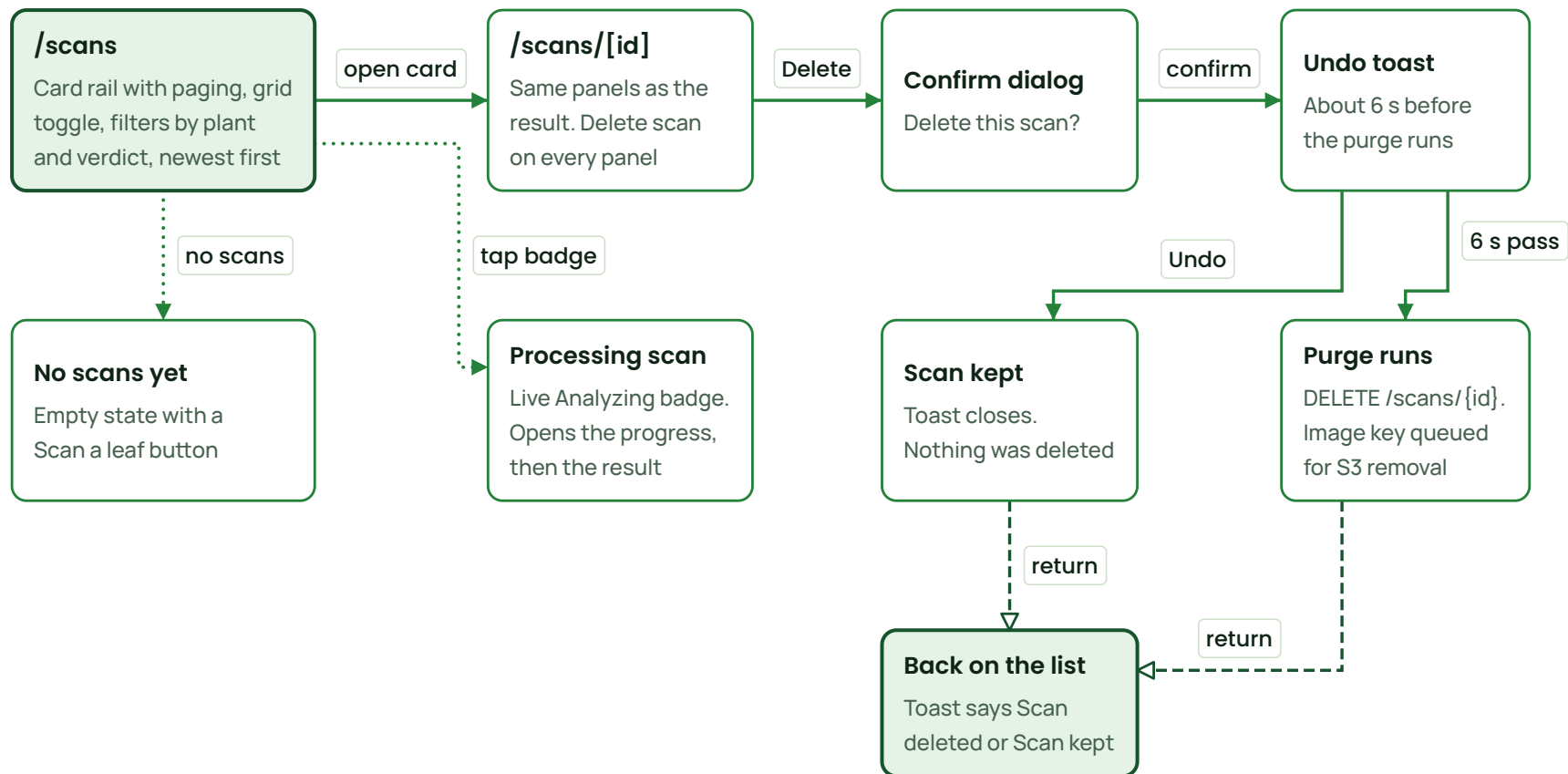


Result opens



Optional or retry

Flow 5: history and delete with undo



Deleting is forgiving: a short Undo window comes before the purge runs.

Arrows



User action

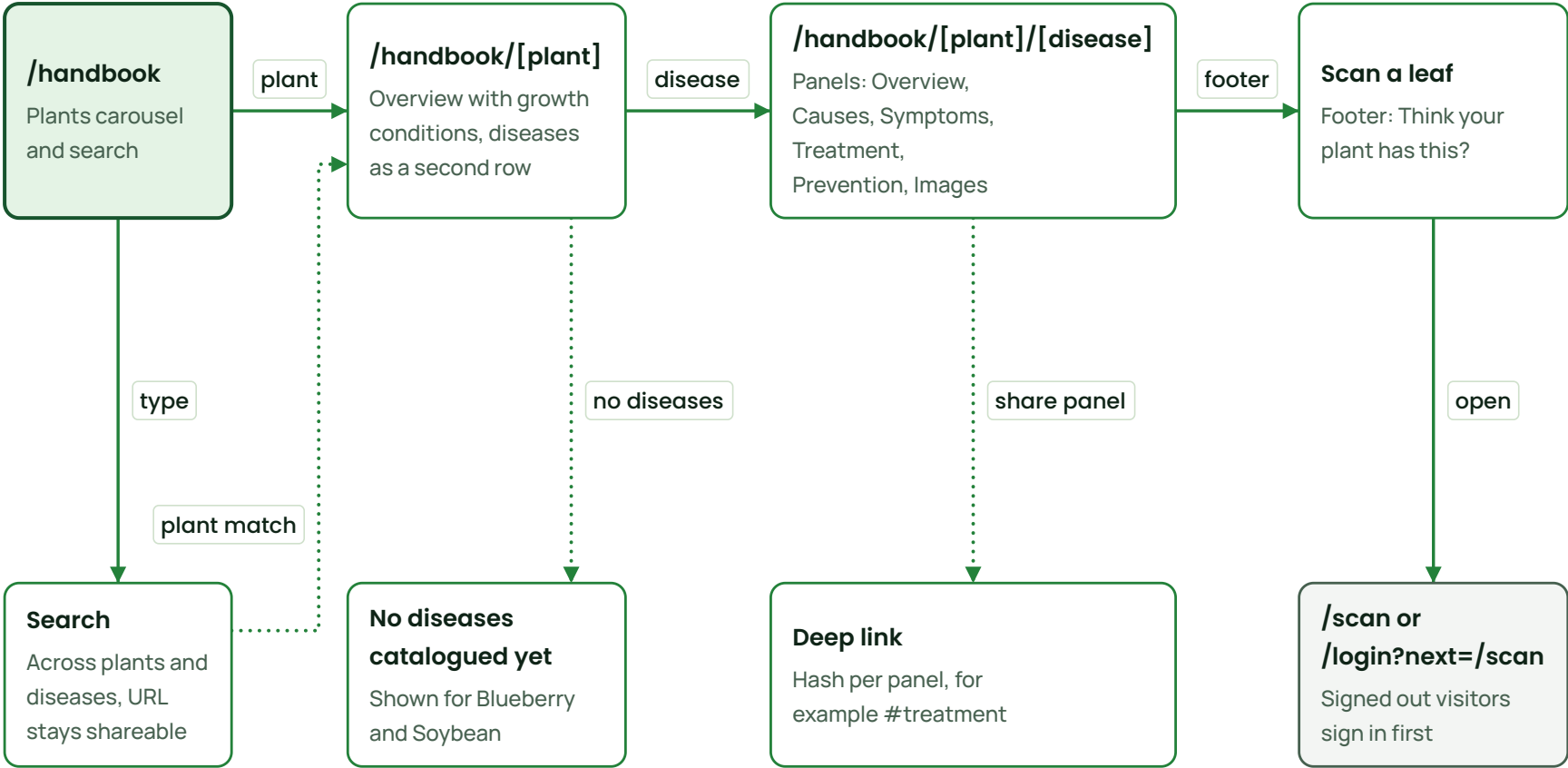


Return



Optional or conditional

Flow 6: handbook browse



The handbook is public. URLs are plant scoped and shareable, and back and forward work throughout.

Arrows

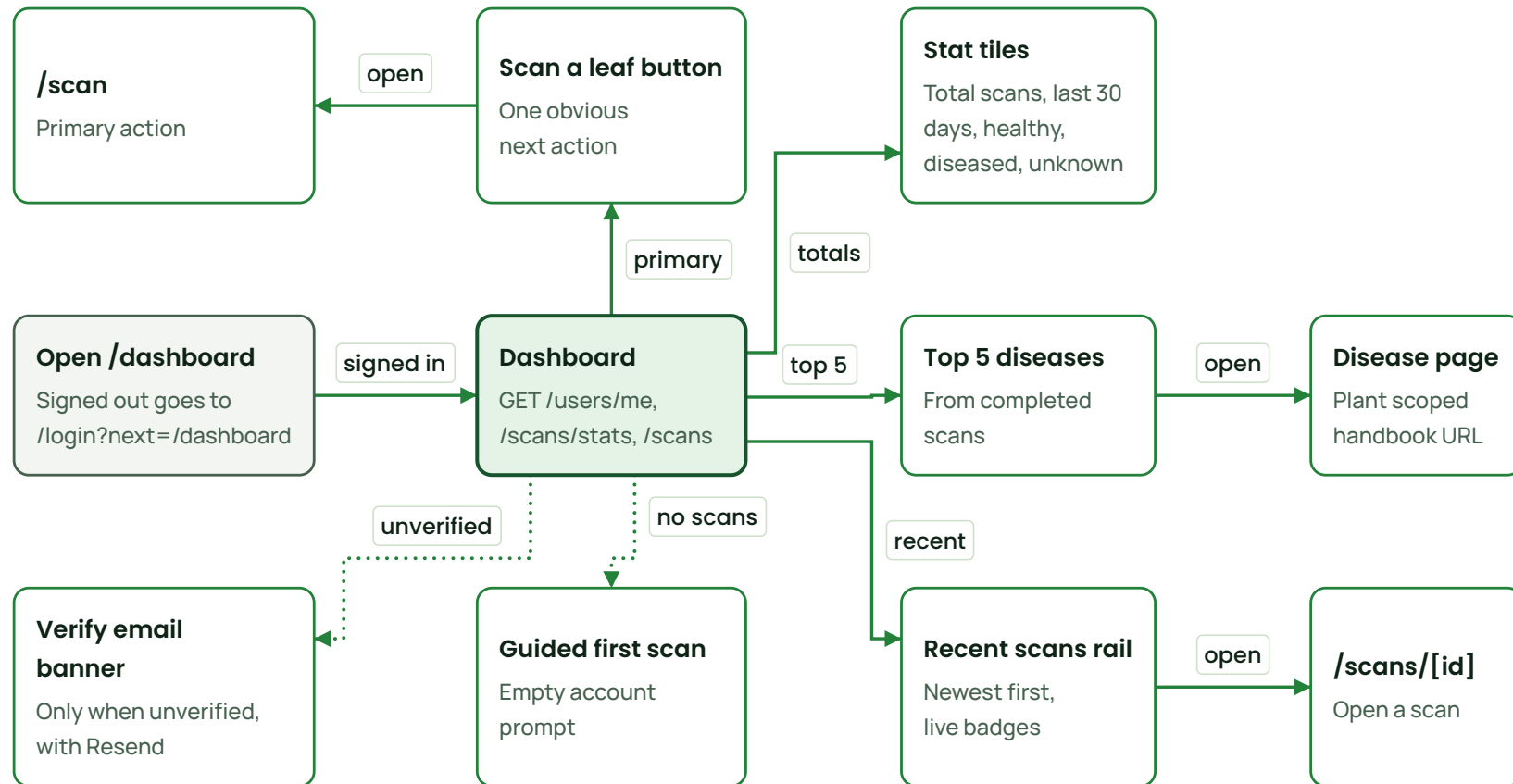


User action



Optional or conditional

Flow 7: dashboard



One screen answers how my plants are doing, with a single primary action: Scan a leaf.

Arrows

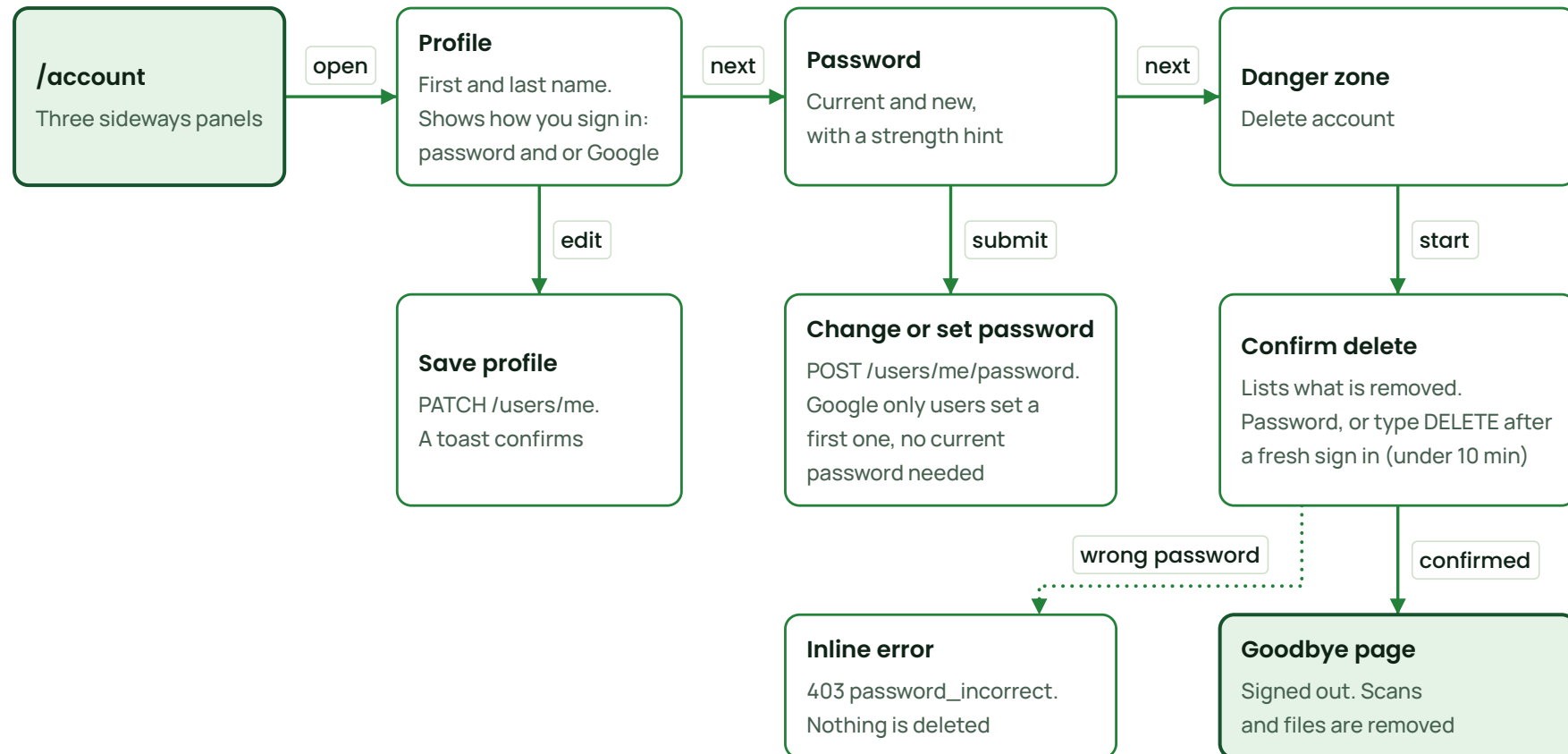


Shows or opens



Only in some states

Flow 8: account



Three panels. Deleting the account needs a password, or typing DELETE for Google only users. Email change is not in v1.

Arrows



User action



Error path

SECTION 04

SCAN STATES AND PROGRESS

A scan is analyzed in the background, so the user can always see where it stands. The API and the UI share one state machine.

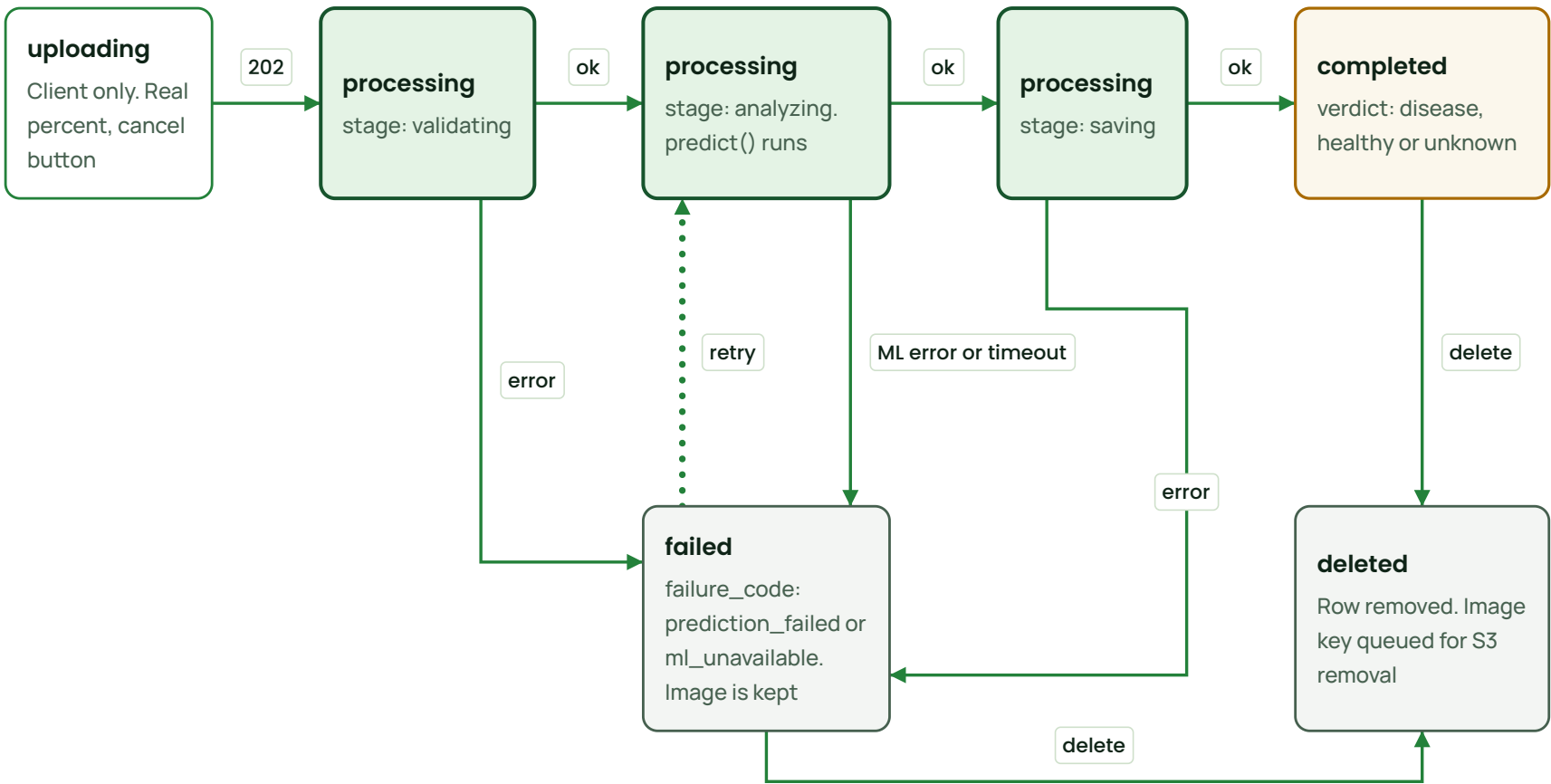
States and stages

State	Stored as	What happens and what the user sees
uploading	client only	The browser sends the photo. A real percentage and a Cancel button.
processing, validating	<code>status=processing</code> <code>stage=validating</code>	<code>POST /scans</code> checked and stored the image and answered 202. The screen says Checking image. Long edge capped at 2048 px and re-encoded as JPEG; the model receives this sanitized JPEG.
processing, analyzing	<code>stage=analyzing</code>	A background worker runs <code>predict()</code> in a threadpool with a timeout. Scanning beam, eased bar, elapsed time.
processing, saving	<code>stage=saving</code>	The prediction is resolved to a plant and disease and written. The screen says Saving result.
completed	<code>status=completed</code> with a <code>verdict</code>	The result panels open. The verdict is disease, healthy or unknown.
failed	<code>status=failed</code> with a <code>failure_code</code>	A reason in plain words and a Retry button that reuses the stored image.

Transitions

Move	Trigger
uploading to validating	<code>POST /scans</code> accepted. Answer 202 with <code>{id, status, stage}</code> .
validating to analyzing	The worker starts and sets <code>stage=analyzing</code> .
analyzing to saving	<code>predict()</code> returned and the resolver matched the slugs.
saving to completed	The verdict is written together with the raw prediction (jsonb).
any processing stage to failed	<code>NotImplementedError</code> gives <code>ml_unavailable</code> . Any other error or a timeout gives <code>prediction_failed</code> . A janitor also fails scans stuck in processing too long.
failed to analyzing	<code>POST /scans/{id}/retry</code> on the stored image. A scan that is not failed answers 409 <code>scan_not_failed</code> .
completed or failed to deleted	<code>DELETE /scans/{id}</code> . Deleting also cancels a processing scan.

Scan state machine



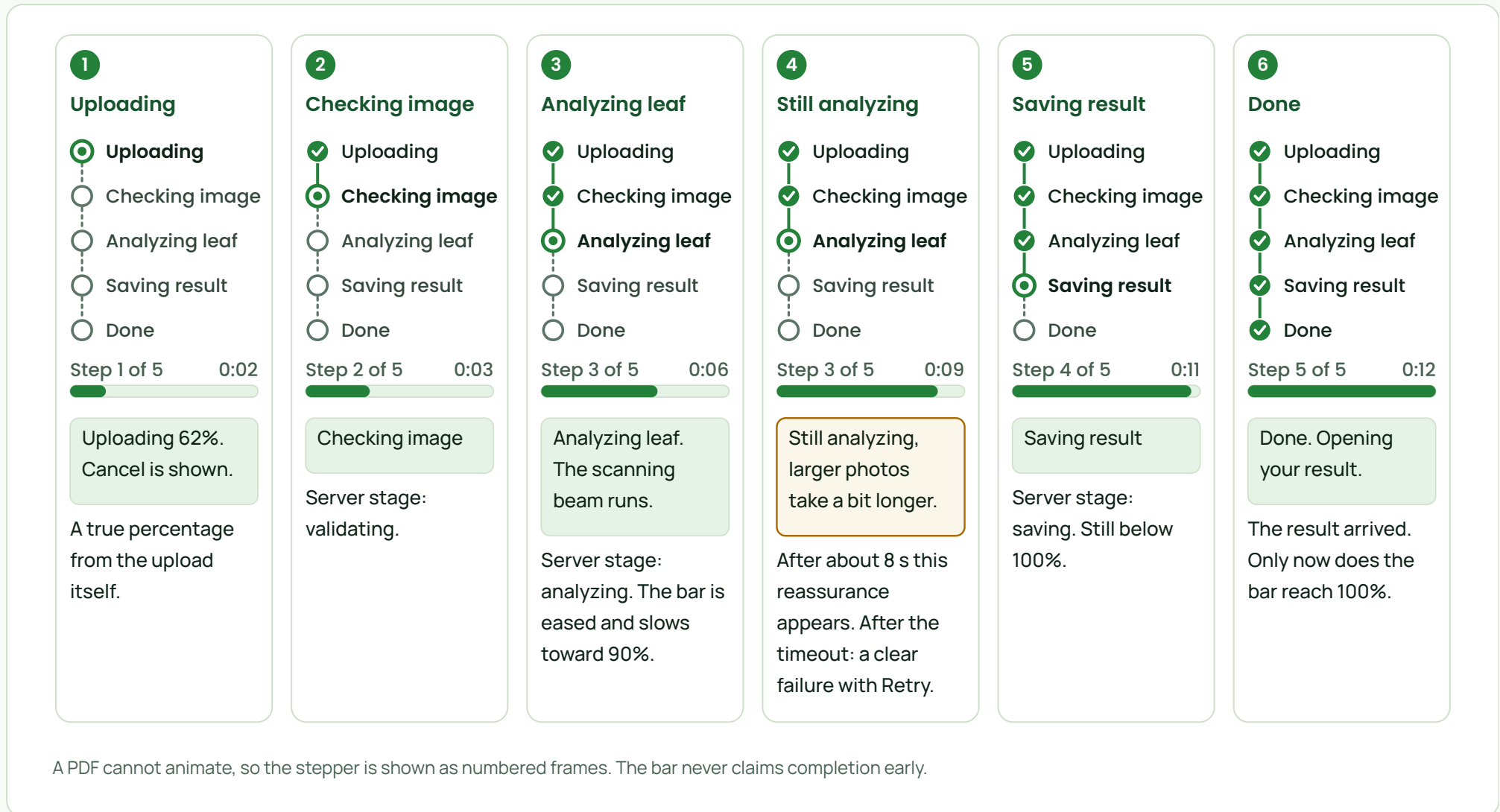
One shared state machine. Retry returns to analyzing on the stored image. Delete is allowed from completed and failed, and cancels processing.

Arrows

Transition

User triggered retry

Progress stepper, frame by frame



Stepper marks

Check mark: step finished

Ring with a dot: current step

Empty ring: step still to come

Honest progress

Stage	Where the number comes from	What the bar does
Uploading	The real upload, from XHR <code>upload.onprogress</code>	Fills with the true percentage.
Checking image	The stage the API reports: <code>validating</code>	Moves a short, fixed step.
Analyzing leaf	The stage <code>analyzing</code> . One <code>predict()</code> call has no true percentage.	Eased from open ended to a number, slowing toward 90%. It only reaches 100% when the result arrives.
Saving result, then Done	The stage <code>saving</code> , then the status <code>completed</code>	Stays below 100% until the result is stored, then reaches 100% and the result opens.

Prediction resolver

Input from <code>predict()</code> , slugs normalized with aliases first	Result (the raw prediction is always kept as <code>jsonb</code>)
Unknown plant, or a disease that matches nothing	Verdict <code>unknown</code> . The plant is kept when it is known. A disease of <code>healthy</code> gives verdict <code>healthy</code> .

Behaviors that keep users informed

POLLING

Ask, do not guess

The page polls `GET /scans/{id}` through the Next proxy every second, backing off to two seconds, until the scan is completed or failed. The screen never invents a state.

WAITING

Reassurance, then a way out

After about 8 s the screen adds Still analyzing, larger photos take a bit longer. After the timeout it shows a clear failure with Retry on the already uploaded image.

LEAVING

Navigating away is safe

The scan keeps processing. It appears in History with a live Analyzing badge and progress, and opening it resumes the progress or shows the result. Cancel is available during upload.

ACCESS

Works for everyone

Screen readers get a live progress value and every change is announced. Reduced motion drops the 3D effects. The same bar serves account deletion.

Rejected before a scan exists

A bad image, a missing email check, storage trouble or a rate limit answers the upload itself. No scan row is created.

SECTION 05

AUTH AND SESSION SEQUENCES

How sign in, sessions, email tokens, Google and account deletion work step by step. Each sequence reads from top to bottom, and each arrow is one call.

Tokens and lifetimes

Token	Form and storage	Lifetime and use
Access token	JWT in the <code>leafy_at</code> cookie. Never readable by page scripts.	15 minutes (900 s). Next.js refreshes it when it is missing or within 30 s of expiry.
Refresh token	Opaque random value in <code>leafy_rt</code> . The database keeps only its hash, with a <code>family_id</code> .	30 days, and the whole family ends after 90 days. It rotates on every use.
Verify email token	Random 256 bit value, stored as sha256 in <code>auth_token</code> .	24 hours. One use, consumed by a single atomic UPDATE.
Reset password token	The same shape as the verify token, type <code>reset_password</code> .	1 hour. One use. A reset revokes every refresh family.
OAuth cookie	State, nonce and the PKCE verifier. httpOnly, SameSite=Lax, <code>__Host-</code> prefix.	10 minutes. Single use, cleared after the callback.
Google ID token	Checked on the server with cached JWKS. Never stored.	Used once to find or create the user.

How to read the sequence diagrams

Mark	Sample	Meaning
Participant box		One participant sits at the top. Time runs down the page, and each arrow is one call.
Lifeline		The thin solid grey line below a participant. It continues behind notes and frame tabs.
Solid arrow		A request. The number in the label is the step order.
Dashed arrow		A response, a redirect or a mail.
Dotted arrow		Optional or async work, such as the S3 drain job.
Yellow note		A rule or detail for the participants beside it.
Frame with alt and else		Alternatives. Exactly one branch of the frame happens.

Rules the sequences follow

UNIFORM

No hints for attackers

Wrong credentials, an unknown email and a user without a password all give the same 401. Forgot password always answers 200. Bad email tokens give one 400.

ROTATION

Reuse revokes the family

A rotated token used again inside 10 s is a harmless concurrent refresh. After 10 s it counts as theft and the whole family is revoked.

SECRETS

Only the API holds them

Next.js and the browser never see `GOOGLE_CLIENT_SECRET`. Email tokens are stored hashed, and no token or code stays in a URL after the callback.

OUTBOX

Delete works without S3

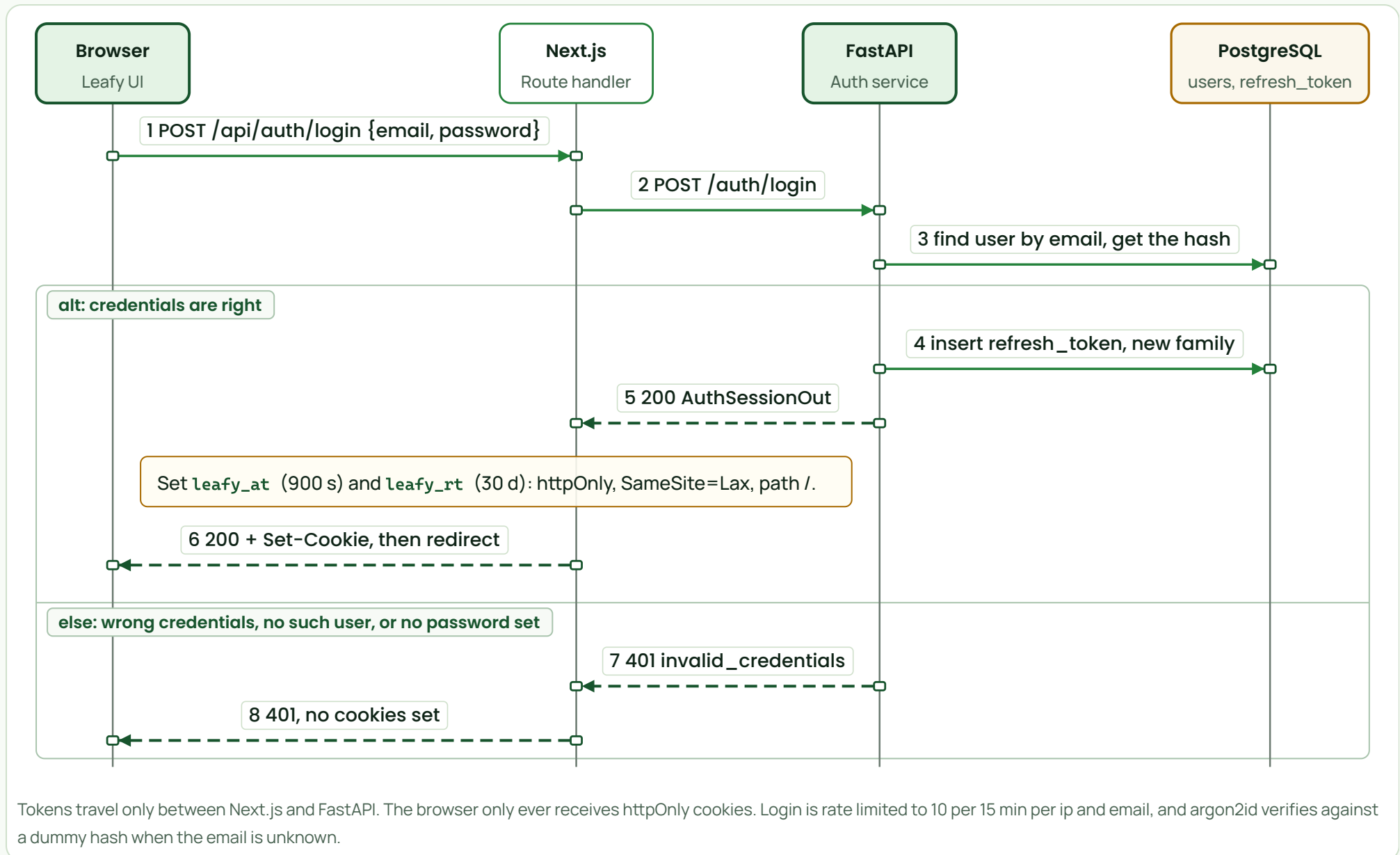
Image keys are queued in `storage_deletion` in the same transaction that deletes the user. A drain job removes the files later.

Sequences in this section

No.	Sequence	Participants	What it shows
1	Login with cookies	Browser, Next.js, FastAPI, PostgreSQL	Where the tokens travel, and why the browser only gets httpOnly cookies.
2	Refresh rotation and reuse detection	Next.js, FastAPI, PostgreSQL	The four outcomes of one refresh call, including the 10 s grace window.
3	Google sign in, part 1	Browser, Next.js, Google	The redirect legs and the short lived OAuth cookie.
4	Google sign in, part 2	Browser, Next.js, FastAPI, Google, PostgreSQL	The code exchange and ID token checks that stay inside FastAPI.
5	Email verification token, in 2 parts	Browser, Next.js, FastAPI, PostgreSQL, SMTP	A hashed token, mailed once and consumed atomically.
6	Password reset token, in 2 parts	Browser, Next.js, FastAPI, PostgreSQL, SMTP	One uniform reply, and a reset that ends every session.
7	Account delete, S3 outbox, in 2 parts	Browser, Next.js, FastAPI, PostgreSQL, purge job, S3	One transaction for the account, and files removed afterwards.

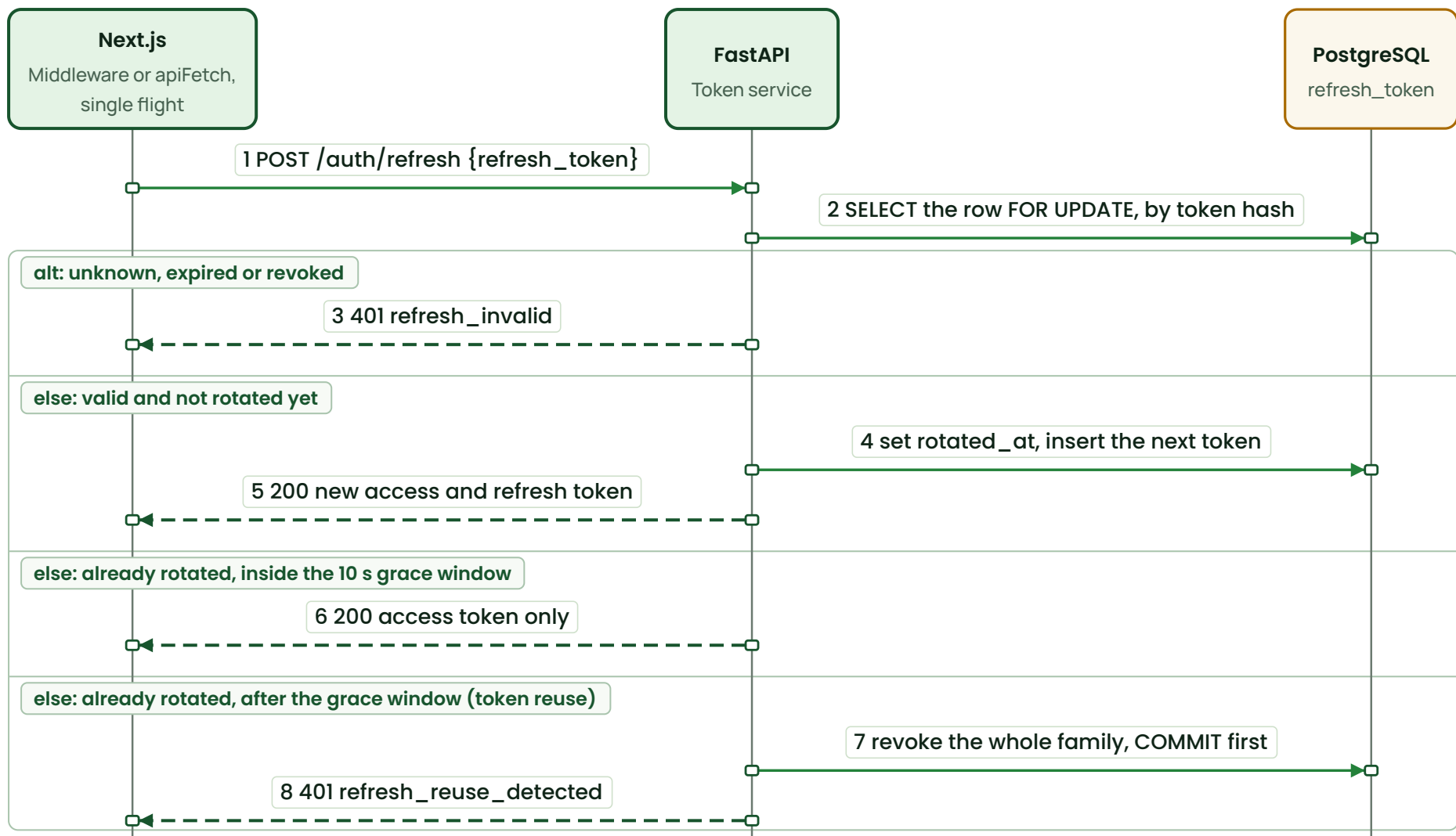
Sequence: login with cookies

Arrows  Request  Response  Lifeline



Sequence: refresh rotation and reuse

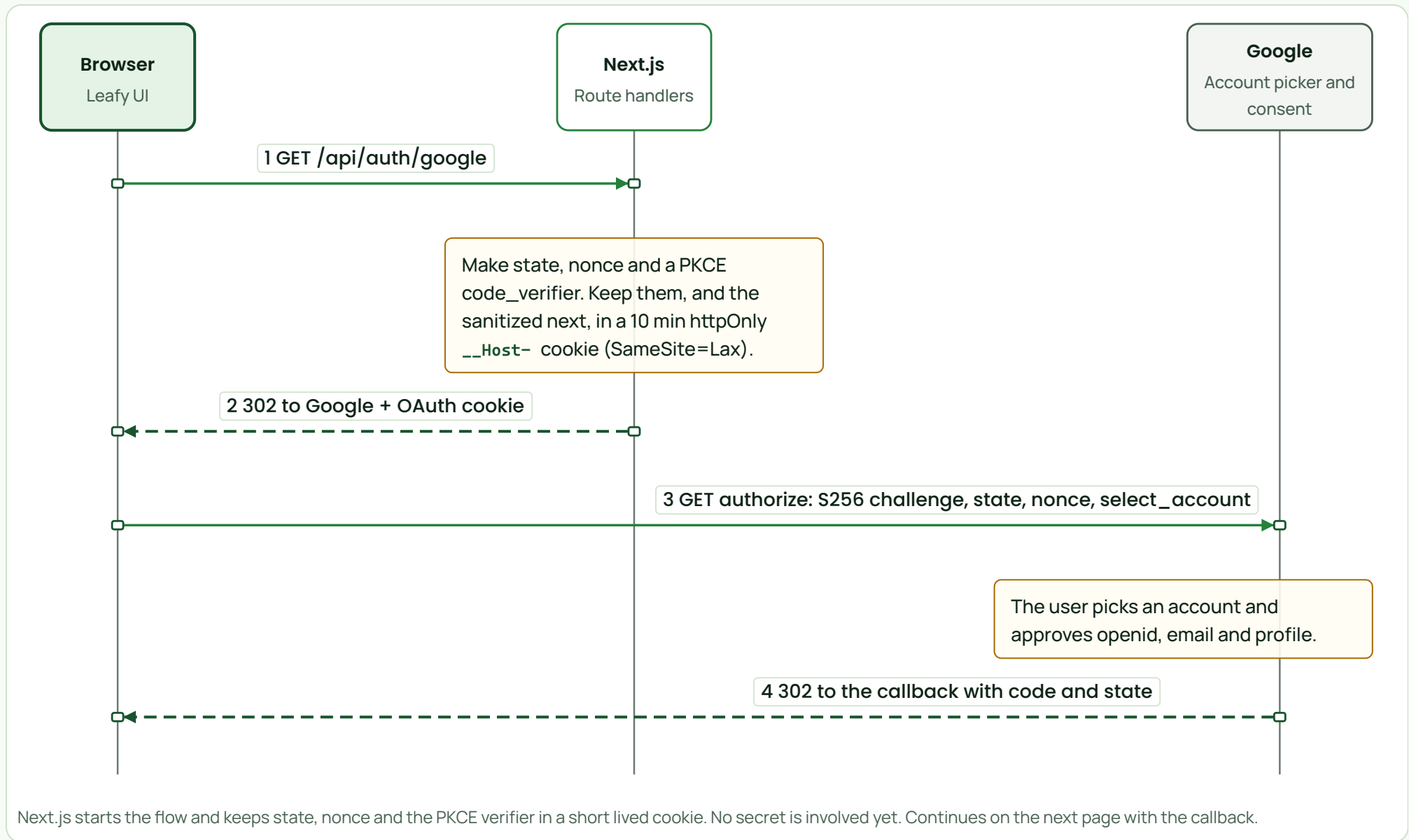
Arrows  Request  Response  Lifeline



The row is locked while it is read. Four outcomes: invalid, rotated, harmless double refresh, and reuse that revokes the family. After a reuse, Next.js clears both cookies and sends the user to `/login?reason=session_expired`.

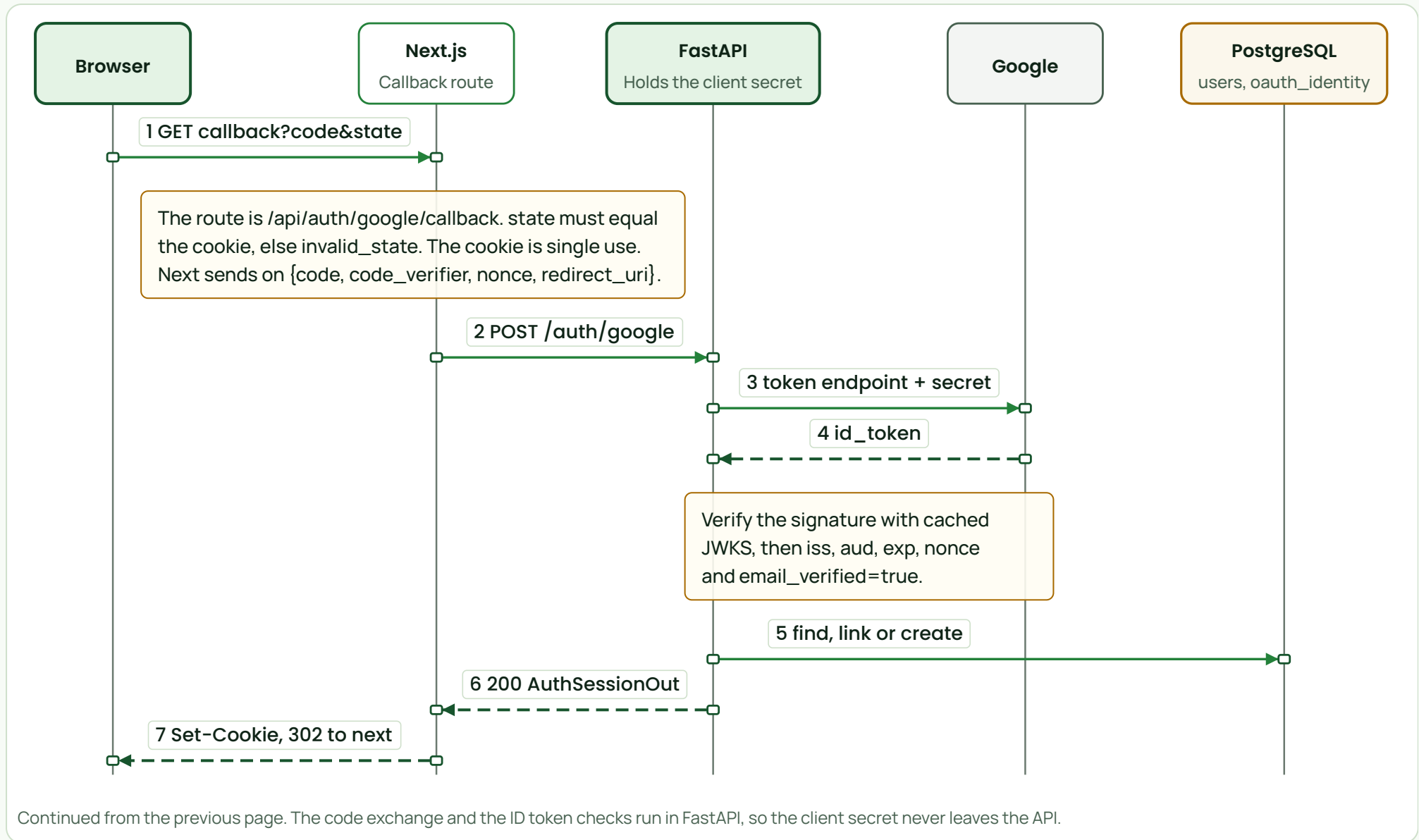
Sequence: Google sign in, part 1

Arrows  Request  Redirect or response  Lifeline



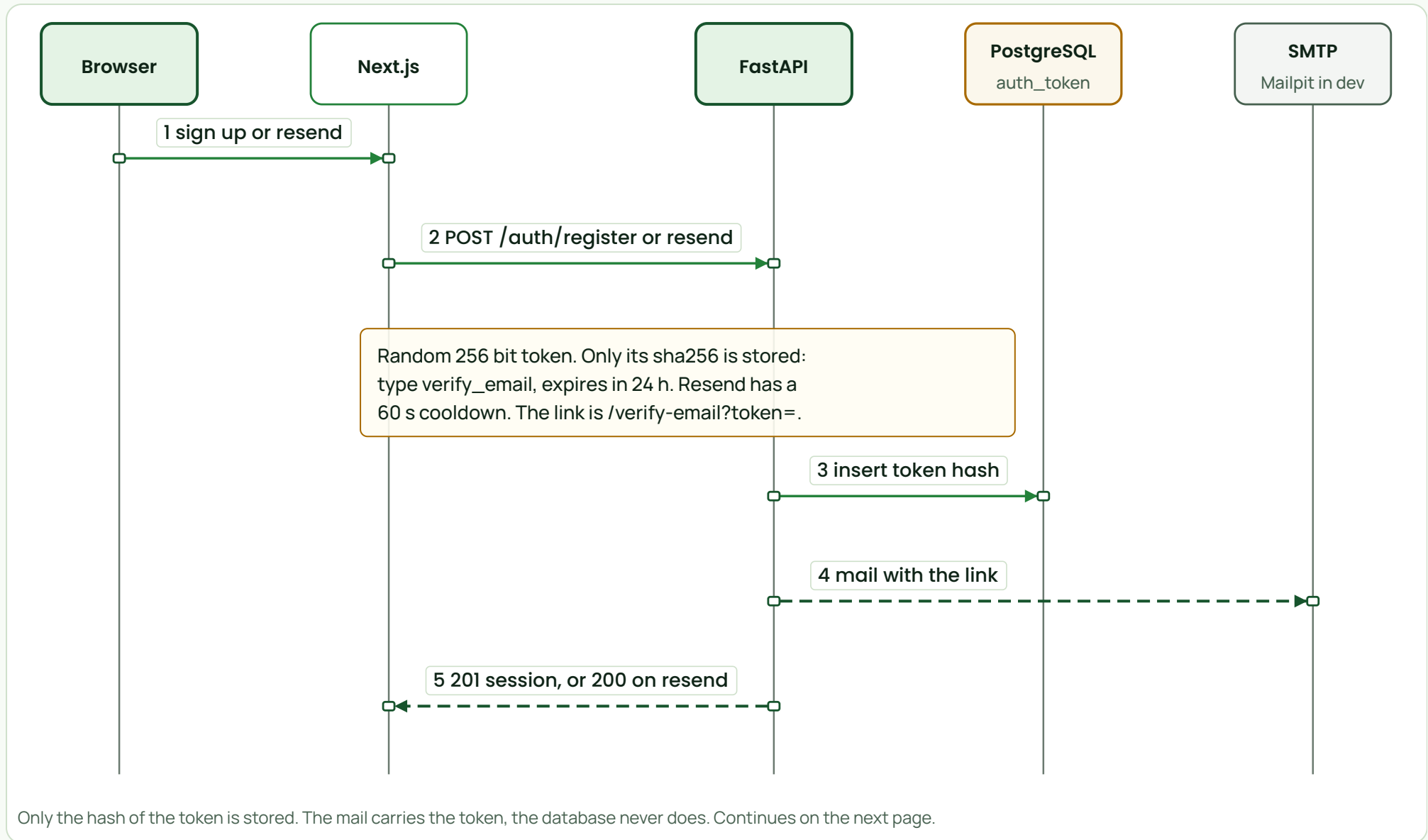
Sequence: Google sign in, part 2

Arrows  Request  Redirect or response  Lifeline



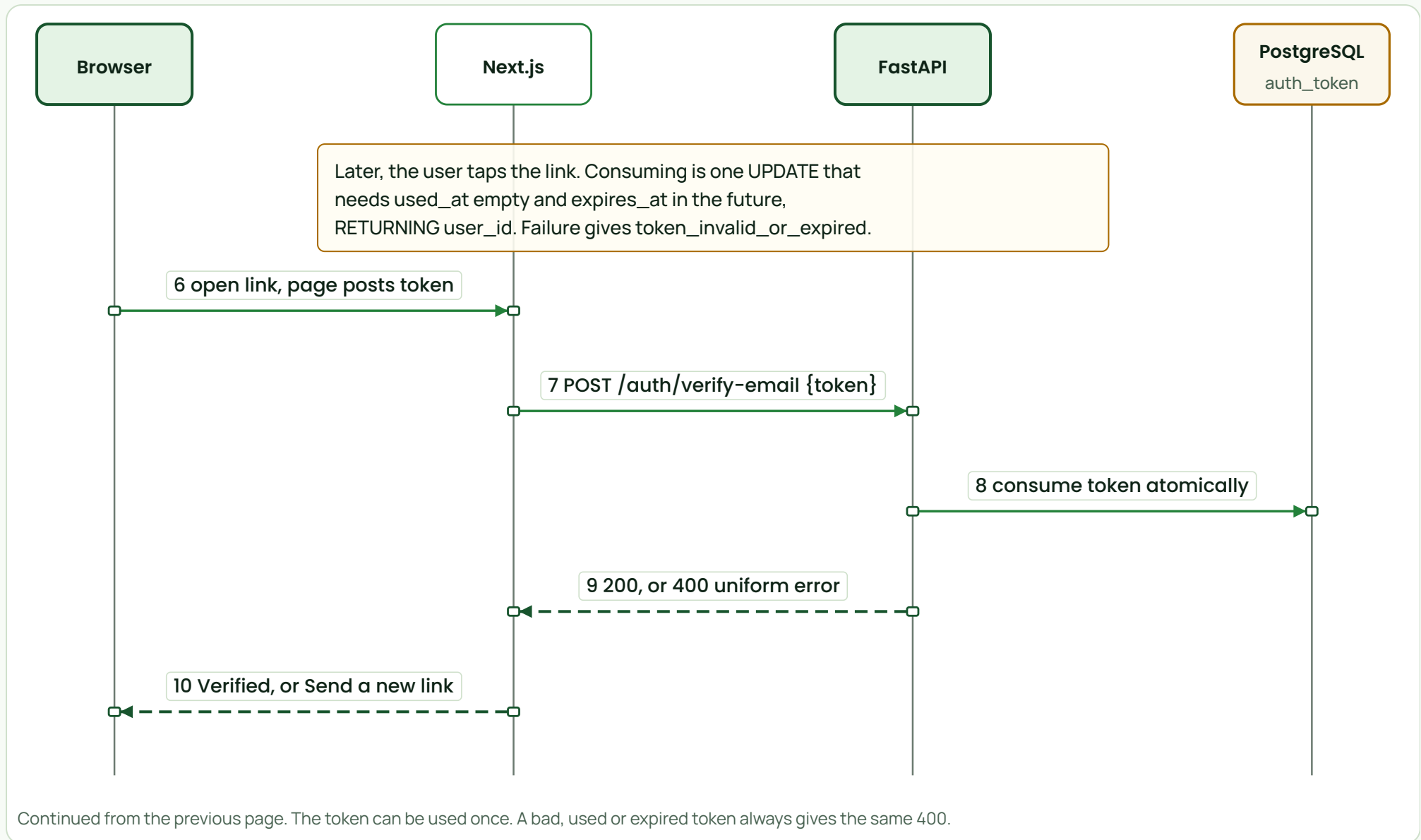
Sequence: email verification token, part 1

Arrows  Request  Response or mail  Lifeline



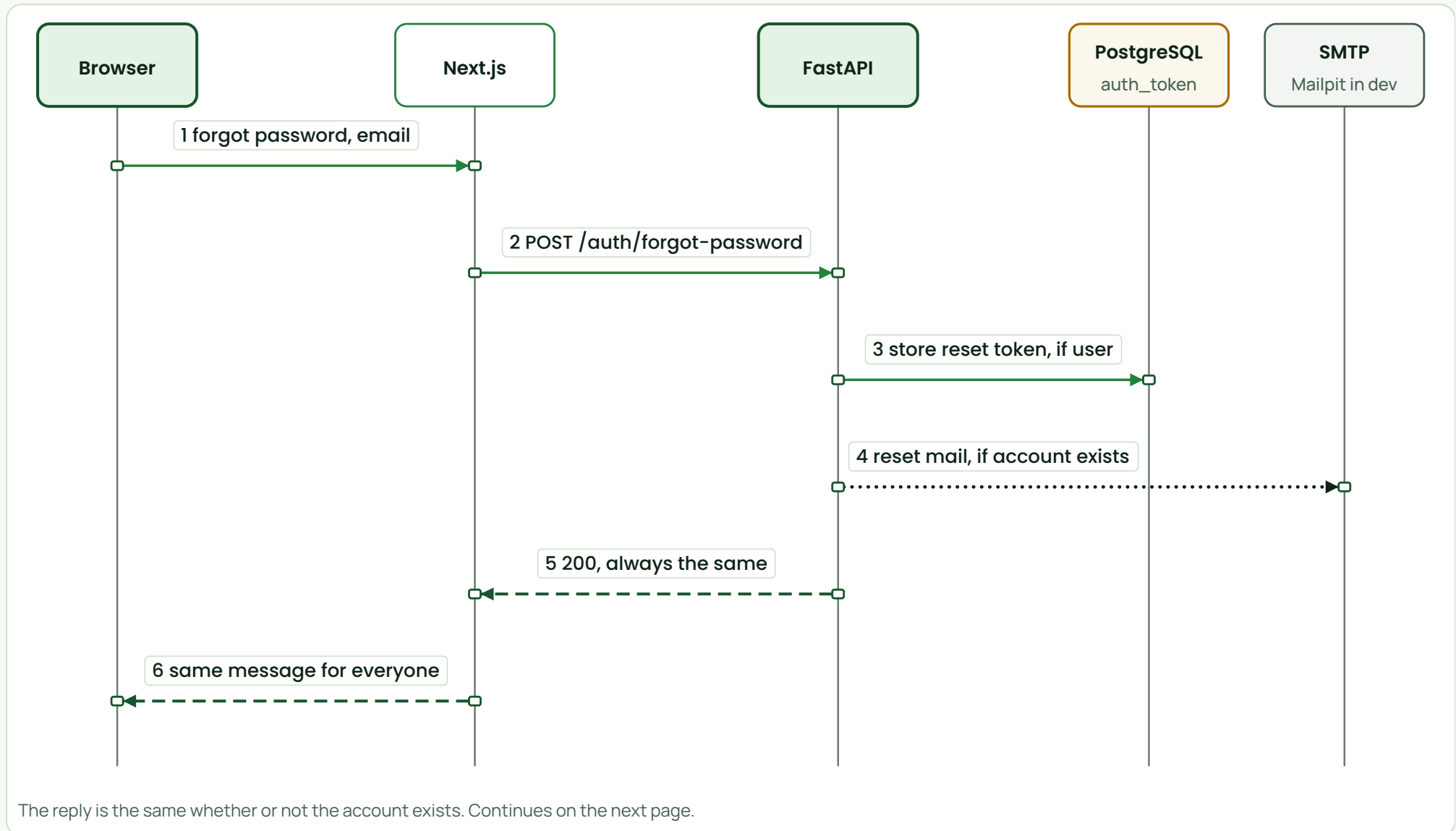
Sequence: email verification token, part 2

Arrows  Request  Response  Lifeline



Sequence: password reset token, part 1

Arrows  Request  Response  Optional  Lifeline



Sequence: password reset token, part 2

Arrows

→

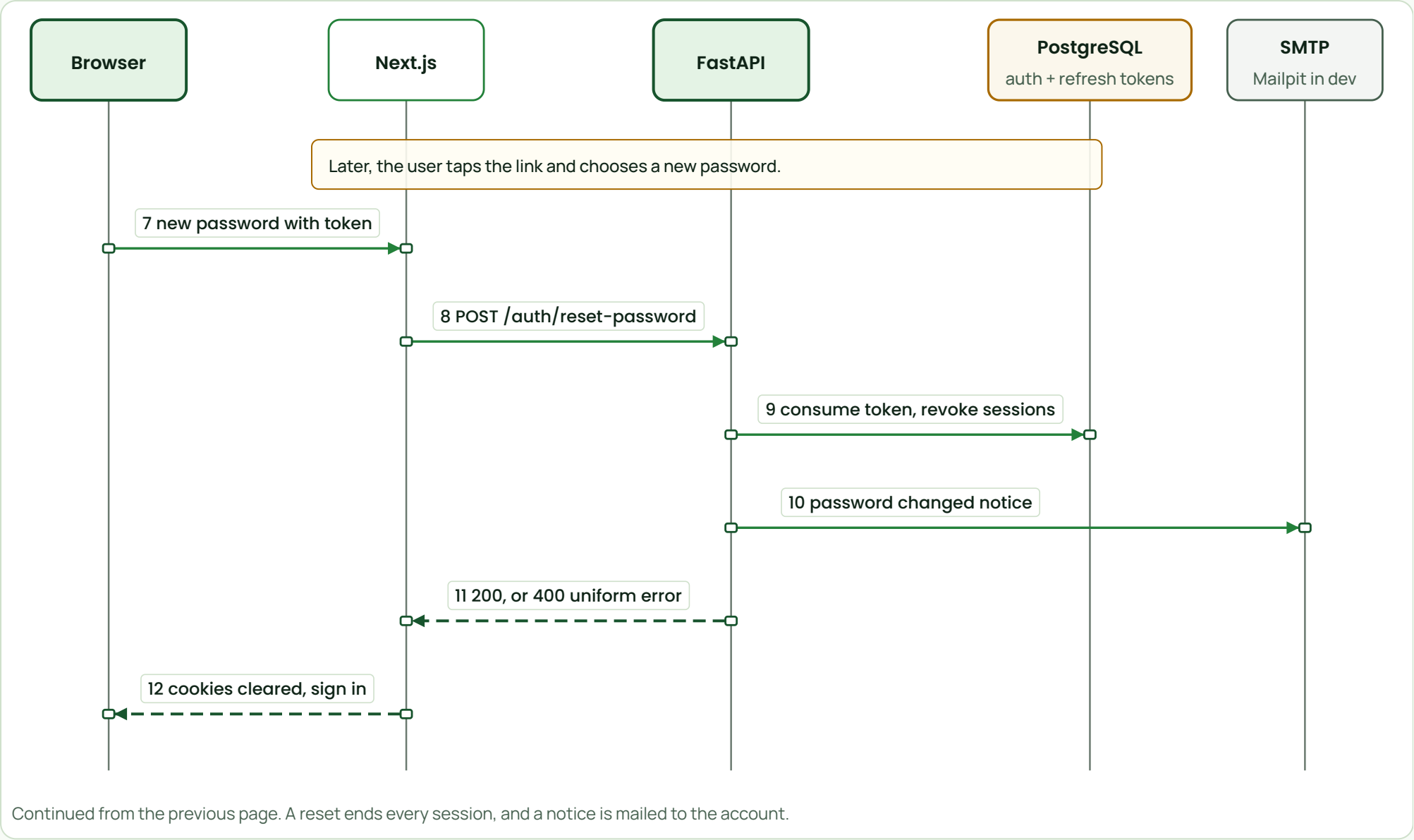
Request

---→

Response

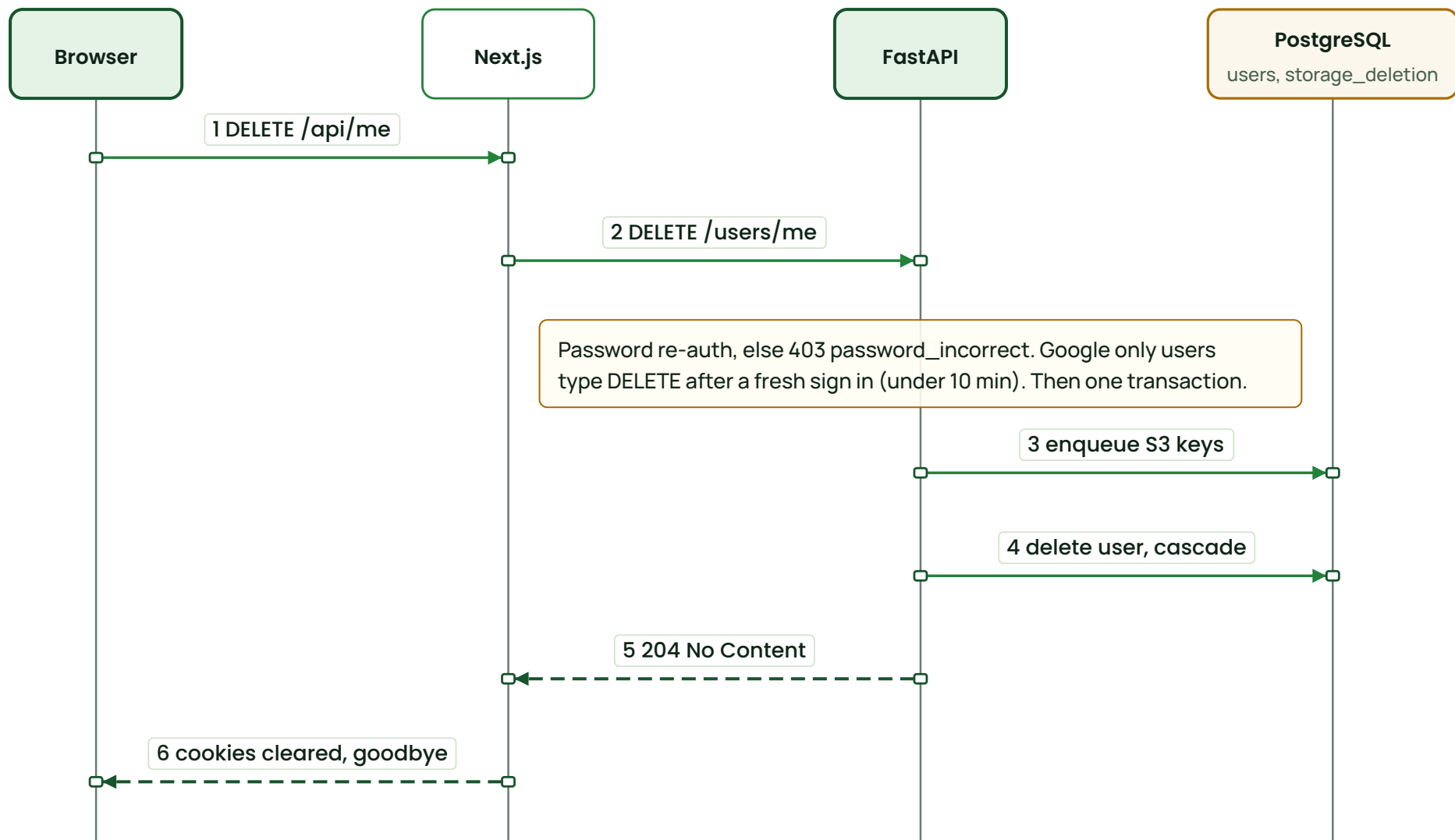
—

Lifeline



Sequence: account delete, part 1

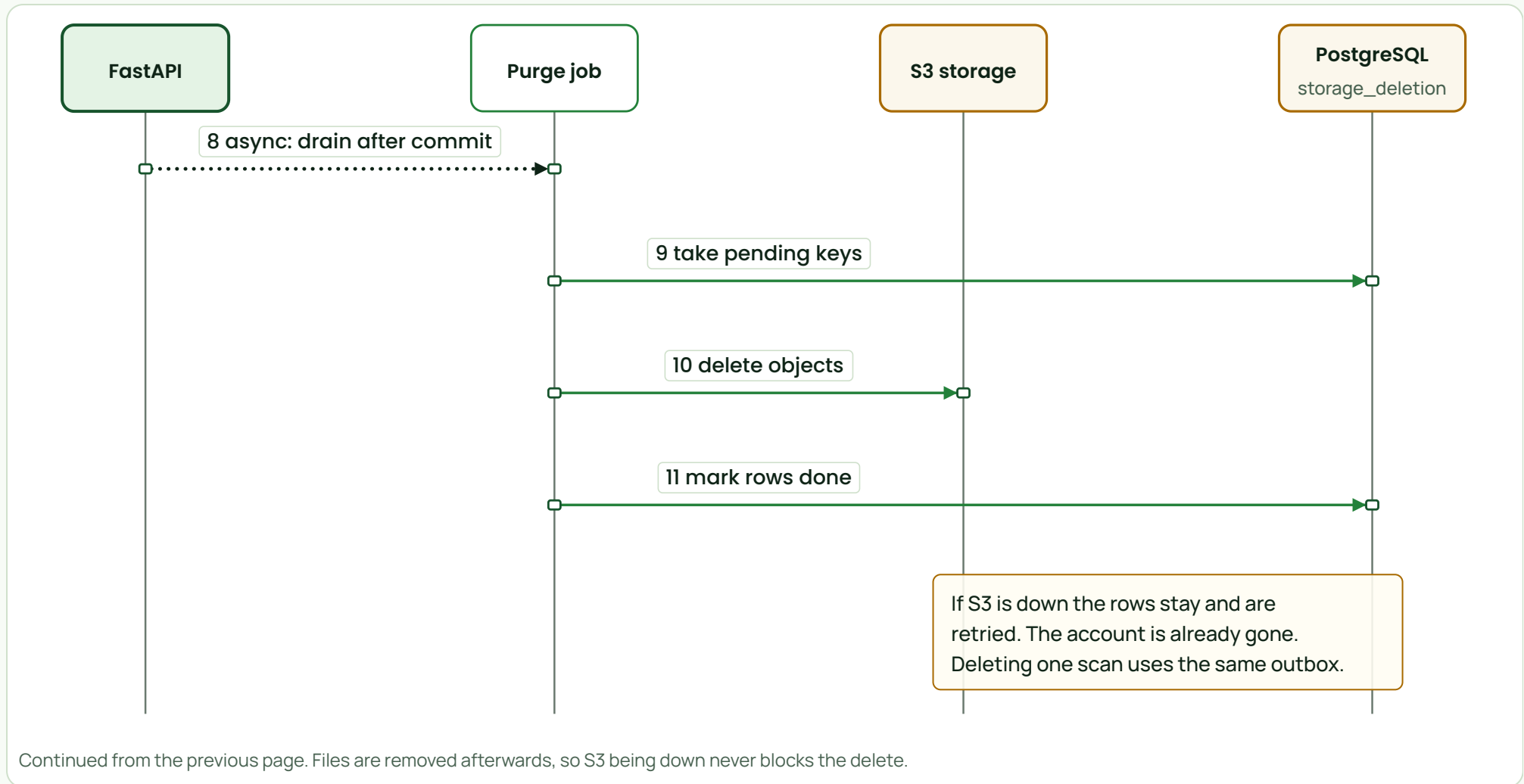
Arrows  Request  Response  Lifeline



The account is deleted in one transaction, together with the queue entries for its files. Continues on the next page.

Sequence: account delete, part 2

Arrows  Request  Async  Lifeline



SECTION 06

REST API

All endpoints live under `/api/v1`. The browser never calls them directly: Next.js route handlers and server components proxy every call.

Conventions

ENVELOPE

One response shape

`{success, data, error:{code, message, details, request_id}}`. Messages are static text, never the raw exception.

TRACING

Request id

Every response carries `X-Request-Id`. The same id appears in the error body and in the structured JSON logs.

AUTH

Bearer access token

Protected calls send the 15 minute access token. Next.js attaches it from the `httpOnly` cookie.

OWNERSHIP

404, never 403

Another user's scan answers 404. Every scan query is scoped by `user_id`.

Next.js route handlers

These are the only URLs the browser calls. State changing handlers run `assertSameOrigin` first. Uploads stream through with a body of at most 8 MiB.

Handler	Purpose
<code>/api/auth/register</code> <code>/api/auth/login</code> <code>/api/auth/logout</code> <code>/api/auth/refresh</code>	Call FastAPI, then set or clear the <code>leafy_at</code> and <code>leafy_rt</code> cookies.
<code>/api/auth/verify-email</code> <code>/api/auth/resend-verification</code> <code>/api/auth/forgot-password</code> <code>/api/auth/reset-password</code>	Email token flows. Reset and account delete clear cookies.
<code>/api/auth/google</code> <code>/api/auth/google/callback</code>	Start the OAuth redirect, then verify <code>state</code> and hand the code to FastAPI.
<code>/api/me</code> <code>/api/me/password</code>	Profile, password and account delete.
<code>/api/scans</code> <code>/api/scans/[id]</code>	Create, poll, list, retry, delete and feedback for scans.

Auth endpoints

Endpoint	Auth	Request	Response	Errors	Limits
POST /auth/register	No	Name, email, password	201 session	409 email_taken 422 validation_error	5 per hour
POST /auth/login	No	Email, password	Session	401 invalid_credentials uniform	10 per 15 min per ip and email
POST /auth/refresh	No	{refresh_token}	New session	401 refresh_invalid 401 refresh_reuse_detected	None
POST /auth/logout	No	{refresh_token}	Success, idempotent	None	None
POST /auth/verify-email	No	Token	Email verified	400 token_invalid_or_expired	None
POST /auth/resend-verification	Yes	None	Success	409 already_verified 429 rate_limited	Cooldown 60 s
POST /auth/forgot-password	No	Email	Always 200	None, never reveals if the account exists	None
POST /auth/reset-password	No	Token, new password	Success, all sessions end	400 token_invalid_or_expired	None
POST /auth/google	No	{code, code_verifier, nonce, redirect_uri}	Session	400 google_auth_failed 400 google_email_unverified	10 per 15 min per ip

Session means the `AuthSessionOut` object with the access and refresh tokens. Reset and verify tokens are single use and return one uniform error for unknown, used or expired.

User endpoints

Endpoint	Auth	Request	Response	Errors
GET /users/me	Yes	None	Profile, including auth_methods	401 not_authenticated
PATCH /users/me	Yes	First and last name	Updated profile	422 validation_error
POST /users/me/password	Yes	current_password only if the user has one, plus new password (10 to 128 characters, not the email, not common)	Success	403 password_incorrect 422 validation_error
DELETE /users/me	Yes	Password, or DELETE typed after a fresh sign in for Google only users	Account and all scans removed	403 password_incorrect

Catalog and health endpoints

Endpoint	Auth	Request	Response	Errors
GET /plants?q=	Public	Optional search text	List of plants	422 validation_error
GET /plants/{plant}	Public	Plant slug	Plant with growth fields and diseases	404 not_found
GET /plants/{plant}/diseases/{disease}	Public	Plant and disease slugs (disease slugs are unique per plant)	Disease with entries and images in one query	404 not_found
GET /diseases?q=&plant=	Public	Optional text and plant filter	List of diseases	422 validation_error
GET /healthz, GET /readyz	No	None	Alive, and ready when the database answers	503 service_unavailable

Scan endpoints

Endpoint	Auth	Request	Response	Errors
POST <code>/scans</code>	Yes, verified	Multipart <code>image</code> Limit 10/min, 100/day (see Rate limits)	202 with <code>id</code> , <code>status</code> (<code>processing</code>) and <code>stage</code>	403 <code>email_not_verified</code> 413 <code>payload_too_large</code> 415 <code>unsupported_media_type</code> 422 <code>invalid_image</code> 429 <code>rate_limited</code> 409 <code>scan_quota_exceeded</code> 503 <code>storage_unavailable</code>
GET <code>/scans/{id}</code>	Yes	None Poll every 1 s, back off to 2 s	Scan with <code>status</code> , <code>stage</code> , <code>failure_code</code>	404 <code>not_found</code>
POST <code>/scans/{id}/retry</code>	Yes	None	Prediction re-runs on the stored image	409 <code>scan_not_failed</code>
GET <code>/scans</code>	Yes	Keyset <code>cursor</code> , <code>limit</code> 1 to 50, filters <code>status</code> , <code>verdict</code> , <code>plant</code>	Page of scans with live status badges	400 <code>invalid_cursor</code>
GET <code>/scans/stats</code>	Yes	None	Totals, last 30 days, verdict split, top 5 diseases (completed only)	None
DELETE <code>/scans/{id}</code>	Yes	None	Scan removed, image purge queued	404 <code>not_found</code>
PUT <code>/scans/{id}/feedback</code>	Yes	<code>{is_correct, correct_plant?, correct_disease?, comment?}</code> , comment up to 300 characters	Idempotent upsert. The team exports it as CSV with <code>python -m app.tools.export_feedback</code>	409 <code>scan_not_completed</code> 404 <code>not_found</code>
DELETE <code>/scans/{id}/feedback</code>	Yes	None	Feedback cleared	404 <code>not_found</code>

Error codes

Every error uses the envelope. The status is the HTTP status. Codes marked Next only never come from FastAPI. The first table covers sign in, tokens, account and upload errors. The second covers server and scan state errors.

Code	HTTP	Meaning
<code>token_invalid_or_expired</code>	400	Email link is unknown, used or expired. One uniform answer.
<code>not_authenticated</code>	401	No valid access token on a protected call.
<code>token_expired</code>	401	Access token expired. Next.js refreshes and retries once.
<code>invalid_token</code>	401	Access token is malformed or fails verification.
<code>invalid_credentials</code>	401	Wrong email or password. Same message for both, and for Google only accounts.
<code>refresh_invalid</code>	401	Refresh token unknown, expired or revoked.
<code>refresh_reuse_detected</code>	401	A rotated token was replayed after the 10 s grace window. The whole family is revoked.
<code>email_not_verified</code>	403	Only <code>POST /scans</code> is blocked for unverified users.
<code>password_incorrect</code>	403	Current password is wrong on change or delete.
<code>csrf_failed</code> (Next only)	403	Origin check failed on a state changing handler.
<code>email_taken</code>	409	Registering with an email that already has an account.
<code>already_verified</code>	409	Resend requested for a verified email.
<code>google_auth_failed</code>	400	Generic Google failure. Details are logged.
<code>google_email_unverified</code>	400	Google reports the email as unverified.
<code>invalid_state</code> (Next only)	400	OAuth <code>state</code> does not match the cookie.
<code>not_found</code>	404	Missing resource, or a scan owned by someone else.
<code>payload_too_large</code>	413	Body over the 8 MiB cap.
<code>unsupported_media_type</code>	415	Upload is not JPEG, PNG or WebP.
<code>validation_error</code>	422	Request fields fail validation.
<code>invalid_image</code>	422	Image fails decode, pixel cap of 40M, or sniff checks.
<code>invalid_cursor</code>	400	Keyset cursor is not valid.
<code>rate_limited</code>	429	Too many attempts. Sends <code>Retry-After</code> .

Code	HTTP	Meaning
<code>internal_error</code>	500	Unexpected failure. Details are logged, not returned.
<code>prediction_failed</code>	502	Prediction errored or timed out. Stored as the scan <code>failure_code</code> .
<code>ml_unavailable</code>	503	The stub raised <code>NotImplementedError</code> . Shown as "Analysis isn't available right now".
<code>storage_unavailable</code>	503	S3 could not store the image.
<code>scan_not_failed</code>	409	Retry on a scan that has not failed.
<code>scan_not_completed</code>	409	Feedback sent before the scan completed.
<code>scan_quota_exceeded</code>	409	Stored scan cap reached. The UI asks to delete old scans.

Additions in the built API, not in the original plan: `service_unavailable` (503), `method_not_allowed` (405) and `reauth_required` (403, a Google only user needs a fresh sign in before deleting). The 502 for `prediction_failed` and 409 for `scan_quota_exceeded` are also build decisions; the plan lists the codes without statuses.

Rate limits

Action	Limit	Scope
Register	5 per hour	Per ip
Login	10 per 15 minutes	Per ip and email
Google sign in	10 per 15 minutes	Per ip
Resend verification	1 per 60 seconds	Per user (cooldown)
Create scan	10 per minute, 100 per day	Per user
Stored scans	500 by default (<code>SCAN_QUOTA</code>)	Per user, then <code>scan_quota_exceeded</code>
Upload size	8 MiB	Next.js, ASGI middleware and the streaming read

How limits behave

Limits use `slowapi` in memory for v1, so they reset when the API restarts. Redis is the upgrade path. A limited call answers 429 `rate_limited` with `Retry-After`, and the UI shows a countdown.

SECTION 07

DATABASE

PostgreSQL 16 with the `citext` and `pg_trgm` extensions. Twelve tables: four for accounts and tokens, five for the catalog, two for scans and one outbox. The diagram on the next page shows how they connect.

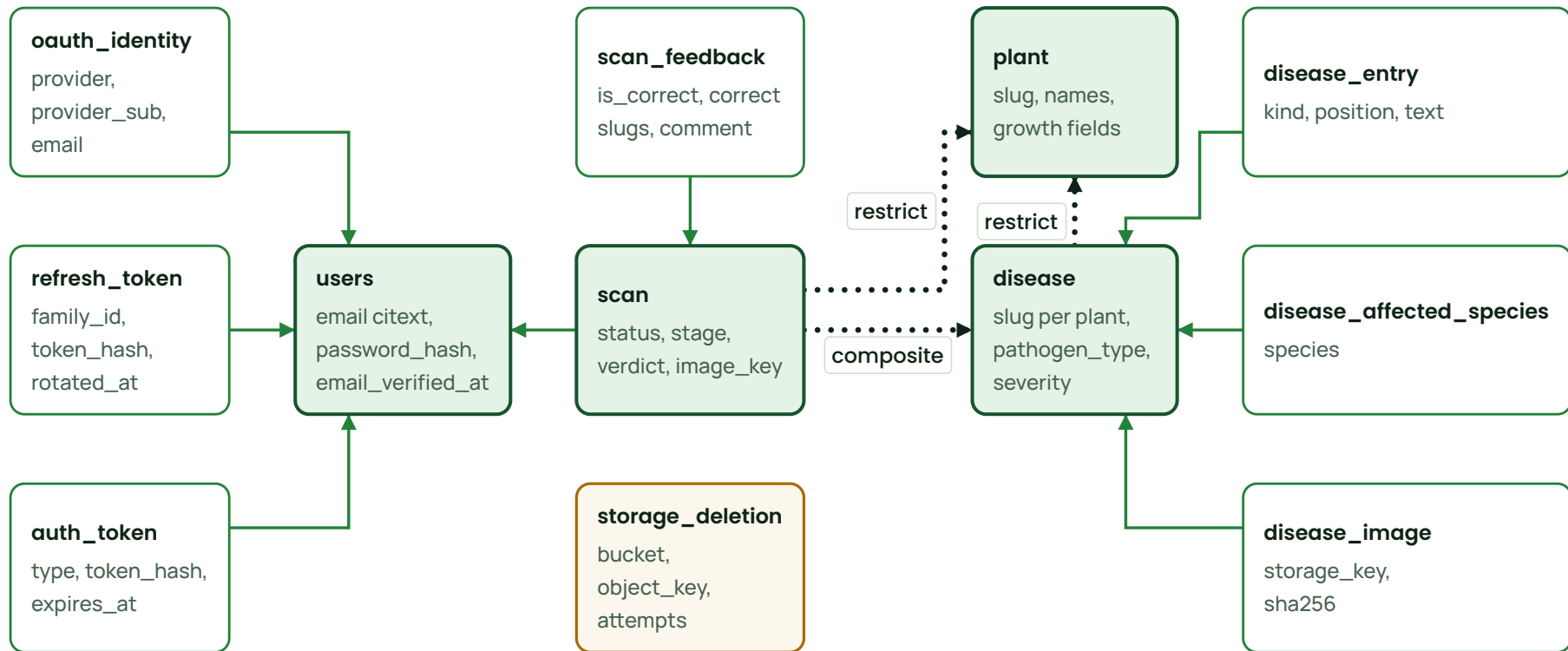
Table map

Table	Group	Purpose
<code>users</code>	Accounts	One row per person: email, optional password hash.
<code>oauth_identity</code>	Accounts	Links a user to a Google identity by its stable <code>sub</code> .
<code>refresh_token</code>	Tokens	Rotating refresh tokens grouped in families, stored hashed.
<code>auth_token</code>	Tokens	Single use verify and reset tokens, stored hashed.
<code>plant</code>	Catalog	13 plants with growth conditions and an image.
<code>disease</code>	Catalog	27 diseases, each belonging to one plant.
<code>disease_entry</code>	Catalog	Ordered symptom, treatment and prevention text.
<code>disease_affected_species</code>	Catalog	Species a disease affects.
<code>disease_image</code>	Catalog	Reference photos kept in natural color.
<code>scan</code>	Scans	One uploaded leaf photo, its progress state and its verdict.
<code>scan_feedback</code>	Scans	The user's answer to "Was this result correct?".
<code>storage_deletion</code>	Outbox	Image keys waiting to be deleted from S3.

Deletion rules

Rule	Effect
Cascade	Deleting a user removes scans, feedback, identities and tokens, and queues the S3 keys in <code>storage_deletion</code> .
Restrict	A plant or disease that a scan uses cannot be deleted.
Pairing	Composite key (<code>disease_id</code> , <code>plant_id</code>) stops a disease pairing with the wrong plant.
Quota	Over the per user cap (default 500, <code>SCAN_QUOTA</code>) the API returns <code>scan_quota_exceeded</code> .

Entity relationship diagram



Arrows **Solid: ON DELETE CASCADE** **Dotted: restrict or composite** **Boxes** **Outbox, no foreign key**

restrict = ON DELETE RESTRICT. **composite** = key on (disease_id, plant_id).

Catalog

The catalog holds 13 plants and 27 diseases from the DAHON seed. Blueberry and Soybean exist as plants with zero diseases.

Seed extraction

The one off tool `tools/extract_dahon.py` turns the DAHON MySQL dump into the committed seed files. These rules decide what ends up in the catalog.

Rule	Effect
Plant prefix split	The DAHON <code>disease</code> table has no plant foreign key. The name <code>Plant: Disease</code> is split on the first <code>:</code> and the prefix is mapped to a plant. An unmapped prefix is a hard failure.
Alias map	<code>Pepper Bell</code> becomes Bell Pepper, <code>Corn (Maize)</code> becomes Corn and <code>Cherry (including sour)</code> becomes Cherry.
Pseudo entries dropped	The diseases <code>Healthy</code> and <code>Unknown</code> and the <code>Unknown</code> plant are removed. That leaves 13 plants and 27 diseases.
Typo fix	<code>Haunglongbing</code> is corrected.
Raspberry not seeded	A <code>raspberry</code> prediction resolves to <code>unknown</code> . Blueberry and Soybean are seeded as plants with zero diseases.
Sample images	Healthy and unknown sample images go to the Next app for the verdict screens. Catalog images are resized to 1280 px and deduped by hash.
Duplicated text	Treatment text that is repeated under prevention is reported for manual review, not deleted.
Outputs	<code>labels.json</code> (the contract <code>predict()</code> must honor), <code>slug_overrides.yaml</code> and <code>cleaning_report.md</code> .
Loader	<code>python -m app.seeds</code> runs as one transaction, is idempotent and never deletes. <code>--check</code> asserts 13 plants and 27 diseases.

plant

Column	Type	Constraints and notes
<code>id</code>	integer	Primary key, auto increment.
<code>slug</code>	varchar(80)	Not null, unique.
<code>common_name</code>	varchar(100)	Not null.
<code>scientific_name</code>	text	Nullable.
<code>family</code> , <code>plant_type</code>	varchar(100), varchar(60)	Nullable.
<code>soil_type</code> , <code>light</code> , <code>water_needs</code> , <code>temperature</code>	text	Nullable. Growth fields merged 1:1 into the plant.
<code>image_key</code> , <code>image_alt</code>	text	Nullable. Public bucket key and alt text.

disease

Column	Type	Constraints and notes
<code>id</code>	integer	Primary key, auto increment.
<code>plant_id</code>	integer	Not null, FK to <code>plant.id</code> , ON DELETE RESTRICT, indexed.
<code>slug</code>	varchar(120)	Not null. Unique per plant: <code>(plant_id, slug)</code> .
<code>name</code>	text	Not null. Full name for the two line detail title.
<code>display_name</code>	varchar(60)	Nullable. Short name for single line cards.
<code>cause</code>	text	Nullable.
<code>pathogen_type</code>	enum	Nullable. Parsed from the cause text.
<code>pathogen_name</code> , <code>severity</code>	text	Nullable.

Unique `(id, plant_id)`, the target of the composite foreign key from `scan`.

Catalog tables, continued

disease_entry

Column	Type	Constraints and notes
<code>id</code>	integer	Primary key.
<code>disease_id</code>	integer	Not null, FK to <code>disease.id</code> , cascade, indexed.
<code>kind</code>	enum	Not null. <code>symptom</code> , <code>treatment</code> or <code>prevention</code> .
<code>position</code>	smallint	Not null, CHECK position at least 0. Unique (<code>disease_id</code> , <code>kind</code> , <code>position</code>).
<code>text</code>	text	Not null. NFC normalized, deduped per kind.

disease_affected_species

Column	Type	Constraints and notes
<code>id</code>	integer	Primary key.
<code>disease_id</code>	integer	Not null, FK to <code>disease.id</code> , cascade, indexed.
<code>species</code>	varchar(150)	Not null. Unique (<code>disease_id</code> , <code>species</code>).

disease_image

Column	Type	Constraints and notes
<code>id</code>	integer	Primary key.
<code>disease_id</code>	integer	Not null, FK to <code>disease.id</code> , cascade, indexed.
<code>storage_key</code>	text	Not null. Key in the public <code>leafy-catalog</code> bucket.
<code>sha256</code>	varchar(64)	Not null. Unique (<code>disease_id</code> , <code>sha256</code>) to dedupe.
<code>alt_text</code>	text	Nullable.
<code>position</code>	smallint	Not null, default 0.

How the catalog fits together

Every disease belongs to exactly one plant. Entries, affected species and images belong to one disease, so deleting a disease removes them. A plant or disease that a scan points to is protected by RESTRICT.

Accounts and tokens

users

Column	Type	Constraints and notes
<code>id</code>	uuid	Primary key, generated by the app.
<code>email</code>	citext	Not null, unique, case insensitive.
<code>password_hash</code>	text	Nullable. argon2id hash. Null for Google only users.
<code>first_name</code> , <code>last_name</code>	varchar(100)	Not null, default empty.
<code>email_verified_at</code>	timestamptz	Nullable. Set when the email is verified or Google verified it.
<code>password_changed_at</code>	timestamptz	Nullable.
<code>created_at</code> , <code>updated_at</code>	timestamptz	Not null, default now.

oauth_identity

Column	Type	Constraints and notes
<code>id</code>	uuid	Primary key.
<code>user_id</code>	uuid	Not null, FK to <code>users.id</code> , cascade, indexed.
<code>provider</code>	enum	Not null. Value <code>google</code> .
<code>provider_sub</code>	varchar(255)	Not null. Google's stable <code>sub</code> , the identity key.
<code>email</code>	citext	Not null.
<code>created_at</code>	timestamptz	Not null, default now.

Unique (`provider`, `provider_sub`). A user must keep a password or an identity, enforced in the service layer.

auth_token

Column	Type	Constraints and notes
<code>id</code>	uuid	Primary key.
<code>user_id</code>	uuid	Not null, FK to <code>users.id</code> , cascade, indexed.
<code>type</code>	enum	Not null. <code>verify_email</code> or <code>reset_password</code> .
<code>token_hash</code>	varchar(64)	Not null, unique. sha256 of a random 256 bit token.
<code>created_at</code>	timestamptz	Not null, default now.
<code>expires_at</code>	timestamptz	Not null. Verify 24 h, reset 1 h.
<code>used_at</code>	timestamptz	Nullable. Single use, consumed atomically.

Refresh tokens and outbox

refresh_token

Column	Type	Constraints and notes
<code>id</code>	uuid	Primary key.
<code>user_id</code>	uuid	Not null, FK to <code>users.id</code> , cascade, indexed.
<code>family_id</code>	uuid	Not null, indexed. One family per sign in.
<code>token_hash</code>	varchar(64)	Not null, unique. sha256 of the opaque token.
<code>created_at</code> , <code>expires_at</code>	timestamptz	Not null. Expiry is the lesser of 30 days and the family cap.
<code>family_expires_at</code>	timestamptz	Not null. The 90 day family cap.
<code>rotated_at</code>	timestamptz	Nullable. Starts the 10 s grace window.
<code>replaced_by</code>	uuid	Nullable, self FK to <code>refresh_token.id</code> , SET NULL.
<code>revoked_at</code>	timestamptz	Nullable.
<code>revoked_reason</code>	varchar(40)	Nullable.
<code>ip</code>	varchar(64)	Nullable.
<code>user_agent</code>	varchar(300)	Nullable.

Only the sha256 hash of a refresh, verify or reset token is stored, so a database leak does not hand out working tokens.

storage_deletion

Column	Type	Constraints and notes
<code>id</code>	integer	Primary key, auto increment.
<code>bucket</code>	varchar(100)	Not null.
<code>object_key</code>	text	Not null. No foreign key, so rows outlive the deleted user.
<code>attempts</code>	integer	Not null, default 0.
<code>last_error</code>	varchar(300)	Nullable.
<code>created_at</code>	timestamptz	Not null.
<code>next_attempt_at</code>	timestamptz	Not null. Index (<code>next_attempt_at</code> , <code>id</code>).

A drain job, also run after commit, deletes objects in chunks and retries failures, so deletion survives an S3 outage.

Scans and feedback

scan

Column	Type	Constraints and notes
<code>id</code>	uuid	Primary key, generated by the app.
<code>user_id</code>	uuid	Not null, FK to <code>users.id</code> , ON DELETE CASCADE.
<code>image_key</code>	text	Not null, unique. Server generated <code>{user}/{scan}.jpg</code> .
<code>status</code>	enum	Not null. <code>processing</code> , <code>completed</code> or <code>failed</code> .
<code>stage</code>	enum	Nullable. <code>validating</code> , <code>analyzing</code> or <code>saving</code> .
<code>failure_code</code>	varchar(40)	Nullable. For example <code>prediction_failed</code> or <code>ml_unavailable</code> .
<code>verdict</code>	enum	Nullable until completed. <code>disease</code> , <code>healthy</code> or <code>unknown</code> .
<code>plant_id</code>	integer	Nullable. FK to <code>plant.id</code> , ON DELETE RESTRICT.
<code>disease_id</code>	integer	Nullable. Composite FK (<code>disease_id</code> , <code>plant_id</code>) to <code>disease(id, plant_id)</code> , RESTRICT.
<code>confidence</code>	varchar(20)	Nullable. Text such as "0.93".
<code>raw_prediction</code>	jsonb	Nullable. The prediction as returned.
<code>attempt</code>	integer	Not null, default 0. Added in migration 0003. Bumped on every retry so a stale worker cannot touch a newer run.
<code>created_at</code> , <code>updated_at</code>	timestamptz	Not null. <code>updated_at</code> feeds the stuck scan janitor.

CHECK constraints keep status, stage and verdict consistent (`state_consistent`, `verdict_consistent`) and require a plant when a disease is set (`disease_needs_plant`). Indexes: keyset (`user_id`, `created_at` DESC, `id` DESC) and (`status`, `updated_at`).

scan_feedback

Column	Type	Constraints and notes
<code>scan_id</code>	uuid	Primary key and FK to <code>scan.id</code> , ON DELETE CASCADE.
<code>is_correct</code>	boolean	Not null.
<code>correct_plant_slug</code>	varchar(80)	Nullable. Validated against the catalog.
<code>correct_disease_slug</code>	varchar(120)	Nullable. Validated against the catalog.
<code>comment</code>	varchar(300)	Nullable. CHECK length at most 300.
<code>created_at</code> , <code>updated_at</code>	timestamptz	Not null.

Foreign keys at a glance

Child column	References	On delete
<code>oauth_identity.user_id</code>	<code>users.id</code>	Cascade
<code>refresh_token.user_id</code>	<code>users.id</code>	Cascade
<code>refresh_token.replaced_by</code>	<code>refresh_token.id</code>	Set null
<code>auth_token.user_id</code>	<code>users.id</code>	Cascade
<code>scan.user_id</code>	<code>users.id</code>	Cascade
<code>scan_feedback.scan_id</code>	<code>scan.id</code>	Cascade
<code>scan.plant_id</code>	<code>plant.id</code>	Restrict
<code>scan (disease_id, plant_id)</code>	<code>disease (id, plant_id)</code>	Restrict
<code>disease.plant_id</code>	<code>plant.id</code>	Restrict
<code>disease_entry.disease_id</code>	<code>disease.id</code>	Cascade
<code>disease_affected_species.disease_id</code>	<code>disease.id</code>	Cascade
<code>disease_image.disease_id</code>	<code>disease.id</code>	Cascade

Migrations

Revision	What it does
0001	Initial schema: extensions, catalog, accounts, tokens, scans, feedback and the deletion outbox.
0002	Index on <code>storage_deletion (next_attempt_at, id)</code> , so the drain job does not scan the table.
0003	Adds <code>scan.attempt</code> (integer, default 0) for safe retries.

SECTION 08

DESIGN AND BRANDING

Leafy looks like a lab inside a forest: calm, precise and reassuring. Below are the real logo files, colour tokens, type rules and components. The brand, tokens, logos, splash and cursor are built.

Brand foundation

NAME Leafy Sentence case in copy. LEAFY in Josefin Sans Light capitals for the wordmark and display titles. DAHON is credited as origin only: dahon means leaf in Filipino.	POSITION A plant doctor in your pocket Snap a leaf, understand the disease, know what to do next. Draft taglines: Know your leaf. Read the leaf. Save the plant. Plant health, understood.	VOICE Calm and precise Short sentences, plain words, active voice. No jargon without a one line explanation, no exclamation marks.
---	--	--

Copy example	Why it works
We found signs of early blight.	States the finding without alarm.
This photo is a little dark. Try daylight and hold the leaf flat.	Says what happened and what to do next.
Treatment: remove affected leaves and avoid wetting the foliage.	Plain action, no jargon.

Brand assets and where they live

Asset	Location (all built unless noted)
Marks, wordmark, lockups	<code>app/public/brand/*.svg</code>
Favicons, touch and PWA icons, share image	<code>app/public/icons/*</code> (16, 32 and 48 px, an SVG favicon that follows <code>prefers-color-scheme</code> , Apple touch icon, maskable 192 and 512 px) and <code>app/public/og/og-image.png</code> (1200 by 630 px)
Custom cursors	<code>app/public/cursors/*.svg</code>
Colour tokens (contrast tested)	<code>app/src/styles/tokens.css</code>
Brand constants and guide	<code>components/brand/</code> , <code>lib/brand.ts</code> (LeafMark3D planned), and the guide at <code>/brand</code> (not indexed)

Logo family

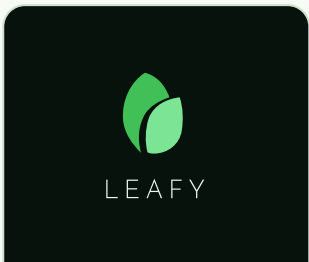
The mark is the DAHON two leaf shape redrawn on a clean grid. These are the built files from `app/public/brand/`.



Horizontal, on dark
Mark plus wordmark



Horizontal, on light
Deep green mark



Stacked, on dark
Narrow spaces



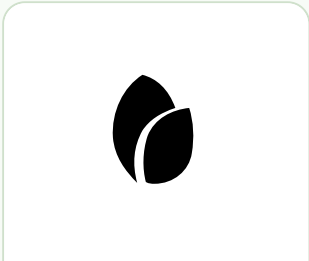
Stacked, on light
Print and email



Mark, on dark
Light green



Mark, on light
Deep green



Mono black
One colour print



Mono white
Busy imagery

Rule	Value
Wordmark	LEAFY in Josefin Sans Light, capitals, tracking about 0.18em.
Clear space	Equal to the height of the mark's small leaf on every side.
Minimum size	Mark 20 px. Lockup 96 px wide.
Two tone	Brand green with a lighter inner leaf. The same paths feed the 3D extrusion (<code>LeafMark3D</code> , planned).
Never	Stretch the logo, recolour outside the variants, or place it on a busy background (use the mono versions).

Minimum size

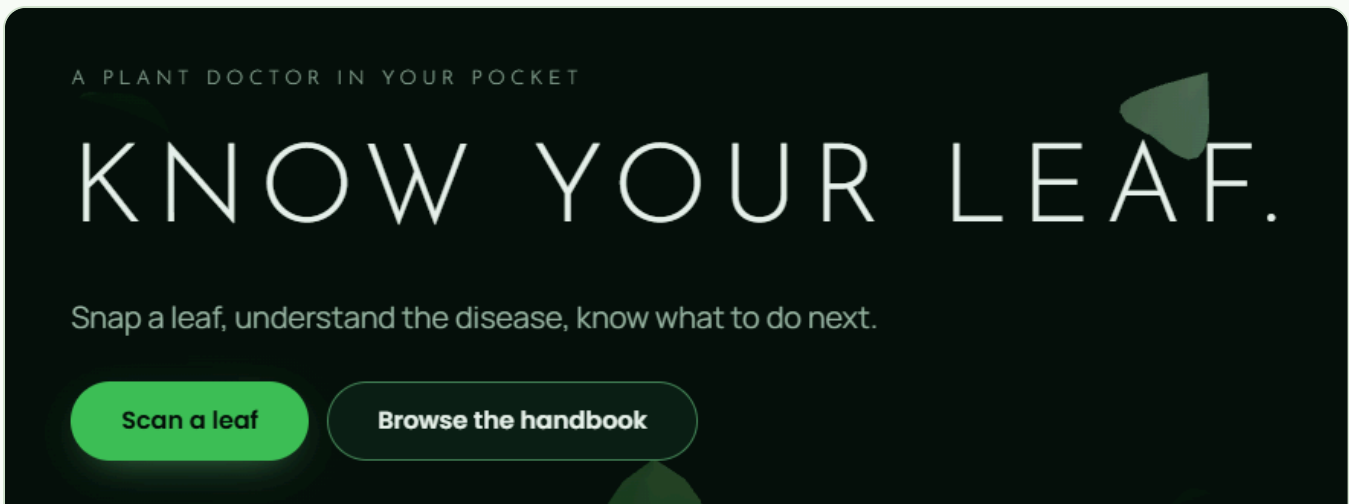


Mark at 20 px, shown at 100%



Lockup at 96 px wide, shown at 100%

Brand in use



Landing hero, dark

Headline Know your leaf, then the line Snap a leaf, understand the disease, know what to do next, and two buttons. The green 3D leaf sits to the right of this crop. Built screenshot, left part of a 1440 by 900 page.



Splash

The glowing mark above the LEAFY wordmark on a soft green glow, shown on every full page load. Built screenshot at 1440 by 900, trimmed slightly at the top and bottom.

Colour tokens

CSS variables on `:root` and `[data-theme=light|dark]`. Dark is the default, the system theme is followed, and the header toggle overrides it.

● Dark theme (default)

Page
#06120b
--bg

Raised
#0b1d12
--bg-raised

Surface
#10281a
--surface

Border
#1e3d2b
--border

Text
#e9f4ec
--text

Muted
#9bb8a6
--text-muted

Brand
#40c057
--brand

Deep
#1f6b3a
--brand-deep

Glow
#7be495
--brand-glow

Accent
#ffb300
--accent

Info
#5cc8d0
--info

Danger
#ff6b6b
--danger

○ Light theme

Page
#f5faf4
--bg

Raised
#ffffff
--bg-raised

Surface
#eaf3e8
--surface

Border
#cfe0cb
--border

Text
#0e2316
--text

Muted
#46604f
--text-muted

Brand
#23813a
--brand

Deep
#17542d
--brand-deep

Glow
#40c057
--brand-glow

Accent
#955d00
--accent

Info
#0f7480
--info

Danger
#c0362c
--danger

Tokens	Role
Page, raised, surface, border	Page, raised sections, cards and panels (glass at 70% with blur over 3D, solid elsewhere).
Text, muted	Primary and secondary text.
Brand, deep, glow	Leaf green (matches the cursor), the DAHON logo family green, and 3D and scan beam highlights. Gradients stay inside the green family.
Accent, info, danger	Amber is the single accent for secondary actions and warnings, used sparingly. Info and danger are for notices and errors.

Contrast ratios

Ratios are computed from the hex values with the WCAG relative luminance formula. AA needs 4.5 for normal text (column Text) and 3.0 for large text and interface parts (column Large). A script in CI repeats this check.

Sample	Pair	Ratio	Text 4.5	Large 3.0	Used for
	Dark: text on background	16.94	✓ Pass	✓ Pass	Body text
	Dark: muted on background	8.92	✓ Pass	✓ Pass	Captions
	Dark: muted on surface	7.31	✓ Pass	✓ Pass	Card captions
	Dark: brand on background	8.09	✓ Pass	✓ Pass	Links, labels
	Dark: background on brand	8.09	✓ Pass	✓ Pass	Primary button text
	Dark: accent on background	10.65	✓ Pass	✓ Pass	Warnings
	Dark: info on background	9.67	✓ Pass	✓ Pass	Notices
	Dark: danger on background	6.89	✓ Pass	✓ Pass	Errors
	Light: text on background	15.63	✓ Pass	✓ Pass	Body text
	Light: muted on background	6.52	✓ Pass	✓ Pass	Captions
	Light: brand on background	4.65	✓ Pass	✓ Pass	Links, labels
	Light: white on brand	4.91	✓ Pass	✓ Pass	Primary button text
	Light: info on background	5.19	✓ Pass	✓ Pass	Notices
	Light: danger on background	5.22	✓ Pass	✓ Pass	Errors
	Light: accent on background	5.17	✓ Pass	✓ Pass	Warnings, secondary actions
	Dark: on accent on accent fill	10.41	✓ Pass	✓ Pass	Amber button text
	Light: on accent on accent fill	5.46	✓ Pass	✓ Pass	Amber button text

One value changed from the plan

The plan started the light accent at `#a86a00`, which reaches only 4.20 on the light background. The built token is `#955d00` (5.17), so every pair in the contrast table passes AA. The CI script `tokens.contrast.test.ts` guards it.

Severity scale

Severity is never carried by colour alone. Each level has a label and an icon. The colours below were computed against the dark background.

 **Healthy**
#40c057

 **Low**
#a3d65c

 **Moderate**
#ffb300

 **High**
#ff7a3d

 **Severe**
#ff4d4f

Level	Colour	On dark bg	On the light theme
Healthy	Green #40c057	8.09	Label and icon in <code>--text</code> ; the green marks only the chip edge.
Low	Lime #a3d65c	11.22	Colour against light is 1.61, so the label and icon carry the meaning.
Moderate	Amber #ffb300	10.65	Colour against light is 1.70, same rule.
High	Orange #ff7a3d	7.37	Colour against light is 2.45, same rule.
Severe	Red #ff4d4f	5.85	Colour against light is 3.09, same rule.

Type specimens

Three families, all self hosted and limited to the weights in use. The same fonts are used inside the Three.js scenes.

Josefin Sans Light
Weight 300, ALL CAPS, wide tracking
Display: hero, page titles, section labels, wordmark

KNOW YOUR LEAF.
ABCDEFGHIJKLMNOPQRSTUVWXYZ 0123456789

Poppins
Weights 500 and 600
Subheadings, buttons, navigation, cards, stats

Early blight on tomato
Scan a leaf · Browse the handbook · 27 diseases · 13 plants
Aa Bb Cc Dd Ee Ff Gg 0123456789 (tabular figures for stats)

Manrope
Weights 400, 500 and 700
Body, forms and disease text

Brown spots with yellow rings appear first on the oldest leaves.
Remove affected leaves and avoid wetting the foliage when you water.
Aa Bb Cc Dd Ee Ff Gg 0123456789

Type scale

Role	Family and weight	Size, fluid <code>cLamp()</code>	Line height	Notes
Display	Josefin Sans 300, caps	About 40 px mobile to 96 px desktop	1.1 to 1.2	One or two lines. Tracking about 0.18em on the wordmark.
H2	Poppins 600	28 to 44 px	1.25	Panel and section titles.
Body	Manrope 400 to 700	16 to 18 px	1.65	Measure 62ch, 1em paragraph spacing.
Stats	Poppins 600	Per component	1.1	Tabular figures so numbers do not jump.

This PDF follows the same rules in print: Josefin Sans Light capitals for section titles, Poppins for headings and labels, Manrope for body. Print sets body at a line height of 1.6; the app uses 1.65, as in the table above.

Paragraph and title rules

Rules from the plan, each shown as a good and a bad example. The bad examples are only here to show what to avoid.

Paragraph shape: measure 62ch, left aligned, never justified

✓ DO

A paragraph keeps a measure of about 62 characters, is left aligned and uses a line height of 1.65. The last line is never a single orphan word.

Short lines and an even left edge are easy to follow.

✗ AVOID

A paragraph that runs across the full width of the screen, is justified and has a tight line height creates uneven gaps and makes it hard to find the start of the next line.

Too wide, justified, tight line height.

Titles: one line, two at most, never three

✓ DO

EARLY BLIGHT
ON TOMATO

Balanced wrap. Cards show a short display name, the detail page shows the full name in two lines.

✗ AVOID

EARLY
BLIGHT
ON
TOMATO

A narrow box forces more than two lines. Playwright fails any h1 or h2 over two lines at 360, 768 and 1440.

Paragraph and title rules, continued

No hyphenation

✓ DO

Plant health, understood. Words move whole to the next line.

hyphens: **none** everywhere, including 3D text.

✗ AVOID

Plant health, under-
stood. A word split with a hyphen at the line end.

No **­**; , no mid word breaks. A seed lint flags them in the data.

Voice: calm, never alarmist

✓ DO

We found signs of early blight. Remove affected leaves and keep the foliage dry.

Plain words, active voice, a next step.

✗ AVOID

Warning!!! Your plant is in serious danger and may die.

Alarmist, exclamation marks, no next step.

More text rules

Also in force	Rule
Wrapping	text-wrap : pretty on paragraphs, balance on headings and short blurbs. Both degrade gracefully where unsupported.
Overflow guard	overflow-wrap : break-word only as a guard, never as a style.
Slugs	URL slugs keep hyphens because they are not displayed text.
Disease photos	Natural colour in a neutral frame, never tinted.

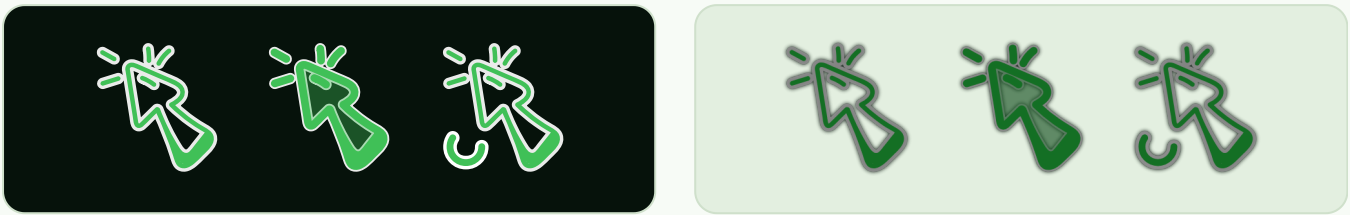
Spacing

4		Icon and label gaps	24		Between cards
8		Tight groups	32		Between blocks
12		Inside cards	48		Maximum gutter, large panels
16		Minimum screen gutter			

Bars are drawn at 1 px = 0.265 mm. The screen gutter follows `clamp(16px, 3vw, 48px)`.

Custom cursor

A green #40c057 outline arrow with a thin white halo, visible on dark 3D scenes and green sections. The hotspot is the arrow tip, about x=10, y=5 at 32 px. Built files: `app/public/cursors/`.



Variant	File	Used on
Default	<code>leaf-cursor.svg</code>	The page, 32 by 32 viewBox.
Pointer	<code>leaf-cursor-pointer.svg</code>	Links, buttons, <code>[role=button]</code> , summary, label for, select. Slightly filled.
Busy	<code>leaf-cursor-busy.svg</code>	Loading and analyzing.
Native kept	none	Text fields keep the I-beam, disabled elements keep not allowed, resize and drag handles keep their own. Touch devices get nothing custom.
Scope	Applied only under <code>(hover: hover)</code> and <code>(pointer: fine)</code> , always with a native fallback after the URL.	

Component notes

Piece	Style
Cards	Thin vein accent, 12 px radius; tilt and glare on hover (see section 9, UI behavior).
Severity badge	Label plus icon plus colour, five levels (see the severity scale).
Panel navigation	Arrows and labelled dots with a visible swipe or arrow keys hint on first view.
Header and nav	Mark, links, sound and theme toggles, user menu; bottom nav on mobile.
Icons	One set (Lucide) at 1.5 px stroke with rounded caps, plus custom leaf, scan and disease icons. Key actions always keep a label.

Shape and surface

12 px

Cards

Radius 12 px, hairline 1 px border

999 px

Pills and buttons

Fully rounded

20 to 28 px

Large panels

Glass over 3D, solid elsewhere

Focus

Glow

Soft green glow instead of heavy shadows

Components

Static renderings of the built component styles on the dark surface. Each is documented with live examples in the `/brand` guide.

Buttons

Scan a leaf

Secondary

Ghost

Disabled

Focus ring

Primary is brand fill with dark text and a glow on focus. Secondary is a glass outline. Pills at 999 px, 44 px touch targets.

Forms

Email
ada@example

Enter an email like name@example.com.

Weak

Fair

Good

Strong

Floating labels, inline validation, show and hide password, strength hint. Input is kept on errors.

Feedback

Your scan is processing

You can leave this page.

That photo did not upload

Check your connection and try again.

Toasts and dialogs on glass. Every error says what happened and what to do next.

Loading and empty

No scans yet

Scan a leaf

Skeletons for slow loads. Empty states are calm and offer one next action.

Progress

Upload

Check

Analyze

Save

Stepper and bar with `role="progressbar"` and a glow head. Reused for slow actions.

Stat tiles

42

Total scans

9

Last 30 days

Poppins 600 with tabular figures. The dashboard also shows the split of healthy, diseased and unknown.

SECTION 09

UI BEHAVIOR

How Leafy moves and responds: panels, cursor, sound, splash, progress and layout. A PDF cannot animate, so each motion is shown as numbered frames, read left to right and top to bottom.

Sideways panels

Every page is a single screen stage. Long content is split into horizontal panels (`SlidePanels`) instead of a long vertical scroll. The base is CSS scroll snap, with framer motion or GSAP on top, so it still works if scripts are late.

Disease detail#overview

OverviewCausesSymptoms

Swipe or use arrows (shown on first view)

1 Rest. One panel fills the stage, labelled dots show where you are, and a hint explains how to move.

Disease detail#overview

OverviewCausesSymptoms

Dot hover previews the title

2 Intent. Pointer over a dot or the arrow: the dot grows and previews the target panel title. Drag gives a rubber band pull.

Disease detail#causes

OverviewCausesSymptoms

Panel sound plays. aria-live says: Causes, 2 of 6

3 Snap. The panel settles in 400 ms, the URL hash updates and screen readers hear the change.

Reduced motion#causes

OverviewCausesSymptoms

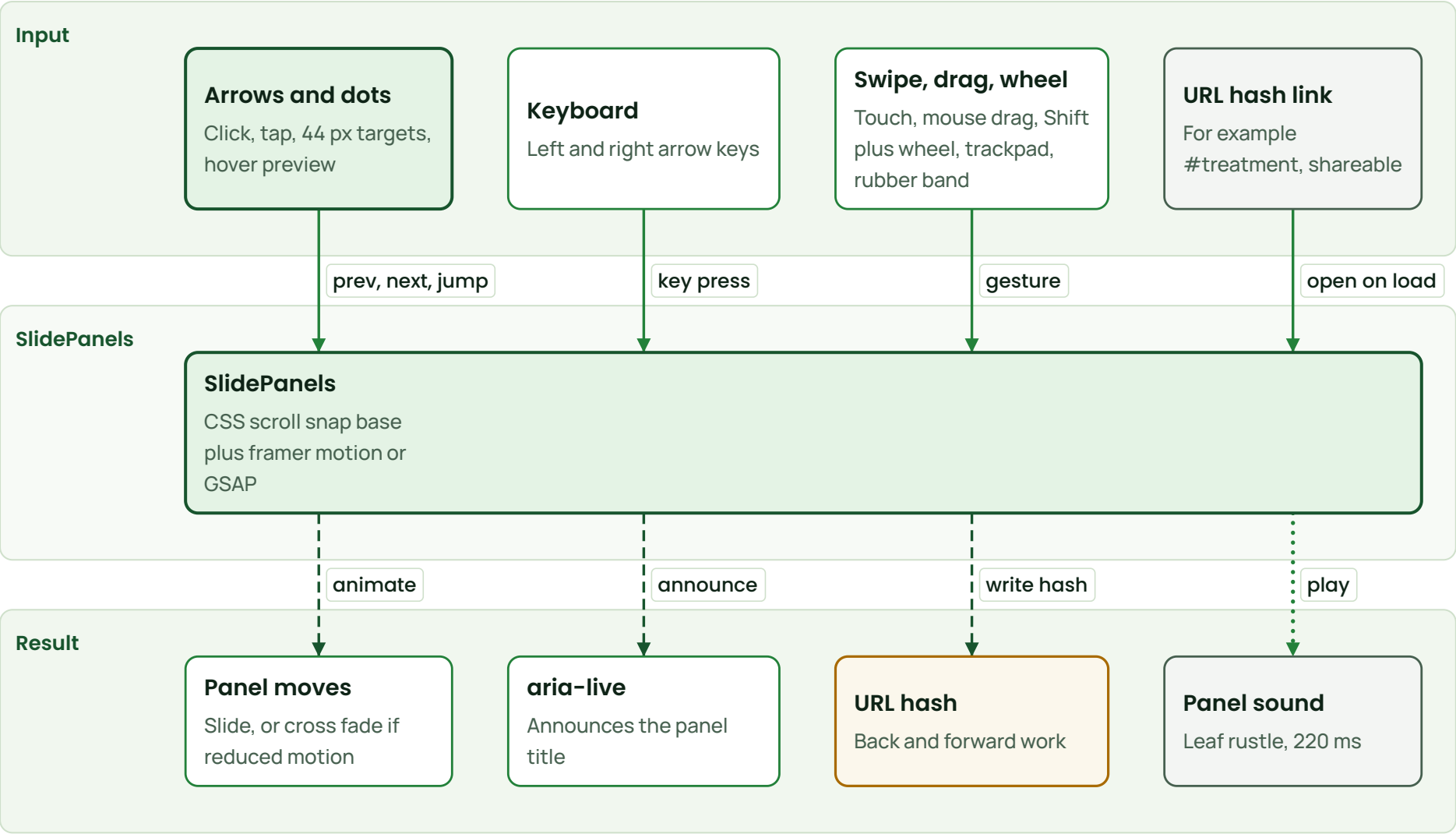
Cross fade instead of a slide

4 Same step with reduced motion on. The sideways slide becomes a plain cross fade.

Panel content budget

Rule	Detail
Short panels	At most 3 short paragraphs or 6 list items.
Overflow	Extra content becomes another panel, never a scrollbar.
Page height test	Playwright: document at most 1.15 times the viewport.

Panel navigation



Every input reaches the same SlidePanels component, so mouse, touch, keyboard and links behave the same way.

Where sideways panels are used

Page	Panels
Landing	Hero, How it works in 3 steps, Plants we cover, Why Leafy, FAQ, final call to action.
Disease detail	Overview, Causes, Symptoms, Treatment, Prevention, Images.
Handbook	Plants carousel, then the chosen plant's diseases as a second row.
Scan history	Card rail with paging and a grid toggle.
Scan result	Result, Causes, Symptoms, Treatment, Prevention, Reference photos.
Dashboard	Rails for stats, top diseases and recent scans.
Account	Profile, Password, Danger zone.
Forms (sign in, register, upload)	Stay on a single screen, no panels.
Legal pages	Exempt from the no scroll rule: sideways section panels with a contents list, or normal reading scroll when a section is long.

Cursor reactions

Everything the cursor touches responds in one calm way. One passive pointer listener, throttled with `requestAnimationFrame`, writes CSS variables, so components never re-render per mouse move.

Variable	Scope	Meaning
<code>--mx</code> , <code>--my</code>	Viewport, set once	Pointer position on the screen.
<code>--px</code> , <code>--py</code>	Per hovered element	Pointer position inside the element, 0 to 1.

Built

The pointer components live in `app/src/components/interaction/`: `PointerProvider`, `Magnetic`, `TiltCard`, `ParallaxImage`, `SpotlightSurface`, `VeinLink` and `Tooltip`. Unit tests cover the math: clamping, damping and reduced motion.

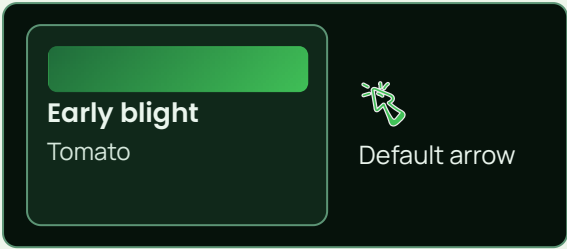
Hover interactions

Element	What happens
Buttons	Magnetic pull of a few pixels toward the cursor, a soft green glow that follows the pointer, a gentle press in, and a ripple or leaf burst on activation. The cursor switches to the pointer variant.
Cards	Plants, diseases, scans and stats. Tilt of at most about 6 degrees with a light glare that tracks the cursor, a lifted glow, a brighter border and an Open hint.
Images	Slow zoom to about 1.04 times and a parallax shift against the cursor, a spotlight vignette, and a caption slide in. Disease reference images get a magnifier lens on desktop.
Text	Links draw a vein shaped underline from the cursor side. Josefin headings get a soft glow and slight letter spacing ease by transform only, so one line titles stay one line. Body paragraphs stay still. Inline terms show a definition tooltip on hover and focus.
Surfaces	A soft radial spotlight follows the cursor over glass panels and the forest background. Vein line dividers light up near the pointer.
3D scenes	Hero, auth and handbook leaves lean and sway toward the cursor with a damped spring. The scan beam can be steered on the preview. The result leaf orbits on drag. Hotspots pulse and show labels.
Navigation	A sliding pill follows the hovered nav item. Panel dots grow and preview the target title. Panels rubber band on drag.

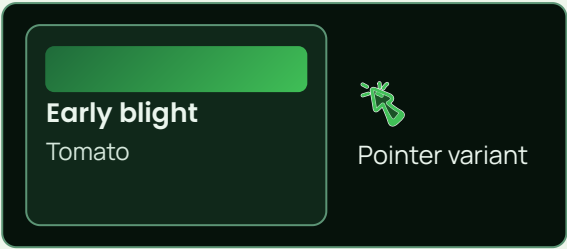
Safeguards for cursor effects

Situation	Rule
Mouse only	Effects run only under <code>(hover: hover)</code> and <code>(pointer: fine)</code> .
Touch	Equivalent press states (scale and glow on touch down), no tilt.
Keyboard	Focus gets the same visual states as hover: glow, lift, underline.
Reduced motion	Tilt, magnetic pull, parallax and sway are off. Only colour and glow changes remain.
Contrast preference	<code>prefers-contrast</code> is honored: borders and text strengthen and decorative glow is dropped. No information is carried by color or 3D alone.
Layout	Effects never move text out of reading position or cause layout shift. Touch targets stay at least 44 px. No effect hides information.
Performance	Limited to visible elements (IntersectionObserver) and capped on pages with many cards, to protect INP and TBT.
Test runs	Lighthouse and <code>?nosplash</code> runs skip decorative effects so scores measure the real page.
Motion tokens	Easing <code>cubic-bezier(0.22, 1, 0.36, 1)</code> . Durations 200 ms micro, 400 ms panels, 700 to 900 ms scene transitions. Nothing bounces or flashes.

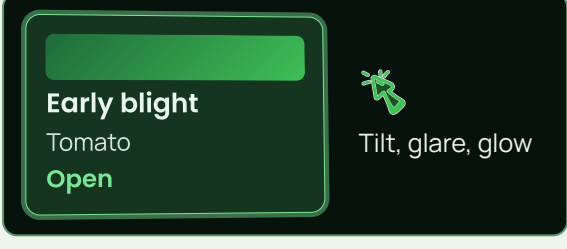
Storyboard: a card under the cursor



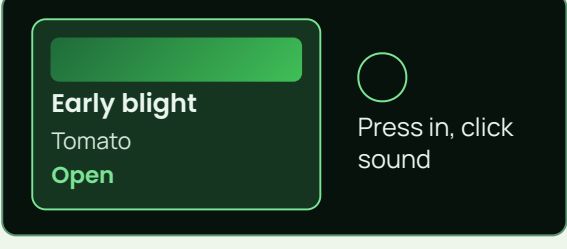
1 Rest. The card is flat and the cursor is the default arrow.



2 Enter. The cursor becomes the pointer variant and a hover sound may play.



3 Hover. Tilt up to about 6 degrees, glare follows the pointer, border brightens, Open appears.



4 Press. The card presses in, a click sound plays and the page opens.

Layout never shifts. Only transform, opacity and filter change on hover, and Playwright checks that hover does not change layout or titles.

Cursor variants by context

Context	Cursor	Status
Page	Default arrow	Built
Links, buttons, interactive	Pointer variant	Built
Draggable panels and rails	Subtle drag variant	Planned, no file yet (optional)
Analyzing	Busy variant	Built
Text fields, disabled elements, resize handles	Native cursors stay: I-beam, <code>not-allowed</code> , resize	Rule
Touch devices	No custom cursor at all	Rule
Hero and auth scenes	Tiny leaves drift in the cursor's wake (instanced particles). No DOM cursor follower. Off for reduced motion and low power devices.	Optional

Sound effects map

Soft, organic UI sounds synthesized with the Web Audio API: oscillators and filtered noise envelopes. There are no audio files and no extra requests. UI sounds stay under 250 ms and scan sounds under 600 ms. Lengths and minimum gaps below are the built values in `app/src/components/sfx/`.

Sound	When it plays	Character	Length	Minimum gap
Click	Buttons, links, toggles (default for <code>button</code> , <code>a[href]</code> , <code>[role=button]</code>)	Soft pluck	100 ms	60 ms
Hover	Primary buttons and <code>[data-sfx-hover]</code> , mouse only	Very quiet tick	50 ms	300 ms
Toggle	Switches such as theme and sound	Two note step	160 ms	80 ms
Panel	Sideways panel slide	Leaf rustle whoosh	220 ms	160 ms
Success	Sign in, saved, scan complete	Small rising chime	240 ms	400 ms
Error	Failed action	Low soft thud, not harsh	220 ms	400 ms
Delete	Delete and undo	Muted	140 ms	200 ms
Scan start	Upload and scan beam begin	Rising sweep	500 ms	800 ms
Scan done	Result arrives	Three note chime	550 ms	800 ms

Brand fit

A small sound palette (pitch family, envelopes, volume levels) is documented in the `/brand` guide so the sounds feel like one product with the visuals and the cursor.

How a sound is chosen

One delegated `pointerdown` and `pointerover` listener on the document maps the element to a sound. Components override with `data-sfx="click|toggle|success|error|none"` or call `sfx.play('success')`. Disabled elements and `data-sfx="none"` stay silent. The diagram after the rules shows the whole path.

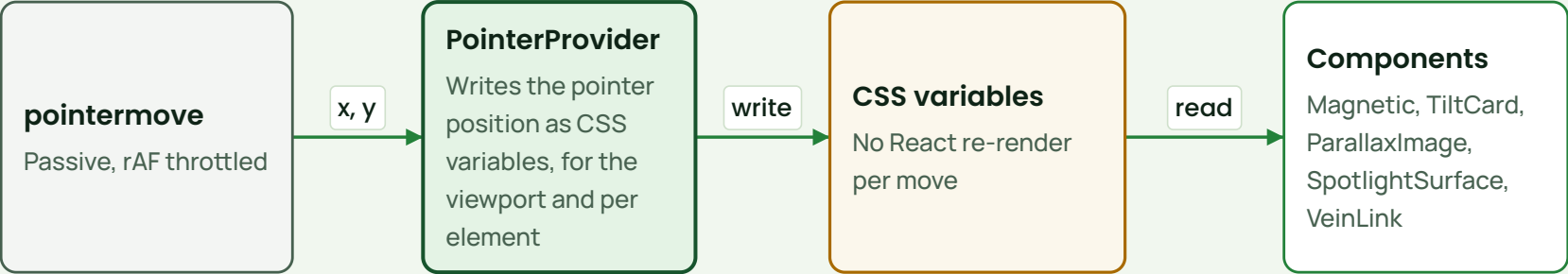
Sound rules for users

Rule	Detail
Starts on a gesture	The audio context starts only after the first user gesture (browser autoplay policy). No sound on the splash.
Visible toggle	A Sound on and off toggle in the header, keyboard accessible, saved in <code>localStorage</code> . The choice is remembered.
Quiet by default	Master volume about 25%.
Reduced motion	Hover sounds off. Only essential click, success and error remain.
Tab hidden	Muted while the tab is hidden.
Never on	Keyboard focus movement, autoplay, or touch hover.
Rate limited	Per sound minimum gap plus a burst limit across sounds, so rapid clicks never stack.
Never the only feedback	Every sound has a visual twin.
Skipped for scores	<code>?nosound</code> and Lighthouse runs never play or load sound.
Performance	Module lazy loaded after idle, under 3 KB, initialized on first interaction, off the main render path. No effect on LCP or INP.
Tests	Unit tests cover the mapping and throttling with a mocked <code>AudioContext</code> .

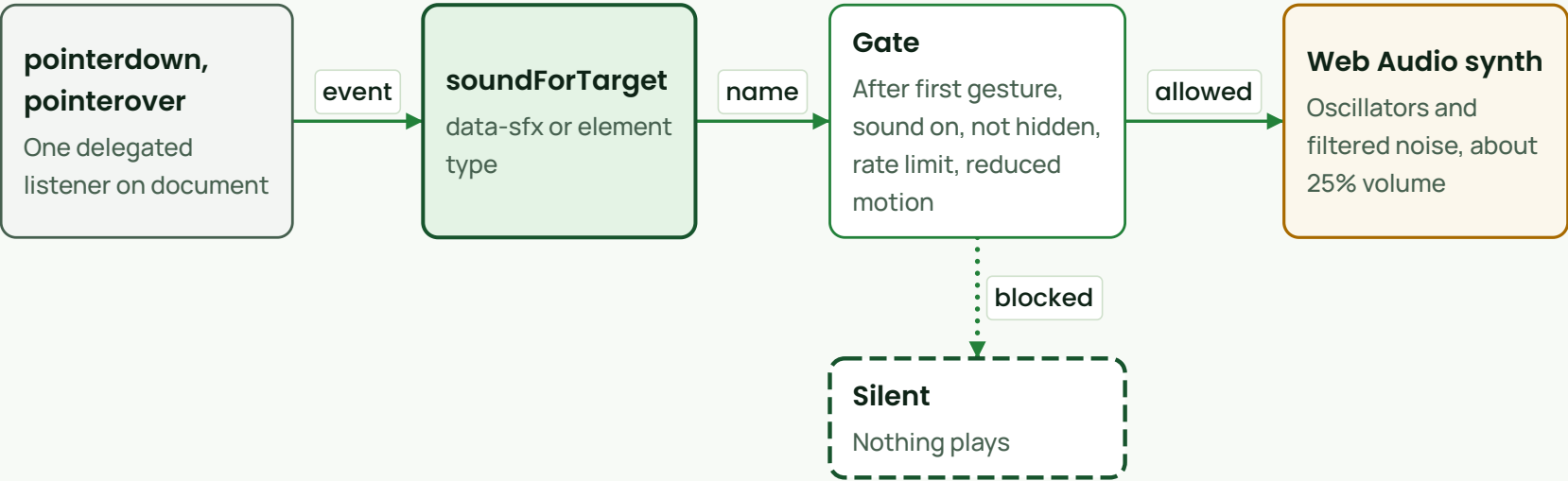
Pointer and sound pipeline



Visual reactions



Sound



Visual reactions read CSS variables written by one pointer listener. Sounds go through a mapping, a rate limit gate and the synthesizer. A blocked sound ends at Silent.

Splash screen

A full screen LEAFY screen shown on every full page load: first open and every browser refresh. It does not show on in app client navigation. The name comes from one `BRAND_NAME` constant.



1 Paint. A server rendered, CSS only overlay covers the page. No JavaScript needed.



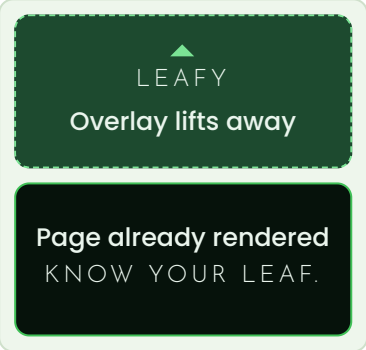
2 Letters. They fade and slide in one after another.



3 Leaf. The mark draws in beside the wordmark.



4 Hold. Mark and wordmark stay at least about 0.9 s, so it never flickers.



5 Lift. The overlay (dashed outline) lifts away by transform and opacity only.



6 Done. The overlay is removed. With reduced motion, steps 2 to 5 use a plain fade.

Topic	Rule
Timing	Minimum about 0.9 s, hard maximum about 2.2 s. A tiny inline script removes it when fonts are loaded and the first route is hydrated. A pure CSS fallback animation hides it if JavaScript fails. In the built checkpoint run it began leaving at 985 ms.
Motion	Letters fade and slide in, a leaf draws beside the wordmark, the overlay lifts (translate and opacity, GPU only).
Accessibility	<code>aria-hidden</code> with <code>role="status"</code> text for screen readers: Loading Leafy. It traps no focus.
Performance	No extra requests: inline CSS, the already loaded font with <code>font-display: swap</code> and preload. <code>position: fixed</code> , so no layout shift. The real LCP element is painted underneath.
Skipped for	Crawlers and Lighthouse (<code>navigator.webdriver</code> or <code>?nosplash</code>), so scores measure the real page.

Progress UI for the scan

The user always sees where things stand. Upload is a real percentage. The server stages come from the status and stage the API reports. The analyzing stage has no true percentage, so its bar eases toward 90% and reaches 100% only when the result arrives.

1 Uploading. A real percentage from the upload progress event. Cancel is available.

2 Checking image. The server reports processing, stage validating.

3 Analyzing. The 3D scanning beam sweeps the leaf, in sync with the stage. The bar eases and slows.

4 After about 8 s. Reassurance copy appears and the bar crawls toward 90%.

5 Saving, then Done. The bar reaches 100% only when the result arrives.

6 Timeout. A clear failure with a Retry button that reuses the stored image.

Stages, polling and timeouts

The stage table, the polling rhythm, what happens when the user navigates away, the timeout and the accessibility rules are written once, under Honest progress in the Scan state section. This page only shows how they look on screen.

Result feedback

Every completed scan carries a quiet question in the Result panel: Was this result correct? It never blocks reading the result.

Result

#result

Was this result correct?

Yes

No

1 1. Ask. Yes or No, shown quietly under the verdict.

Result

#result

What is it really?

Pick it

Not sure

Comment, up to 300 characters

2 2. Correct (optional). After No: pick the plant, then the disease, or choose Not sure, and add a short comment.

Result

#result

Thanks, this helps us improve

3 3. Thank. Calm copy, and the answer can be changed or cleared later.

Topic	Detail
When it shows	Only on completed scans. It is stored in <code>scan_feedback</code> through <code>PUT /scans/{id}/feedback</code> and cleared with <code>DELETE</code> .
Export	The team exports feedback for the ML teammate with <code>python -m app.tools.export_feedback</code> , a CSV of scan id, predicted and corrected labels, and image key.
Training use	Scan images are not used for training unless the team decides so and updates the Privacy Policy.
Status	Built. Shown on the result panel and on every opened scan.

Layout rules: fill the screen

Every page is a single screen stage (`min-h-dvh`) that fits 1440 by 900 and 390 by 844. Content fills the width with small gutters and no dead strips, and long content moves sideways, so scrolling stays minimal.

Header: mark, nav, sound, theme, user

Text, title and actions

3D scene or imagery

Panel arrows and labelled dots

Desktop 1440 by 900. Columns side by side: text beside 3D or imagery, or a dashboard side rail. Text keeps its readable measure.

Header

Card or panel

Dots

Bottom nav

Mobile 390 by 844. Bottom nav, safe area insets.

Rule	Value
Stage height	<code>dvh</code> heights with a <code>vh</code> fallback and CSS grid <code>fr</code> rows.
Width	Full bleed sections and 3D canvases with content layered on top. Wide container about 1600 px or fluid.
Gutters	<code>clamp(16px, 3vw, 48px)</code> .
Spacing tokens	4, 8, 12, 16, 24, 32, 48.
Page height test	Playwright: document at most 1.15 times the viewport on main routes.
Title test	Playwright: every h1 and h2 at most 2 lines at 360, 768 and 1440.
Breakpoints	360, 480, 768, 1024, 1440, 1920. Design is checked at 360, 768 and 1440.
Navigation	Header on desktop, bottom nav on mobile, current location clear, breadcrumbs on handbook pages, panel progress dots with labels.
Touch	44 px minimum targets. On phones the scan offers Take photo, which opens the camera through <code>capture="environment"</code> .

Global states

State	What the user sees
Offline and failed requests	A banner explains what happened and offers a Retry button. Form input is kept.
Page not found	Calm message with links to the handbook and home.
Route error	Reassuring message with Retry.
Signed out on a private route	Sent to <code>/login?next=</code> with the destination kept.
Rate limit	Message with a countdown.

3D scenes

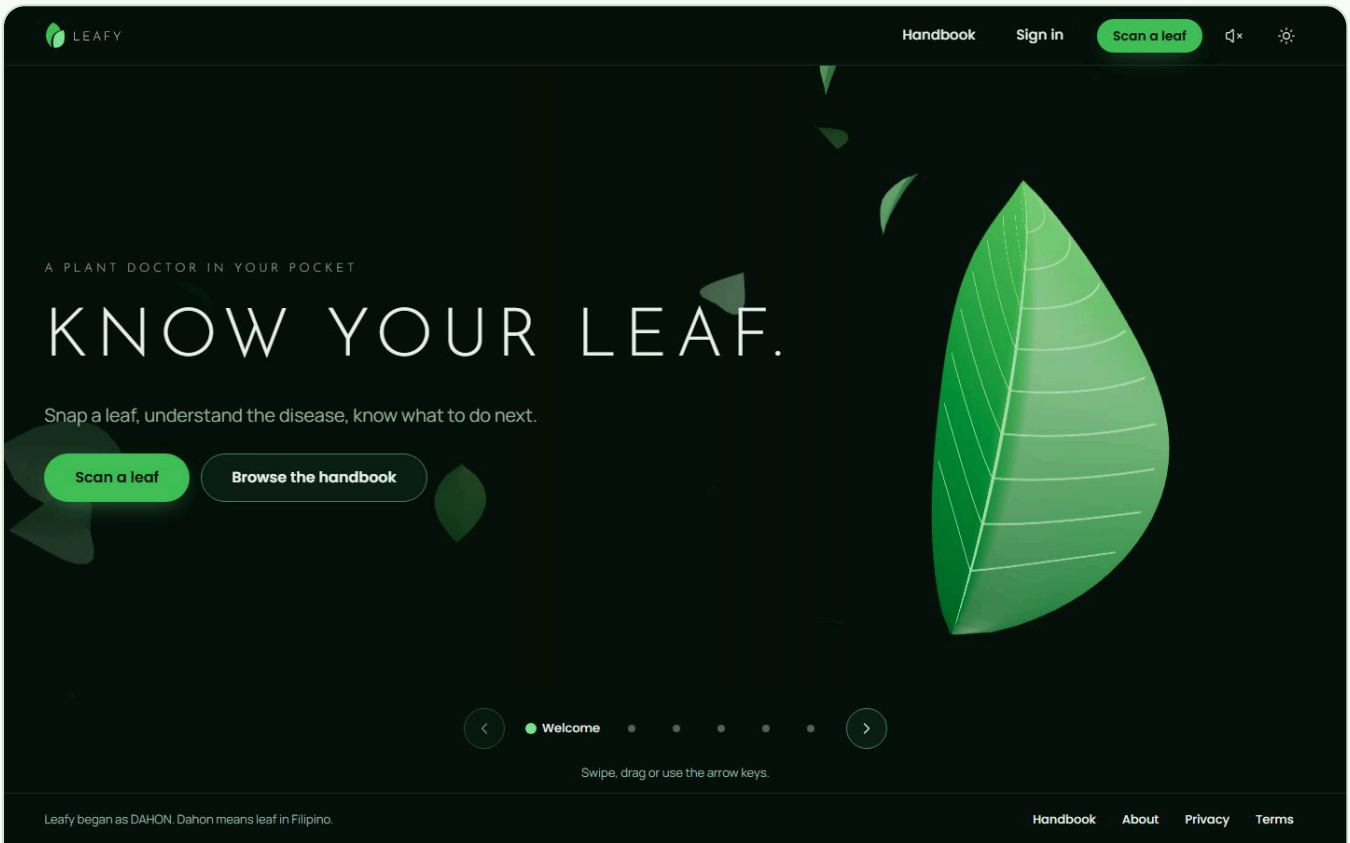
All 3D code is client only and lazy loaded per route. Content is real HTML on top, and canvases are `aria-hidden`. The shared module lives in `app/src/components/three/` and is built, with the scan beam and the orbit view of the result in `three/scan/`.

Piece	Role and budget
<code>SceneCanvas</code>	Shared canvas with Suspense and an error boundary. DPR capped at 1.5. <code>frameLoop="demand"</code> where the scene is static.
<code>LeafModel</code>	Procedural leaf or a Draco glTF under 300 KB, shared by splash, hero, auth and scan scenes.
<code>FloatingLeaves</code>	Instanced drifting leaves with pointer parallax.
<code>useReducedMotion</code>	Switches scenes to still or plain states.
<code>useWebGLSupport</code>	No WebGL gives a static poster instead.
<code>PerfGuard</code>	Pauses off screen canvases. One active canvas per page.
Shared <code>View</code> canvas	Handbook and dashboard plant models render through one drei <code>View</code> canvas, not one WebGL context per card.
Mobile budget	At most 100k triangles and no postprocessing.

SECTION 10

THE FINISHED APP

Real screenshots of the built app, taken in a browser at 1440 by 900 and 390 by 844 pixels, in dark and light. Every page shown here is built and tested. Scan results come from the dev only fake model, so the plant and disease shown with each photo are demonstration data.

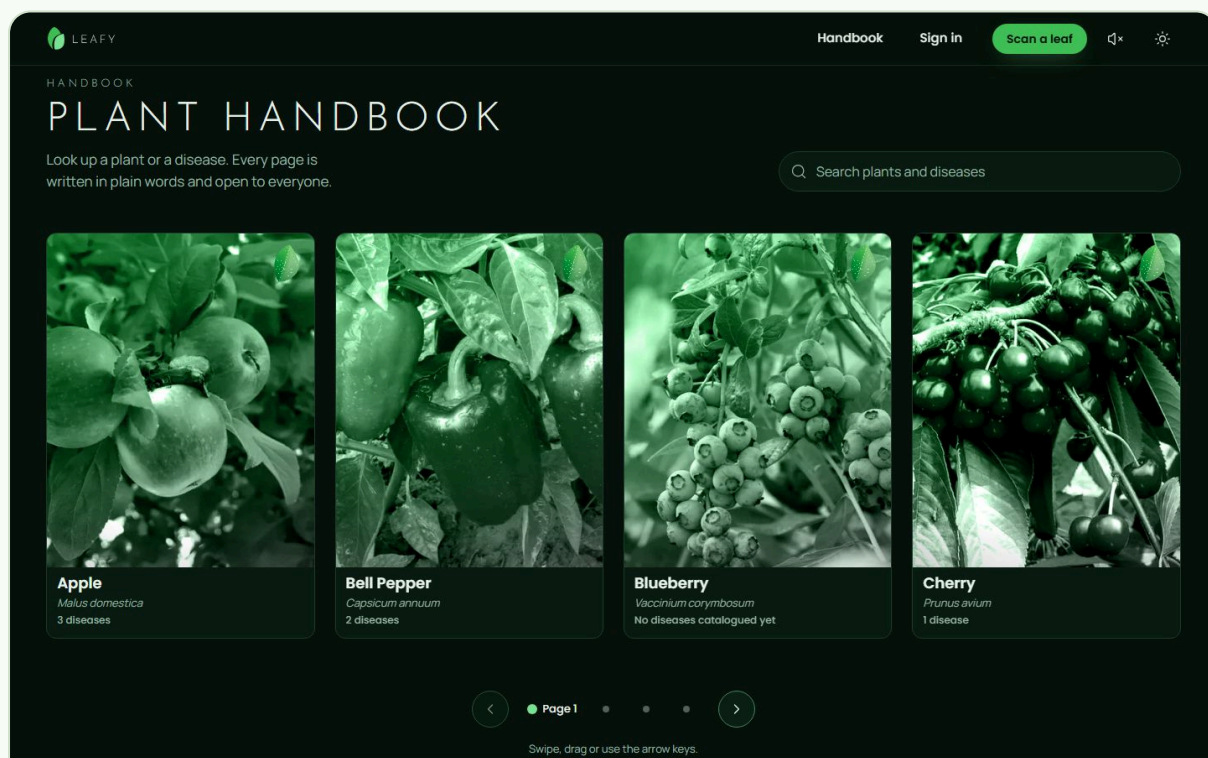
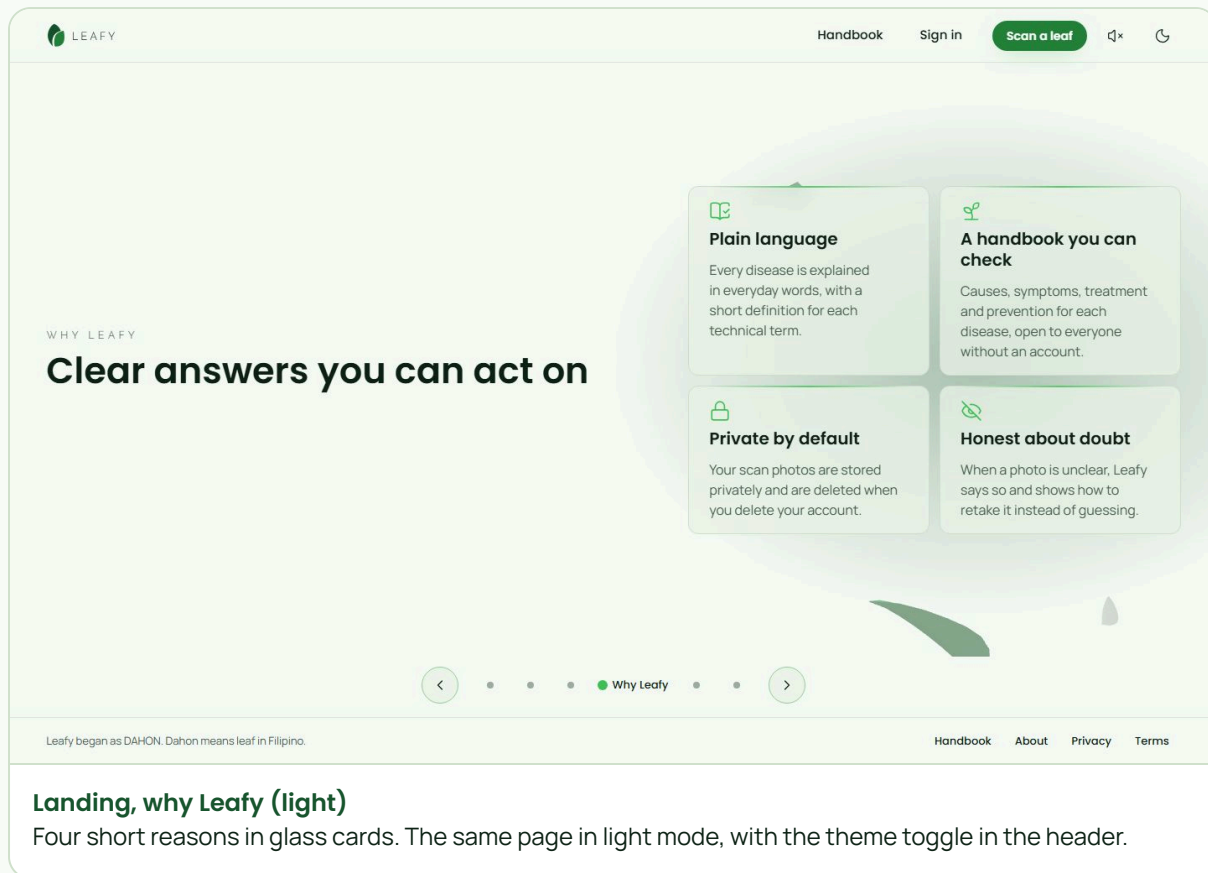


Landing, hero (dark)

The first panel: the one line headline, one supporting line, and two buttons. The 3D leaf drifts and leans toward the cursor. The page moves sideways in panels.

Pages that follow	What they show
Landing and handbook	Landing panels, the plants carousel, a plant page, disease pages and their panels.
Auth and dashboard	Sign in, create account, the member dashboard.
Scan journey	Upload, preview, progress, result, outcomes, history and account. Phone views close the group.

Landing and handbook



Plant and disease pages

LEAFY

HandbookSign inScan a leaf

Handbook / Tomato

SOLANACEAE - FRUIT

TOMATO

Solanum lycopersicum

Soil

Well-drained, loamy soil; pH 6.0 to 6.8

Light

Full sun (6 to 8+ hours/day)

Water

Regular, deep watering (keep soil consistently moist but not soggy)

Temperature

Warm-season; ideal temperature range: 70 to 85°F (21 to 29°C)



<Overview>

Swipe, drag or use the arrow keys.

Plant page, tomato (light)

Growth conditions beside the plant photo, and a second panel with its diseases.

LEAFY

HandbookSign inScan a leaf

Handbook / Tomato / Yellow Leaf Curl Virus

TOMATO YELLOW
LEAF CURL VIRUS

Severity: Severe

Overview

Plant	Tomato
Cause type	Virus
Pathogen	Tomato yellow leaf curl virus
Severity	Severe Severe; can result in complete crop loss if not managed.



Severity: Severe

<Overview>

Swipe, drag or use the arrow keys.

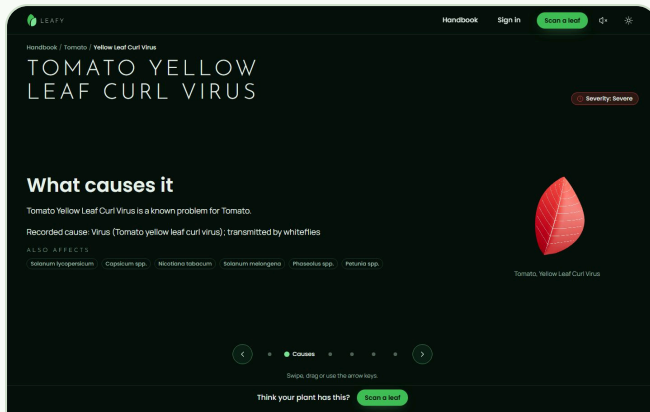
Think your plant has this?

Scan a leaf

Disease page, overview (dark)

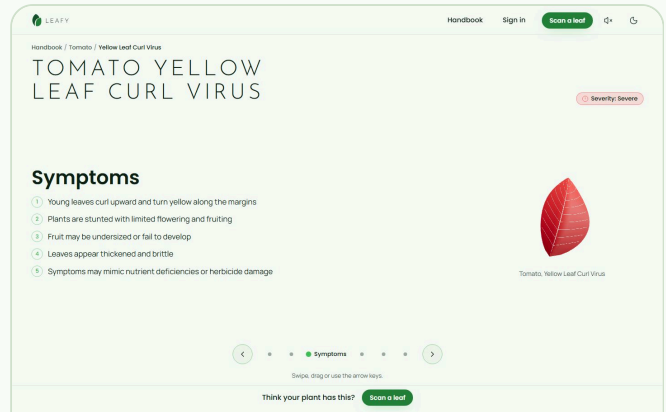
A short display name in a two line title, a severity badge, and the facts in the first panel. Six panels follow sideways.

Disease panels



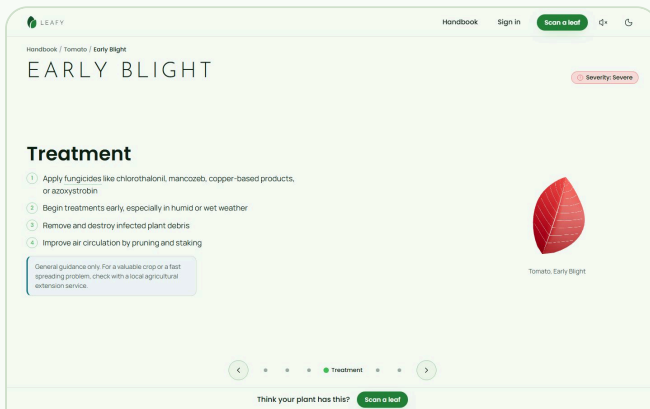
Causes

What causes it, and the other plants it affects.



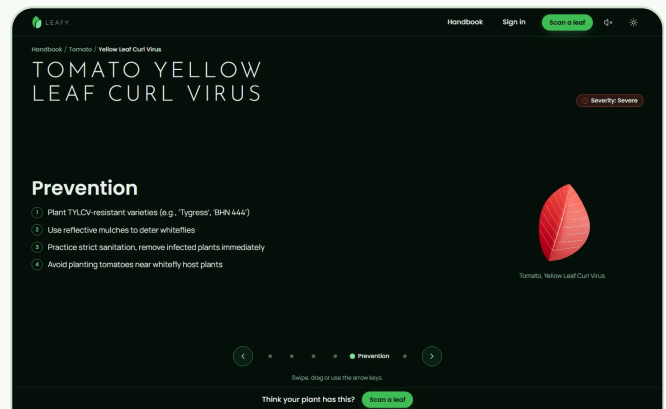
Symptoms

A short numbered list, never more than six items.



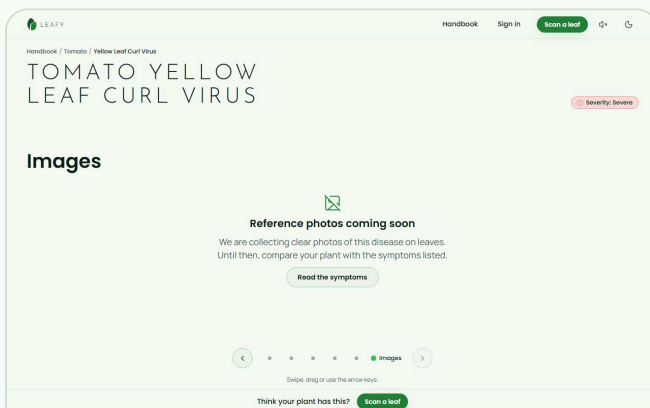
Treatment

Early blight treatment steps and a calm advice note.



Prevention

Steps that lower the risk. Every panel ends with a link to scan a leaf.



Images, not yet available

The panel says Reference photos coming soon. The DAHON sample images are corrupt, so no clean set exists yet.

Open item

Disease reference photos wait for a clean image set. Everything else on these pages is real handbook text from the seed data.

Sign in and create account

The screenshot shows the 'Sign in' page of the Leafy application in a dark theme. The background is dark with faint green leaf patterns. On the left, the text 'WELCOME BACK' is above 'SIGN IN' in large white letters, followed by the subtitle 'Pick up where you left off with your scans and the handbook.' On the right, a white login form contains a 'Continue with Google' button, a link 'or use your email', input fields for 'Email' and 'Password' (with an eye icon for toggling visibility), a green 'Sign in' button, a link 'Forgot your password?', and a link 'New to Leafy? Create an account'. The top navigation bar includes 'Handbook', 'Sign in', and a green 'Scan a leaf' button. The footer contains the text 'Leafy began as DAHON. Dahon means leaf in Filipino.' and links for 'Handbook', 'About', 'Privacy', and 'Terms'.

Sign in (dark)

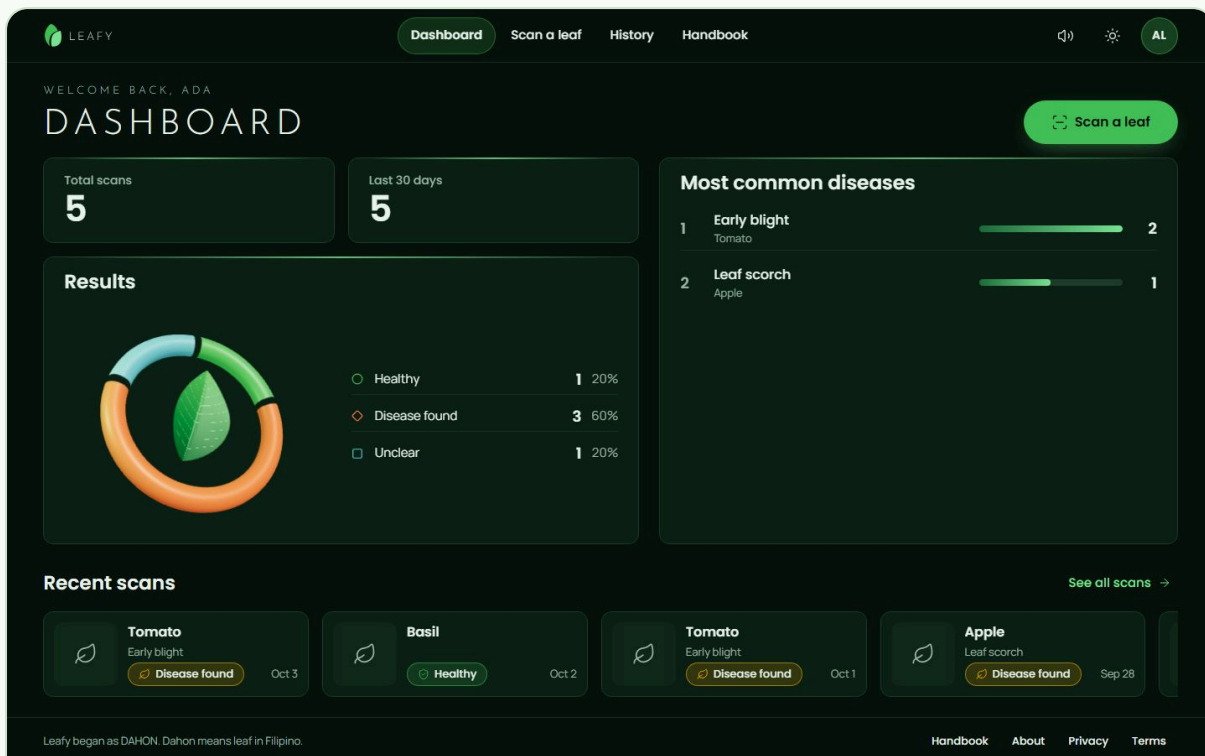
Continue with Google, or email and password. Wrong credentials give one uniform message, and a rate limit shows a countdown.

The screenshot shows the 'Create account' page of the Leafy application in a light theme. The background is light green with faint leaf patterns. On the left, the text 'START WITH LEAFY' is above 'CREATE ACCOUNT' in large dark letters, followed by the subtitle 'A free account keeps your scans private and your history in one place.' On the right, a white registration form contains a 'Continue with Google' button, a link 'or use your email', input fields for 'Your name', 'Email', and 'Password' (with an eye icon for toggling visibility), a strength hint 'Use at least 10 characters.', a green 'Create account' button, a link 'By continuing you agree to the Terms and Privacy Policy.', and a link 'Already have an account? Sign in'. The top navigation bar includes 'Handbook', 'Sign in', and a green 'Scan a leaf' button. The footer contains the text 'Leafy began as DAHON. Dahon means leaf in Filipino.' and links for 'Handbook', 'About', 'Privacy', and 'Terms'.

Create account (light)

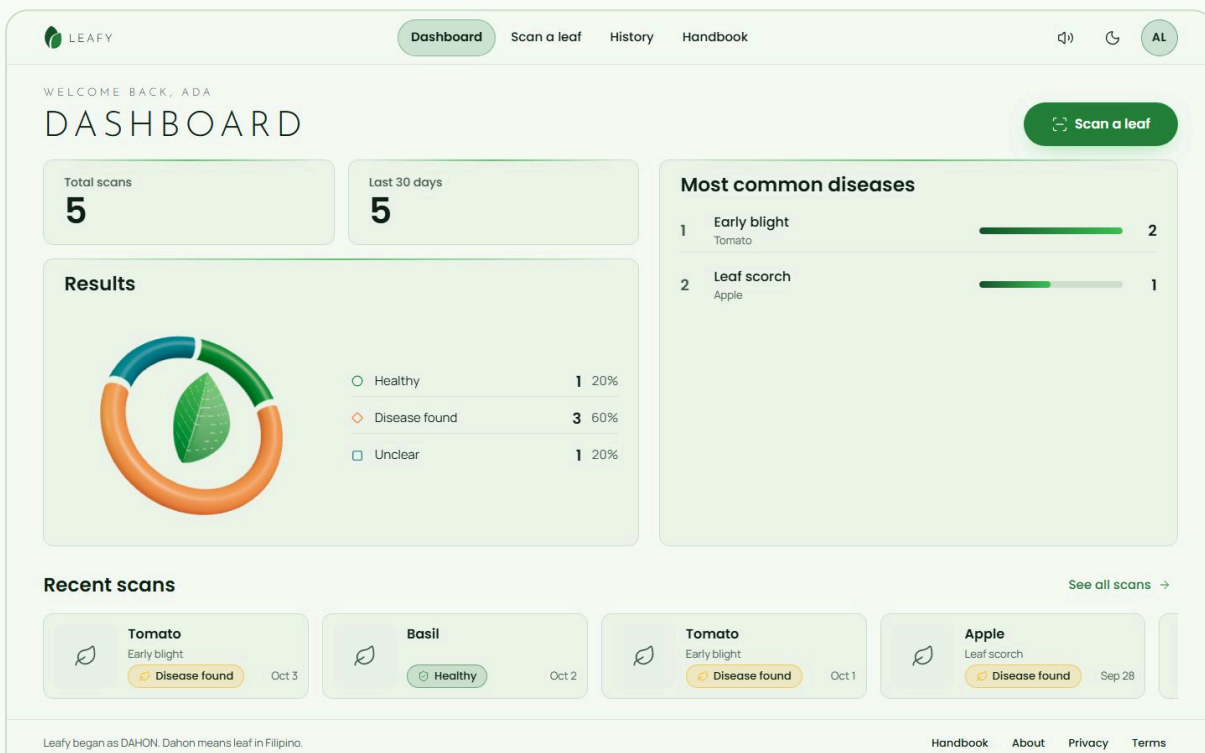
Name, email and a password with a strength hint. Links to the Terms and Privacy Policy sit under the button.

Dashboard



Dashboard (dark)

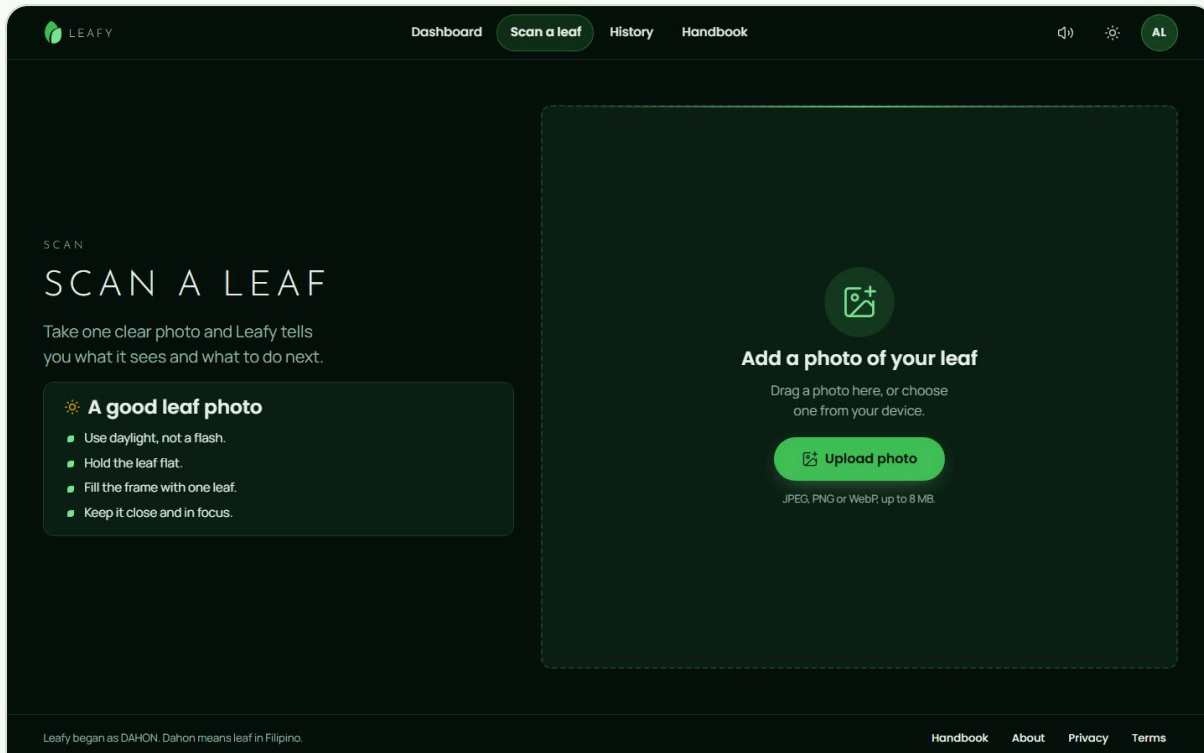
Total scans, the last 30 days, the healthy, diseased and unclear split, the most common diseases, and a rail of recent scans.



Dashboard (light)

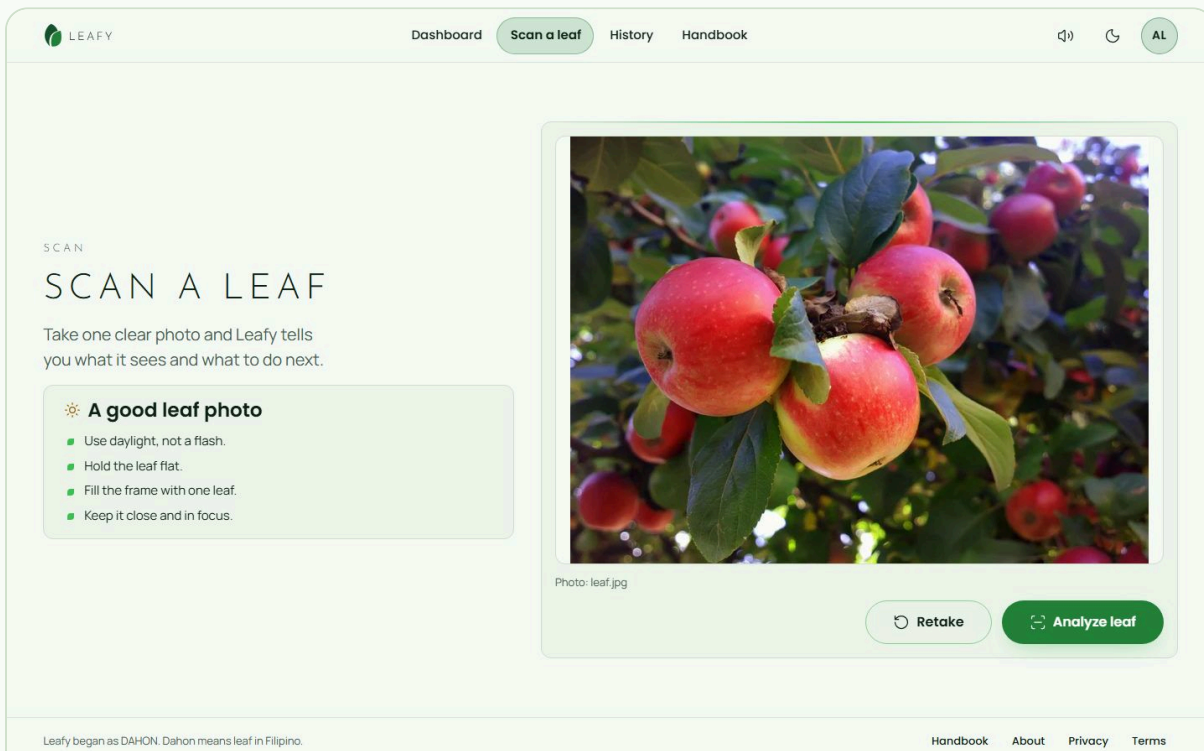
The same page in light mode, with the primary Scan a leaf button at the top right.

Scan: upload and preview



Scan, ready (dark)

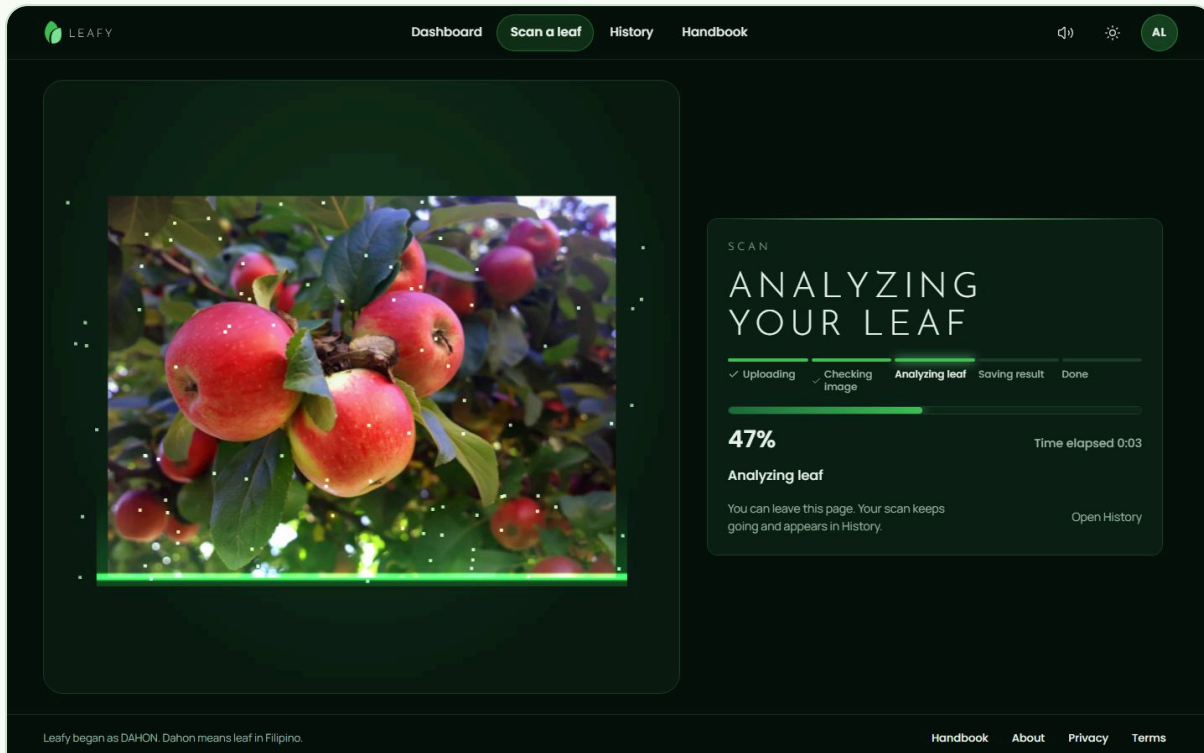
A tip card for a good photo, and a large drop zone. Phones also get a Take photo button for the camera.



Scan, preview (light)

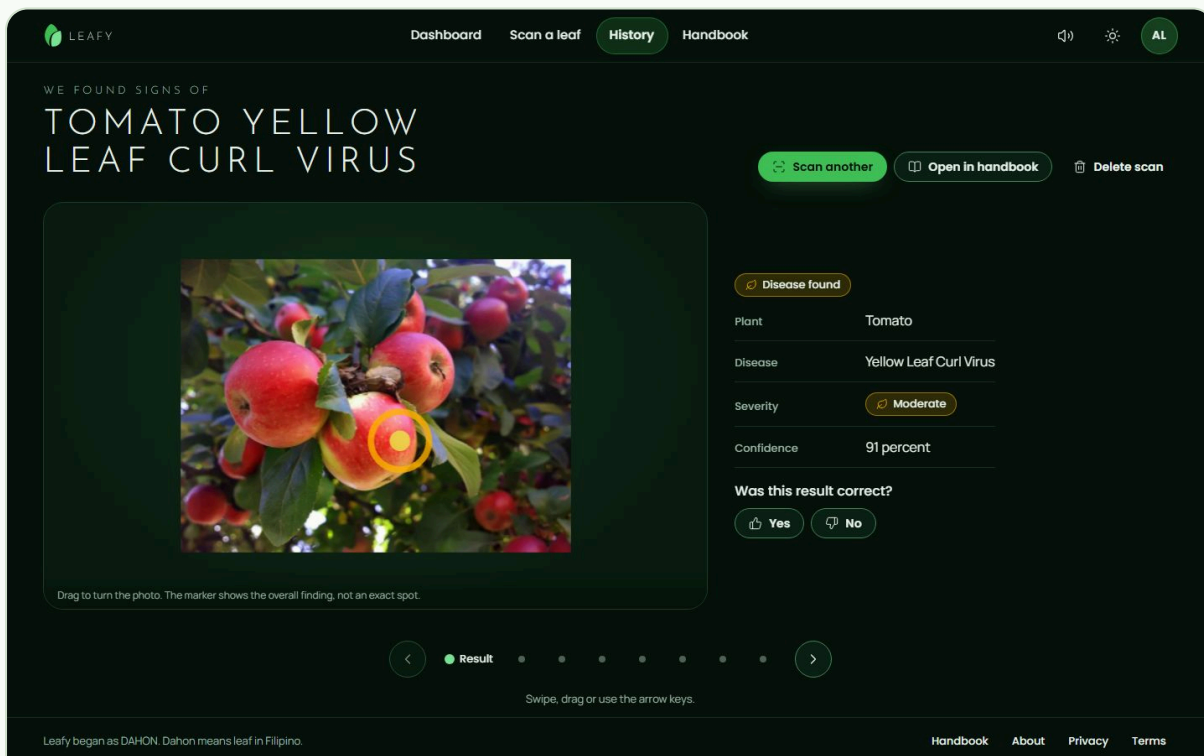
The chosen photo with Retake and Analyze leaf. Type, size and dimensions are checked before anything is sent.

Scan: progress and result



Progress (dark)

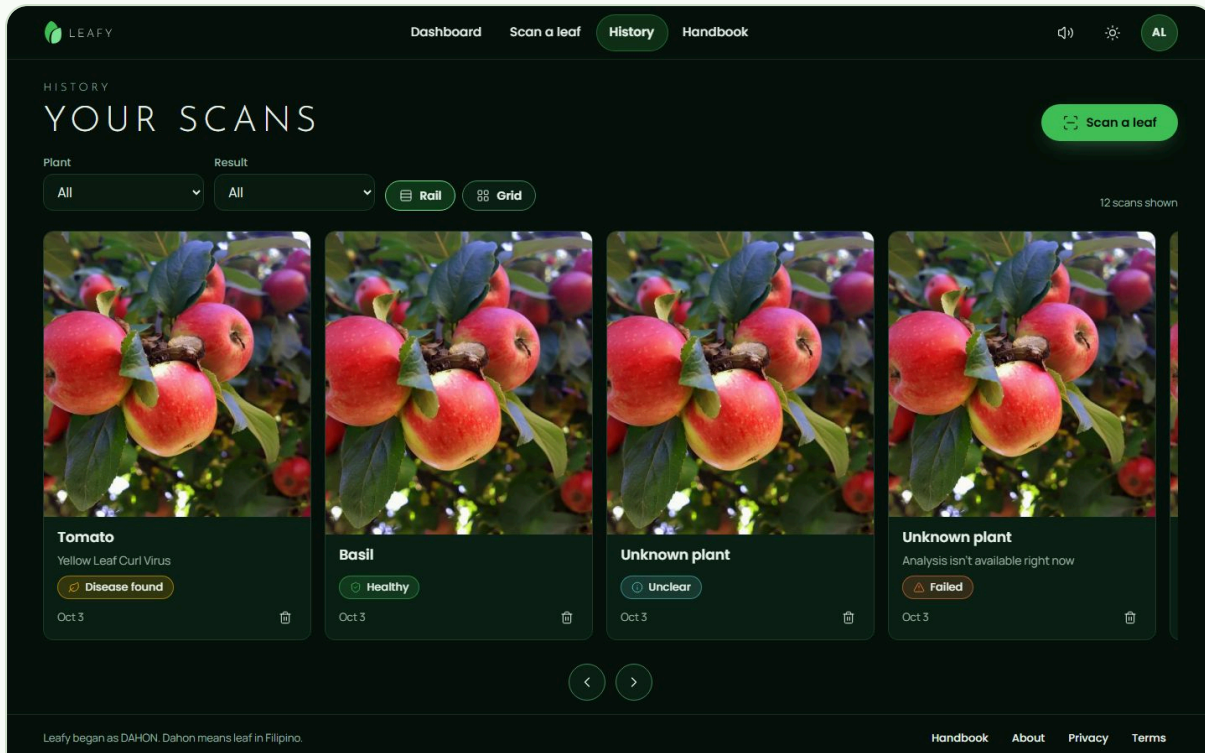
The scanning beam runs over the photo while the stepper shows Uploading, Checking image, Analyzing leaf, Saving result and Done, with percent and elapsed time.



Result (dark)

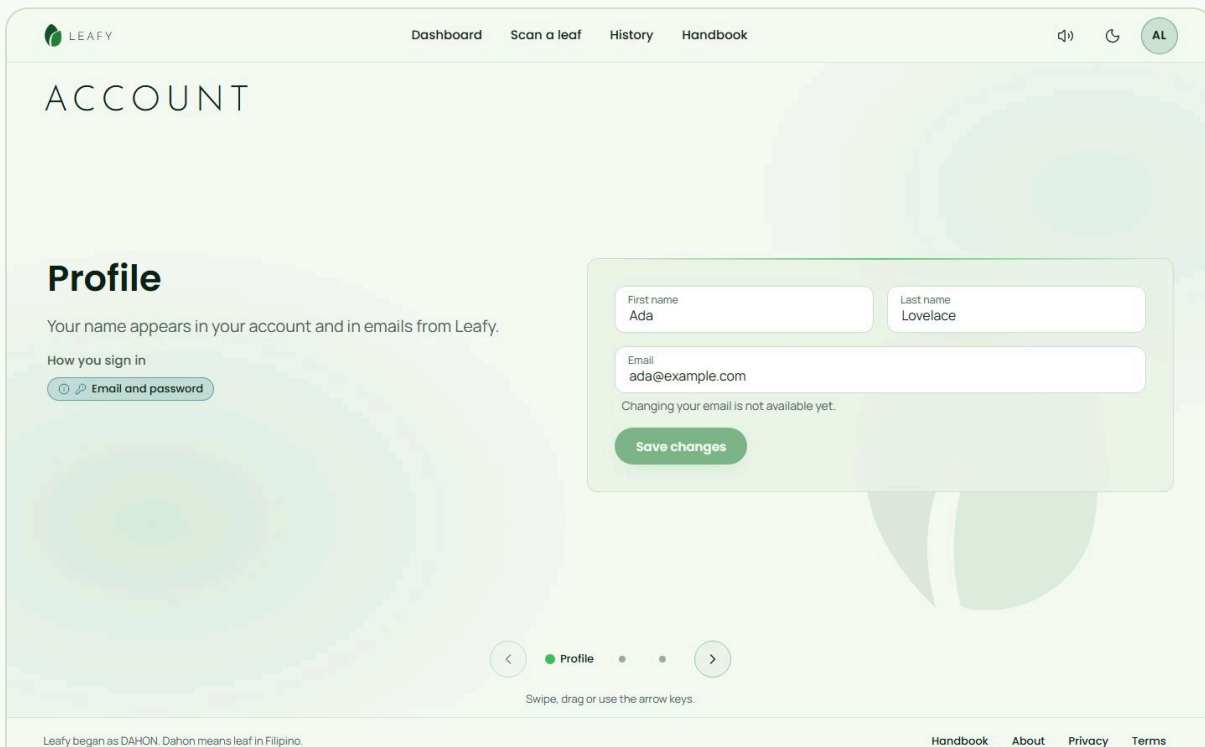
The verdict first, then plant, disease, severity and confidence, an orbitable photo with a hotspot, and the Was this result correct question.

History and account



History, rail (dark)

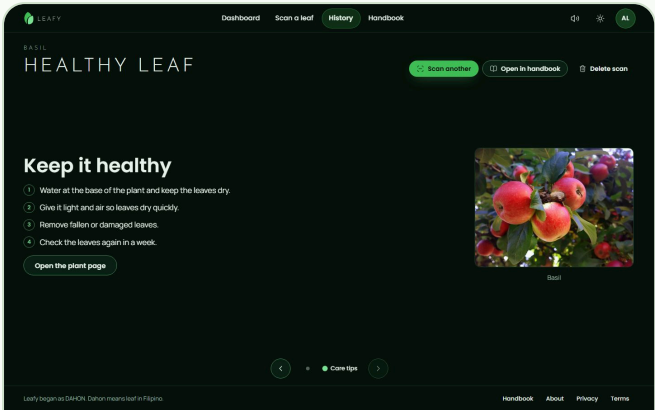
Scans as a card rail with filters by plant and result, and a grid toggle. A processing scan shows a live badge.



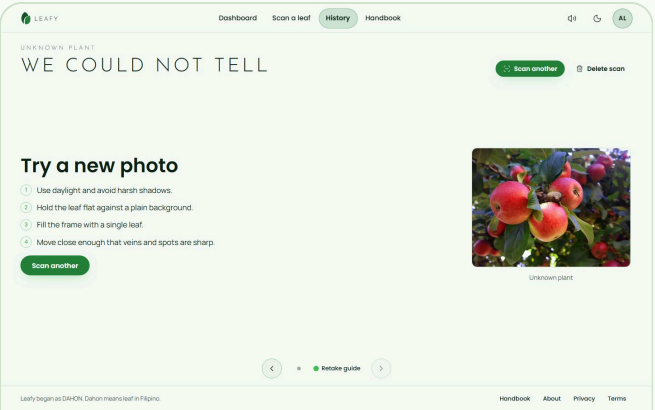
Account, profile (light)

Profile, Password and Danger zone panels. It shows how you sign in, with password, Google or both.

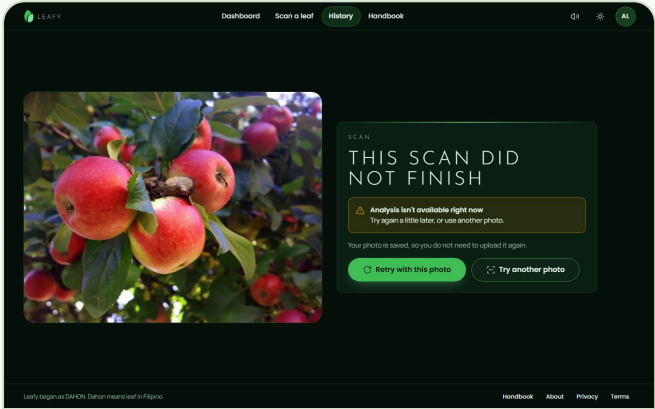
Other scan outcomes



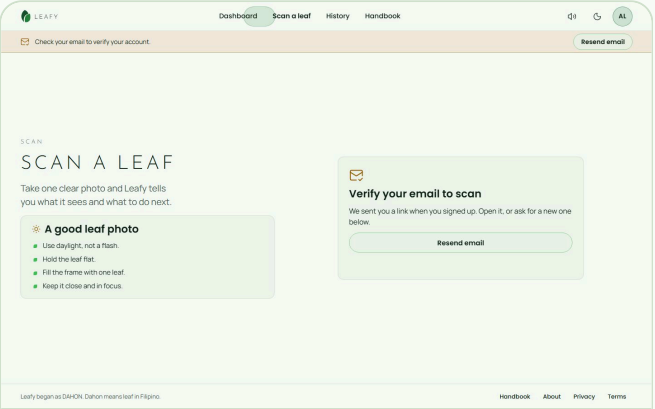
Healthy
Reassuring care tips for the plant.



Unknown
A calm explanation and a retake guide instead of a dead end.



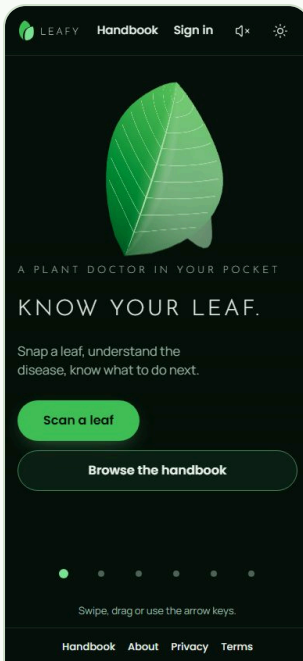
Failed
The reason in plain words. Retry reuses the stored photo, so nothing is uploaded again.



Email not verified
Unverified members can browse but are asked to verify before they scan, with a Resend action.

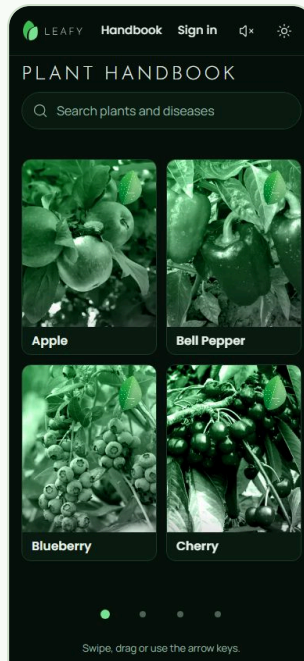
Outcome	What the member can do next
Disease found	Read causes, symptoms, treatment, prevention and photos, mark the result right or wrong, scan another leaf, or open the handbook page.
Healthy or unknown	Follow care tips, or retake the photo with the guide.
Failed	Retry on the stored image or try another photo. With the model stub, every scan ends here with Analysis isn't available right now.

On a phone



Landing

Dark. Buttons stack and the nav is one row.



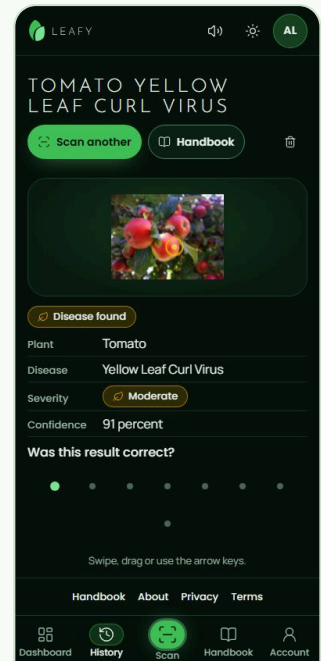
Handbook

Dark. Two column plant cards.



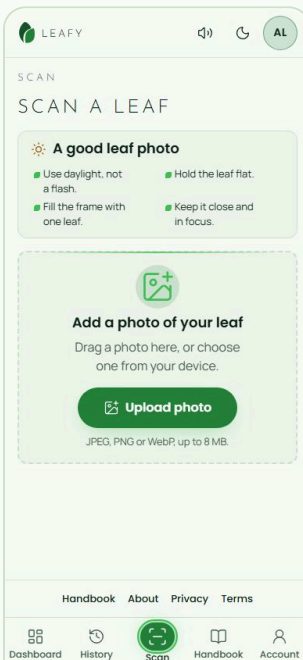
Progress

Dark. The stepper and bar fit one screen.



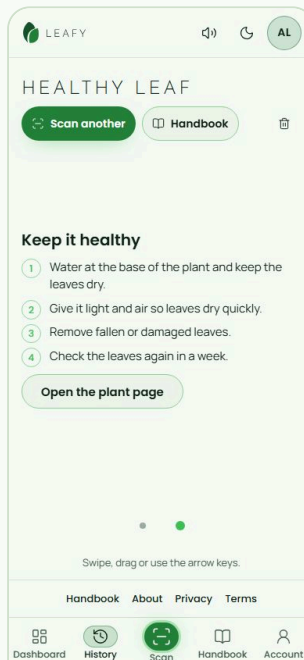
Result

Dark. Verdict, photo and feedback.



Scan

Light. The bottom nav keeps Scan in the center.



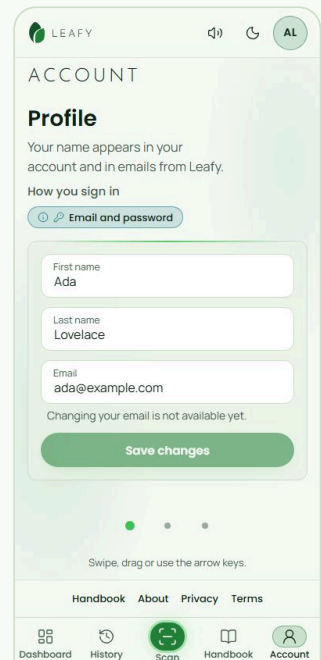
Healthy

Light. Care tips for the plant.



Dashboard

Light. Stat tiles stack.



Account

Light. One panel at a time.

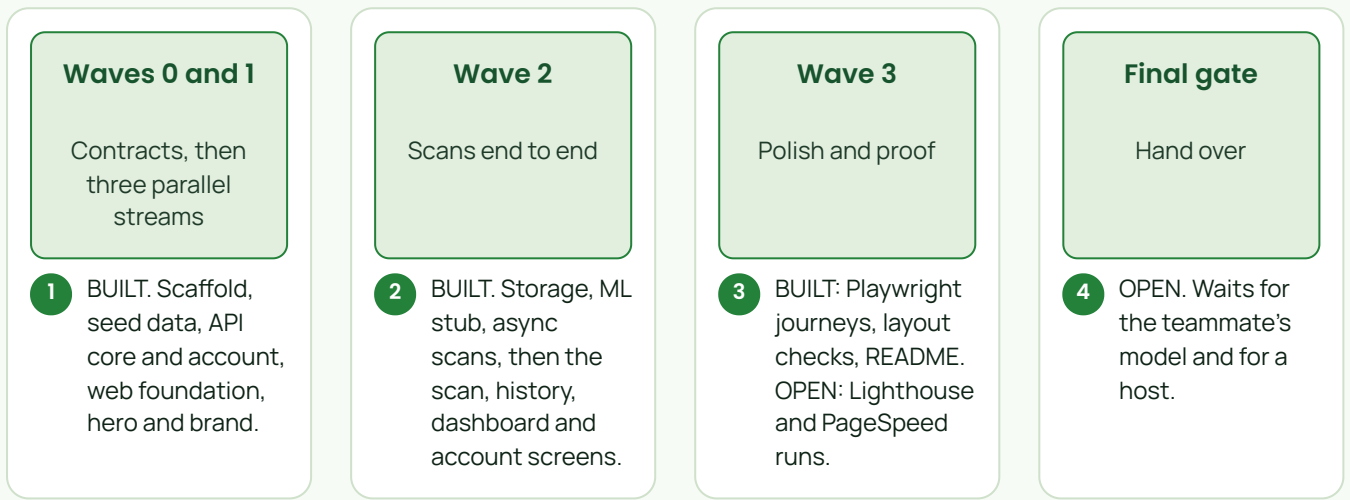
Phone screens use a bottom navigation with Dashboard, History, Scan, Handbook and Account, edge to edge cards and safe area insets. Each page fits one screen, and longer content moves sideways in panels.

SECTION 11

BUILD PHASES AND WORKFLOW

Work followed phases 0 to 15, plus 12b, in order of dependency. Tests came first, independent work ran in parallel with disjoint folders, and no wave started before the previous gate was green. Everything is built except the real ML model and the items that need a host.

Timeline in four waves



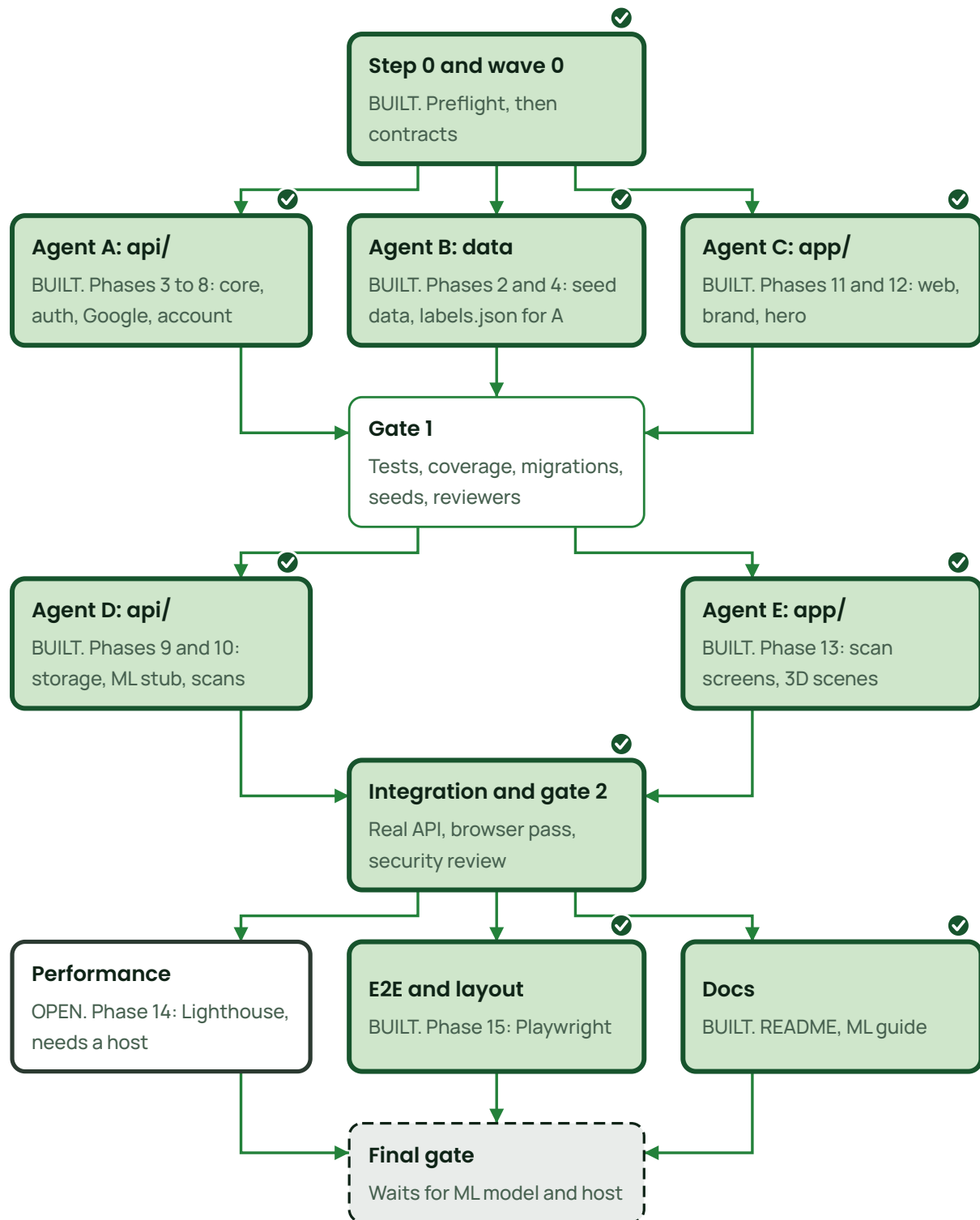
Operating rules and safety

Rule	What it means
Dependency first	Phases run in order of dependency. Independent work runs in parallel with subagents, each owning disjoint folders. No phase starts before the previous gate is green.
TDD by default	Tests first for the API and Next logic. UI visuals are verified in a real browser.
Small conventional commits	One commit per green phase (<code>feat:</code> , <code>fix:</code> , <code>test:</code> , <code>chore:</code>), so any phase can be reverted alone. No push until the team says so, no <code>--no-verify</code> , no force operations.
Preflight	Tools, free ports and inputs are checked first. With no Docker, development uses native services.
Protected input	The DAHON folder is never modified, and destructive commands are not used without asking.

Phases

No.	Work	Wave	Status
0	<code>git init</code> in <code>C:Leafy</code> , <code>.gitignore</code> , origin set to the Leafy GitHub repository. The first push waits for the go ahead.	0	Built (no push)
1	Scaffold monorepo, compose, <code>.env.example</code> , lint and CI skeleton.	0	Built
2	Seed extraction tool and committed data: 13 plants, 27 diseases, <code>labels.json</code> , cleaning report.	1 (B)	Built
3	Core and database: config, errors, envelope, models, migrations, unit of work, constraint tests.	1 (A)	Built
4	Seed loader: idempotent, <code>--check</code> .	1 (A, B)	Built
5	Catalog read API.	1 (A)	Built
6	Auth core: register, login, token service, rotation, reuse detection with a concurrent rotate test.	1 (A)	Built
7	Email token flows (verify, resend, forgot, reset), then Google sign in: API verifier, <code>oauth_identity</code> migration, mock Google, Next handlers, Google button.	1 (A)	Built (mock Google)
8	Account: profile, change password, delete account and outbox purge.	1 (A)	Built
9	Storage, image validator, ML abstract class and stub, prediction resolver with table driven tests, dev only fake model.	2 (D)	Built
10	Scans API: async 202 flow, stages, retry, stuck scan janitor, ownership, keyset paging, stats, feedback and its CSV export.	2 (D)	Built
11	Next foundation: env, <code>apiFetch</code> , cookies, single flight refresh, middleware, auth handlers, CSRF, Vitest.	1 (C)	Built
12	Design system: tokens, fonts, <code>prose</code> , <code>SlidePanels</code> , cursor, interaction primitives, SFX, splash, three.js module, landing, handbook and auth pages.	1 (C)	Built
12b	Public extras: About, Privacy, Terms, footer, SEO, PWA manifest, no flash theme script, branded emails. The install prompt is not built.	2 to 3	Built (one item open)
13	Scan, history, dashboard and account screens with result feedback, 3D scenes, reusable progress UI.	2 (E)	Built
14	Performance pass: Lighthouse CI on mobile and desktop, bundle budgets, PageSpeed Insights after deploy.	3	Open: needs hosting
15	Playwright E2E and layout assertions, README, final review. Dockerfiles are not written yet.	3	Built (Dockerfiles open)

Execution workflow



Waves, agents and gates. Arrows show the order of work. BUILT and OPEN show where it stands.

Boxes



Built: green fill, thick border, check badge



Open: thick dark border, white

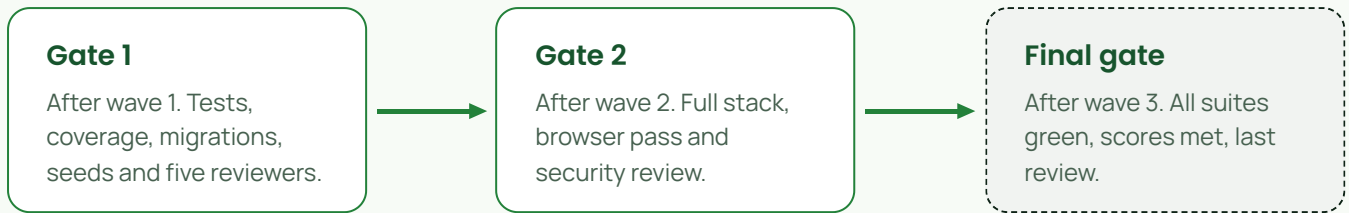


Gate: thin green border, white



Final gate: grey, dashed

Gates



Gate	What must be true
Gate 1	• <code>pytest</code> green, 80 percent or more coverage on the backend work. • <code>pnpm test</code> and <code>pnpm build</code> green. • Migrations apply on an empty database. • Seeds <code>--check</code> passes. • Five reviewers (code, FastAPI, security, React, TypeScript) run in parallel. Every CRITICAL and HIGH finding is fixed.
Gate 2	Full compose stack up. Manual browser pass: register, verify via Mailpit, upload with a fake ML service, progress UI, result panels, history, delete, forgot and reset, delete account. Ownership and refresh tests. Security review of upload and S3 paths.
Final gate	All suites green, Lighthouse thresholds met, a last code and security review of the whole diff, and a short report of what was and was not verified.

Stop and ask

Even in auto mode the work stops and asks before these steps.

A step that fails twice for the same cause stops and reports.

Trigger	Until then
Pushing to GitHub or opening a pull request	Work stays local on a feature branch.
Real credentials (Google OAuth, SMTP, production S3, deploy)	Everything runs against the Google mock, Mailpit and MinIO.
Licensing of DAHON sample images	Not deployed publicly.
A change that conflicts with a confirmed decision	The decision stands.
The teammate's <code>predict()</code> implementation	Only the abstract class, stub and contract ship.

SECTION 12

HOW TO RUN

Five commands on Windows, no Docker. The script starts Postgres, storage, Mailpit, the migrations, the seed, the API and the web build, with throwaway local secrets and the mock Google provider.

Quickstart

You need Node 20 or newer, pnpm, uv, and the PostgreSQL 17 binaries. Set `PG_BIN` if they are not in `C:\Program Files\PostgreSQL\17\bin`. Run these from the repository root.

```
git clone https://github.com/BunquinTheodore/Leafy-Ph-.git; cd Leafy-Ph-
cd api; uv sync; cd ..          # 1. API dependencies
cd app; pnpm install; cd ..     # 2. web dependencies
.\scripts\dev\run-all.ps1      # 3. services, migrations, seed, API, web
start http://127.0.0.1:3000     # 4. open the app
.\scripts\dev\run-all.ps1 -Stop # 5. stop everything
```

Where things run

Address	What it is
<code>http://127.0.0.1:3000</code>	The app, a Next.js production build.
<code>http://127.0.0.1:8000/docs</code>	The API and its interactive docs.
<code>http://127.0.0.1:8025</code>	Mailpit inbox. Verification and reset emails land here.
<code>127.0.0.1:5433</code>	Project local Postgres. Port 5432 is never touched.

The script is safe to run again: it starts what is not running, applies `alembic upgrade head`, runs `python -m app.seeds`, and writes logs to `.dev/logs`.

The model while it is missing

The script runs with `ML_SERVICE=dev-fake`. It returns a label picked from the image hash, so the whole scan journey can be tried. It logs a loud DEV ONLY warning and refuses to start when `ENV=prod`.

Switch	Effect
<code>-MLService stub</code>	Use the unimplemented stub. Every scan fails with Analysis isn't available right now.
<code>-SkipBuild</code>	Reuse the last <code>next build</code> .

Switch	Effect
<code>-E2E</code>	Rate limits off, a 1 second resend cooldown, and a second stack with the stub (API :8001, web :3001).
<code>-Stop</code>	Stop the API, the web servers and the dev services.

Tests

Command	What it runs
<code>cd api; uv run pytest</code>	API unit and integration tests on a real Postgres.
<code>cd app; pnpm test</code>	Web unit and component tests (Vitest).
<code>cd app; pnpm e2e:all</code>	Builds once, then the auth, member, scan, ui and full browser suites, with a summary.
<code>cd app; pnpm screens:audit</code>	Screenshots every route at 1440, 768 and 390 pixels in both themes and reports layout problems.

Before launch: placeholders and open items

Details the team still has to supply live in `app/src/lib/site-config.ts`. Each one shows on the site as a visible TODO marker, and a unit test fails if one stops being a marked value without being replaced on purpose.

Item	What to do
<code>LEGAL_CONTACT_EMAIL</code>	Put the real contact address into the Privacy Policy and Terms of Use.
<code>TEAM_MEMBERS</code>	Add the three real names for the About page.
Privacy Policy and Terms	The text is a plain language draft. Get a legal review.
Google sign in	Create the OAuth client, set <code>GOOGLE_CLIENT_ID</code> , <code>GOOGLE_CLIENT_SECRET</code> , <code>GOOGLE_REDIRECT_URI</code> and <code>GOOGLE MOCK=0</code> .
ML model	Plug it in with <code>ML_SERVICE=package.module:ClassName</code> (see the ML integration section).
<code>S3_PUBLIC_ENDPOINT</code>	Set the real public storage host for both <code>api</code> and <code>app</code> .
Photo provenance	Check licensing for the handbook images.

The full suite signs in with Google through the mock provider:

`/api/auth/google?email=someone@example.com` picks the identity. With Docker instead, copy `.env.example` to `.env`, edit the secrets, and use `docker compose`. The Dockerfiles are not written yet.

SECTION 13

ML INTEGRATION

For whoever writes the leaf classifier. The API ships the contract, a stub and a fake. The real `predict()` is yours, and it is the only piece the app still needs.

The contract

Subclass `MLInferenceService` (`api/app/services/ml/base.py`) and implement one method.

```
from app.services.ml.base import MLInferenceService

class MyModel(MLInferenceService):
    def predict(self, image: bytes) -> dict[str, str]:
        ...
        return {"plant": "tomato", "disease": "early-blight", "confidence": "0.93"}
```

Part	Rule
image	A sanitized JPEG made by the API: long edge at most 2048 px, EXIF and GPS removed.
Return value	A <code>dict[str, str]</code> with string keys and string values only.
plant	A plant slug from the table below, or <code>unknown</code> .
disease	A disease slug of that plant, or <code>healthy</code> , or <code>unknown</code> .
confidence	Optional. A decimal string from <code>"0"</code> to <code>"1"</code> , such as <code>"0.93"</code> .
Threads and time	The call blocks. The API runs it in a worker thread with a timeout, so it must be safe to call from several threads. Load the model once in <code>__init__</code> .
Failure	Raise any exception. The scan is stored as failed (<code>prediction_failed</code>) and the message is never shown. <code>NotImplementedError</code> maps to <code>ml_unavailable</code> (503).

Wrong or unknown slugs never crash a scan: the API resolves them to an `unknown` verdict. They do hurt accuracy, so stay inside the table.

Valid slugs

The source of truth is `api/app/seeds/labels.json`. Load that file instead of hard coding the list. Blueberry and Soybean have no diseases, so for them return `healthy` or `unknown`. Raspberry is not in the catalog: answer `unknown`.

Plant slug	Disease slugs
apple	apple-scab black-rot cedar-apple-rust
bell-pepper	bacterial-spot phytophthora-blight
blueberry	none
cherry	powdery-mildew
corn	common-rust gray-leaf-spot northern-corn-leaf-blight
grape	black-rot esca leaf-blight
orange	huanglongbing
peach	bacterial-spot
potato	early-blight late-blight
soybean	none
squash	powdery-mildew
strawberry	leaf-scorch
tomato	bacterial-spot early-blight late-blight leaf-mold septoria-leaf-spot spider-mites target-spot tomato-mosaic-virus tomato-yellow-leaf-curl-virus

Names that differ from the original DAHON prompt: Corn (maize) is `corn`, Cherry (including sour) is `cherry`, Bell Pepper is `bell-pepper`, Huanglongbing (Citrus greening) is `orange` and `huanglongbing`, Esca (Black Measles) is `grape` and `esca`. Common alternative spellings are tolerated through `api/app/seeds/slug_overrides.yaml`, but return the canonical slug.

Plugging it in

Install your package into the API environment and set one variable. The factory (`api/app/services/ml/factory.py`) imports the module, requires the class to subclass `MLInferenceService`, and calls it with no arguments, so read weight paths from environment variables inside `__init__`.

```
ML_SERVICE=my_package.my_module:MyModel
```

ML_SERVICE value	Behavior
<code>stub</code> (default)	Raises <code>NotImplementedError</code> on every scan, so the API answers 503 <code>ml_unavailable</code> .
<code>dev-fake</code>	Dev only. Picks a label from the image hash so the UI can be exercised. Refuses to start when <code>ENV=prod</code> .
<code>package.module:Class</code>	Your model. A bad spec fails fast at startup with <code>MLConfigurationError</code> .

Testing your class

Check it against the contract without running the API.

```
labels = json.loads(Path("api/app/seeds/labels.json").read_text())
result = MyModel().predict(Path("some_leaf.jpg").read_bytes())

assert set(result) <= {"plant", "disease", "confidence"}
assert all(isinstance(v, str) for v in result.values())
assert result["plant"] in labels["plants"] or result["plant"] == "unknown"
allowed = {*labels["plants"].get(result["plant"], []), "healthy", "unknown"}
assert result["disease"] in allowed
```

To exercise the whole API flow with no model, use `FakeMLService` in `api/tests/unit/test_ml_service.py`. It returns tomato, healthy, 0.93 for any image. Run `uv run pytest` in `api/` to confirm nothing else broke.

Where feedback goes

Corrections from members

Members can mark a result as wrong and say what it really was. Export those corrections as CSV with `python -m app.tools.export_feedback` (scan id, predicted and corrected labels, image key) to see where the model is weak. Scan images are not used for training unless the team decides so and updates the Privacy Policy.

SECTION 14

SECURITY AND VERIFICATION

What protects accounts and uploads, the speed targets the site must meet, how the work is proven, and what is still open.

Security checklist

Area	Control
Cookies	httpOnly, Secure in production with the <code>__Host-</code> prefix, SameSite=Lax (Strict would drop the session from email links), Path <code>/</code> . <code>leafy_at</code> lasts 900 s, <code>leafy_rt</code> 30 days.
Request origin	Origin check (<code>assertSameOrigin</code>) on every state changing Next handler. <code>sanitizeNext</code> accepts only paths that start with <code>/</code> and not <code>//</code> .
Network exposure	FastAPI is not published in production. It trusts <code>X-Forwarded-For</code> only from the compose network.
Passwords	argon2id, with a dummy hash check on unknown email so timing does not reveal accounts. Length 10 to 128, not the email, not in a common list.
Tokens	Refresh and email tokens are stored hashed (sha256). Refresh rotation with reuse detection revokes the whole family. Email tokens are single use with atomic consume.
Uniform errors	One message for wrong credentials, one for bad email links, and forgot password is safe against enumeration (always 200).
Uploads	8 MiB cap enforced at Next, in ASGI middleware and in the streaming read. Magic byte sniff, Pillow decode, 40M pixel cap, EXIF and GPS stripped by re-encoding, server generated keys. Long edge capped at 2048 px and re-encoded as JPEG; the model receives this sanitized JPEG.
Rate limits	Per ip, email and user limits (slowapi in memory for v1).
Storage	Presigned URLs signed against <code>S3_PUBLIC_ENDPOINT</code> . Private scan bucket links last 10 minutes.
Browser headers	CSP with nonce, HSTS, <code>frame-ancestors 'none'</code> , nosniff. Token pages send <code>Referrer-Policy: no-referrer</code> .
Google sign in	PKCE, state and nonce. Exact redirect URI allowlist. ID token checked server side (signature via cached JWKS, <code>iss</code> , <code>aud</code> , <code>exp</code> , <code>nonce</code> , <code>email_verified</code>). Secret only on the API. OAuth cookie single use and 10 minutes long.
Data access	Every scan query is scoped by <code>user_id</code> . Another user's scan answers 404.
Logging	No secrets or tokens in logs. Static error messages, never <code>str(exc)</code> . Audit log line for Google link and create events.
Supply chain	<code>pip-audit</code> , <code>pnpm audit</code> and a gitleaks secrets scan in CI. No third party analytics or trackers.

Performance targets

Measured on mobile with throttling on the four main routes.

Metric	Target	Tool
Lighthouse Performance, Accessibility, Best Practices, SEO	90 or more (aim 95 or more)	Lighthouse CI
Largest Contentful Paint (LCP)	2.5 s or less	Lighthouse CI, PageSpeed Insights
Interaction to Next Paint (INP)	200 ms or less	Lighthouse CI, field data
Cumulative Layout Shift (CLS)	0.1 or less	Lighthouse CI
Total Blocking Time (TBT)	200 ms or less	Lighthouse CI
JavaScript per route	Under about 170 KB gzipped before the lazy 3D chunk	Bundle budget file
3D models	Under 300 KB, Draco or meshopt, mobile budget 100k triangles	gltf-transform pipeline
Splash screen	Shown 0.9 s to 2.2 s, no effect on LCP or CLS	Playwright

How we get there

Area	Approach
Rendering	Server components and ISR for the public handbook and landing. Route level code splitting. No heavy libraries on public pages (tree shaken drei imports, framer-motion LazyMotion).
Fonts	next/font self hosted and preloaded: 3 families, limited weights, display: swap , adjustFontFallback .
Images	next/image for static imagery (AVIF and WebP, sized, priority only on the LCP image). Catalog images pre-resized at seed time.
3D	Three.js loads after idle by dynamic import, with a static poster as the LCP candidate. One canvas per page, frameLoop="demand" , DPR cap. Models are Draco or meshopt compressed under 300 KB.
Caching and network	Brotli or gzip. Cache-Control immutable for hashed assets and 3D models. preconnect to the S3 public host.
Layout stability	Explicit sizes, reserved panel heights, and content-visibility: auto for offscreen panels.
Third party scripts	None.

API speed

Measure	Approach
Queries	Indexed keyset queries. Async I/O everywhere. Thin JSON payloads.
Disease detail	A single query with eager loading, so there is no N+1. This fixes DAHON's one request per disease pattern.
Catalog endpoints	Public catalog endpoints cached with <code>ETag</code> and <code>Cache-Control</code> , plus Next ISR.
Images	Presigned URLs instead of proxying image bytes through the API.

How we measure

Tool	Use
<code>lighthousec.json</code>	Lighthouse CI assertions on the targets above, mobile and desktop, against a production build (<code>next build && next start</code>) in CI and locally.
PageSpeed Insights	Lab data plus real user CrUX field data on the public staging or production URL. It needs a public URL, so locally Lighthouse CI is used.
Bundle budget file	Bundle analyzer with size limits, plus <code>useReportWebVitals</code> logging in development.

Verification plan

Check	Pass condition
Documentation PDF	Passes the overlap and clipping checks and the page by page visual review. Bookmarks, links and embedded fonts work. Matches the plan section by section.
Compose stack	<code>docker compose up</code> : postgres, minio and mailpit healthy. Migrations and <code>python -m app.seeds --check</code> pass (13 plants, 27 diseases).
API tests	<code>pytest</code> green with 80 percent or more coverage, including ownership (user B gets 404), refresh rotation and reuse, and account deletion while S3 is down.
ML contract	With a fake ML service, <code>POST /scans</code> stores the image in MinIO and returns disease details. The stub returns 503.
Browser flow	Register, verify via Mailpit, upload, result panels, history, delete, logout. A signed out <code>/dashboard</code> redirects to <code>/login</code> .
E2E, layout and speed	Playwright passes at 360, 768 and 1440 widths. The splash shows on a hard refresh, disappears within 2.2 s, and does not show on client navigation or in Lighthouse runs. Speed: Lighthouse CI meets the thresholds on the four main routes, and PageSpeed Insights on the deployed URL confirms (both wait for hosting).
Google tests	Run against a fake Google (<code>GOOGLE_MOCK=1</code> , ID tokens signed with a test key): new user, existing user link, unverified email rejected, wrong <code>aud</code> , <code>iss</code> or <code>nonce</code> , expired token, replayed code, state mismatch, Google only user password login fails uniformly, set password for a Google only user, delete account without a password.

Public extras and engineering basics

Additions from the completeness review. All are built except three items: the install prompt, a Lighthouse job in CI, and the Dockerfiles.

SEO on public pages

Item	Plan
Metadata	Per page <code>generateMetadata</code> : title, description, canonical, Open Graph and Twitter cards using the OG image.
<code>sitemap.xml</code>	Generated from the catalog.
<code>robots.txt</code>	Disallows private routes, <code>/brand</code> and the API.
JSON-LD	<code>WebSite</code> , plus <code>Article</code> or <code>FAQPage</code> where it fits.
Structure	One h1 per page, descriptive alt text on every plant and disease image, readable slugs. Supports the Lighthouse SEO target.

Installable PWA (no offline)

Item	Plan
Manifest	<code>manifest.webmanifest</code> named Leafy, theme and background colours from the tokens, standalone display.
Icons	Maskable 192 and 512 icons.
Shortcuts	“Scan a leaf” and “Handbook”.
Meta tags	Correct <code>theme-color</code> per scheme and iOS meta tags.
Install prompt	An unobtrusive “Install Leafy” prompt only after a successful scan. Not built yet.
Service worker	None that caches user data.

Theme flash and share image

Item	Plan
No flash of wrong theme	A tiny inline script in <code><head></code> sets <code>data-theme</code> from the saved choice or the system preference before first paint. The toggle persists in <code>localStorage</code> and a cookie, so the server can render the right theme.
OG and Twitter image	1200 by 630, dark hero with the mark, in <code>app/public/og/</code> .

Browser and device support

Item	Plan
Browsers	Evergreen, last 2 versions of Chrome, Edge, Firefox and Safari, including iOS Safari 16 and later.
Fallbacks	<code>text-wrap: pretty</code> is progressive and works without. The SVG cursor falls back to the native cursor. No WebGL or low power gives the static poster. <code>dvh</code> falls back to <code>vh</code> .
Breakpoints	360, 480, 768, 1024, 1440 and 1920. The design is checked at 360, 768 and 1440.

Testing 3D

Item	Plan
WebGL in CI	Runs with software rendering (SwiftShader).
Component tests	Mock React Three Fiber.
Playwright	Asserts the fallback path (<code>?nowebgl</code>) and one real canvas render per scene.

Engineering basics

Item	Plan
CI	<code>.github/workflows/ci.yml</code> runs ruff, pytest (with a Postgres service), <code>pnpm lint</code> , <code>typecheck</code> , <code>test</code> and <code>build</code> , and a gitleaks secrets scan. Lighthouse CI is configured in <code>app/lighthouse.rc.json</code> but is not a CI job yet.
Local hooks and docs	<code>pre-commit</code> hooks, <code>CONTRIBUTING.md</code> , and a README with a 5 command quickstart, env table, folder map, Google setup steps and the ML integration pointer.
Logging and health	Structured JSON logs with request ids. <code>/healthz</code> and <code>/readyz</code> .
Analytics and errors	No third party analytics or trackers by default. Error reporting hook point (Sentry or similar) is an optional env flag, off by default.
Content upkeep	Handbook text lives in the committed seed JSON, reviewed by the team. Editing it means changing the JSON and re-running the idempotent seed. Image alt text and short display names for long disease titles are seed fields, so the title rule is enforced at the source.

Risks

Risk	Mitigation
Slug contract drift between the model and the catalog	<code>labels.json</code> is the contract, and the resolver degrades to <code>unknown</code> .
Refresh races across several instances	10 s grace window on the API plus single flight refresh in Next.
Next 16 renames <code>middleware.ts</code> to <code>proxy.ts</code>	Pin Next.js 15 and keep the logic in <code>lib/auth</code> .
Presigned host mismatch	A separate presign client signed against <code>S3_PUBLIC_ENDPOINT</code> .
In memory rate limits reset on restart	Accepted for v1. Redis is the upgrade path.
Heavy 3D on low end devices	Static poster fallback, triangle budgets, one canvas per page.

Open items

Item	Owner and next step
Real <code>predict()</code> implementation	Teammate. The only missing piece of the app. Plug in with <code>ML_SERVICE=pkg.mod:Class</code> using <code>docs/ML_INTEGRATION.md</code> (see the ML integration section).
Google OAuth client id and secret	Team creates the client in Google Cloud Console. Until then the Google mock is used.
Sample image licensing and corrupted blobs in the DAHON dump	Team reviews before a public deploy. Unreadable blobs are skipped by the extraction tool.
Privacy Policy and Terms drafts	Team and a legal check. Drafts are not legal advice.
Tagline choice	Pick one at the brand review: "Know your leaf.", "Read the leaf. Save the plant." or "Plant health, understood."
Feedback export and training use	Only on completed scans. Export for the ML teammate with <code>python -m app.tools.export_feedback</code> (CSV). Scan images are not used for training unless the team decides so and updates the Privacy Policy.
Review <code>labels.json</code> and <code>slug_overrides.yaml</code>	Teammate, before writing <code>predict()</code> . Slugs are per plant. Raspberry is not seeded, so a raspberry prediction resolves to <code>unknown</code> .
Joel's dataset and first push	Swap the dataset into the seed format when ready. The first push to GitHub waits for the team's go ahead.
Hosting, Dockerfiles, placeholders	Hosting is undecided. Dockerfiles for <code>api</code> and <code>app</code> follow it, then PageSpeed Insights runs. The legal email and team names in <code>site-config.ts</code> show as TODO markers until replaced.

Glossary

Term	Meaning
Access token	Short lived (15 min) JWT sent as a bearer token to the API.
Refresh token	Opaque token valid 30 days that is replaced on every use. A family lasts at most 90 days.
Reuse detection	Replaying an already rotated refresh token after the grace window revokes the whole family.
PKCE	Proof key for code exchange. A secret created per sign in that stops a stolen authorization code from being used.
Nonce	A one time value tied to the Google ID token so it cannot be replayed.
Envelope	The fixed JSON shape <code>{success, data, error}</code> of every API response.
Slug	A lowercase URL name for a plant or disease, such as <code>early-blight</code> .
Verdict	The scan outcome: <code>disease</code> , <code>healthy</code> or <code>unknown</code> .
Stage	The step of a processing scan: <code>validating</code> , <code>analyzing</code> or <code>saving</code> .
Keyset cursor	Paging by the last seen <code>(created_at, id)</code> instead of an offset, so deep pages stay fast.
Presigned URL	A temporary signed link that lets the browser read one stored image directly.
Outbox	The <code>storage_deletion</code> table that records image keys to delete, so deletes survive an S3 outage.
ISR	Incremental static regeneration: pages built ahead and refreshed on a timer (1 hour for the handbook).
Wave and gate	A wave is a batch of parallel work. A gate is the set of checks that must pass before the next wave.
LCP, INP, CLS	Web vitals for load speed, responsiveness and layout stability.
DAHON	The original prototype. The word means leaf in Filipino. It is credited as the origin, not used as the brand.